

Performance web

PHP / Laravel / PostgreSQL / Angular JS

Ali Koudri - ali.koudri@janus-academy.fr

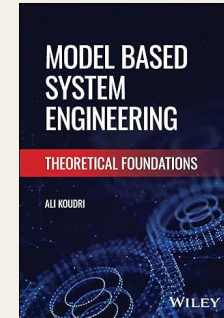
Votre formateur

Ali Koudri

- PhD en informatique
- 25+ ans en ingénierie système et logicielle, modélisation, performance, sécurité
- IRT SystemX - Thales - CEA
- Janus Academy : formation & conseil indépendant
- Approche cross-stack : front, back, ci/cd, observabilité

Publications

Model Based Systems Engineering: Theoretical Foundations



Posture pédagogique

Démarche systémique, mesure avant optimisation, capitalisation transposable

Le projet fil rouge : Application Resonance

Une application prod-like, volontairement non optimisée, pour mesurer l'effet de chaque levier

Nuxt 3

Frontend SSR/CSR

TypeScript

Laravel

API REST / GraphQL

Queues, jobs, cache

PostgreSQL

Base relationnelle

Index, plans, stats

Sept branches

- main : baseline prod-like, perf volontairement dégradée
- solution-cdn - solution-bundle - solution-dashboard - solution-laravel - solution-postgres
- final : version optimisée intégrale, cible de la formation

Le déroulé sur 3 jours

Un module de cadrage, cinq modules théorie-lab-debrief, un module de synthèse

J1 matin

Cadrage

J1 après-midi

Module 1 - CDN & cache

J2 matin

Module 2 - Bundle frontend

J2 après-midi

Module 3 - Dashboard & refactor Vue

J3 matin

Module 4 - Laravel & backend

J3 après-midi

Module 5 - PostgreSQL

J3 fin

Synthèse & transposition

Méthode de travail

Cycles courts théorie → pratique → debrief, répétés 5 fois

1. Théorie

25-30 min

Concepts, vocabulaire, patterns et anti-patterns. Ce que la mesure va valider.

2. Lab

60-90 min

Branche solution, fiche markdown, modifications guidées, mesures à chaque étape.

3. Debrief

15-20 min

Confrontation des chiffres, capitalisation, transposition à votre propre stack.

Les chiffres sont mesurés sur vos machines, en live. Vous repartez avec un repo opérationnel pour vos propres projets.

Partie 1

Pourquoi la performance ?

Au-delà des slogans, ce qui change vraiment sur le terrain

La performance n'est pas une feature

C'est une propriété systémique émergente, ce que la chaîne entière conditionne



Chaque maillon peut affaiblir la chaîne.

Question: De quelle manière ces différents maillons peuvent être affaiblis ?

Aucune optimisation isolée ne peut compenser un autre maillon défaillant.

Corollaire pédagogique : Il est essentielle de garder une vue holistique de la performance

Les 5 maillons s'affaiblissent - comment

Chaque maillon a ses pathologies typiques. Diagnostiquer = identifier le maillon, puis la cause

1

Navigateur

Le code client lourd

JS bundle massif
→ longtasks > 50 ms

DOM non virtualisé
→ reflows coûteux

Listeners orphelins
→ memory leaks

2

Réseau

La distance physique

Latence (RTT)
→ utilisateur lointain

Bande passante
→ mobile 3G/4G dégradé

Multi-RTT TLS
→ handshake bavard

3

Edge / CDN

Le cache distribué

Cache miss
→ TTL trop court

Géo-distribution
→ pas d'edge proche

Invalidation cassée
→ stale contenu chaud

4

Backend

Le serveur applicatif

Code lent
→ boucles, locks, sync I/O

Workers saturés
→ queue qui s'allonge

Cold start
→ FPM qui réinitialise

5

Base de données

Le fond de la pile

Requêtes non indexées
→ full scan

N+1
→ multiplication de queries

Connexions saturées
→ pas de pooling

Une perf qui se dégrade n'est jamais « aléatoire », c'est un maillon précis qui a faibli pour une raison précise.

Les pénalités d'une mauvaise performance

4 coûts directs, souvent mesurés séparément, mais en réalité couplés

UX & conversion

Au-delà d'1 sec, l'attention décroche. Chaque seconde coûte une fraction du tunnel de conversion.

SEO

Core Web Vitals = signal de classement Google depuis 2021. INP remplace FID en mars 2024.

Coût d'infra

Un code lent compensé par du sur-dimensionnement : autoscale, instances, cache. Le coût grimpe avec le trafic.

Empreinte CO₂

kWh par requête, équipements client surchargés, électricité réseau. La perf est un levier **GreenOps** direct.

Quelques ordres de grandeur

Études récentes : la magnitude compte plus que la décimale exacte

53%

Utilisateurs mobiles
abandonnant un site > 3s
(Google Mobile UX studies)

+10%

Conversion moyenne
par 100 ms gagnés
(Deloitte Digital, retail)

60%

Pages échouant aujourd'hui aux
Core Web Vitals
(Chrome UX Report agrégé)

1.8g

CO₂eq par page vue moyenne
(Website Carbon, moyenne
mondiale)

Note : les chiffres bougent. Ce qui compte, c'est l'ordre de grandeur, pas la décimale.

Le piège du « ça marche en dev »

Trois mondes très différents - votre poste de dev est le moins représentatif des trois

Dev local

- CPU M2 / Ryzen 9
- Latence < 1ms
- Cache navigateur chaud
- Données réduites
- Pas de couches edge

Staging

- Specs proches prod
- Réseau LAN ou VPN
- Données partielles
- Pas de vraie charge
- Quelques edge cases

Prod / RUM

- Téléphones milieu de gamme
- Réseaux 4G/5G hétérogènes
- Cache froid fréquent
- Volume réel, queues longues
- Distance géographique

« Ça marche sur le poste du dev » n'est pas une mesure, c'est une absence de mesure.

Le piège du score Lighthouse 100

Un audit synthetic n'est pas un constat de terrain - c'est un signal parmi d'autres

Ce que Lighthouse mesure

- Une seule page, ouverte une seule fois
- Machine puissante, dans un datacenter proche
- Réseau simulé fixe, conditions standardisées
- Cache initial vide, mais déterministe
- Snapshot : pas la vie réelle de la page

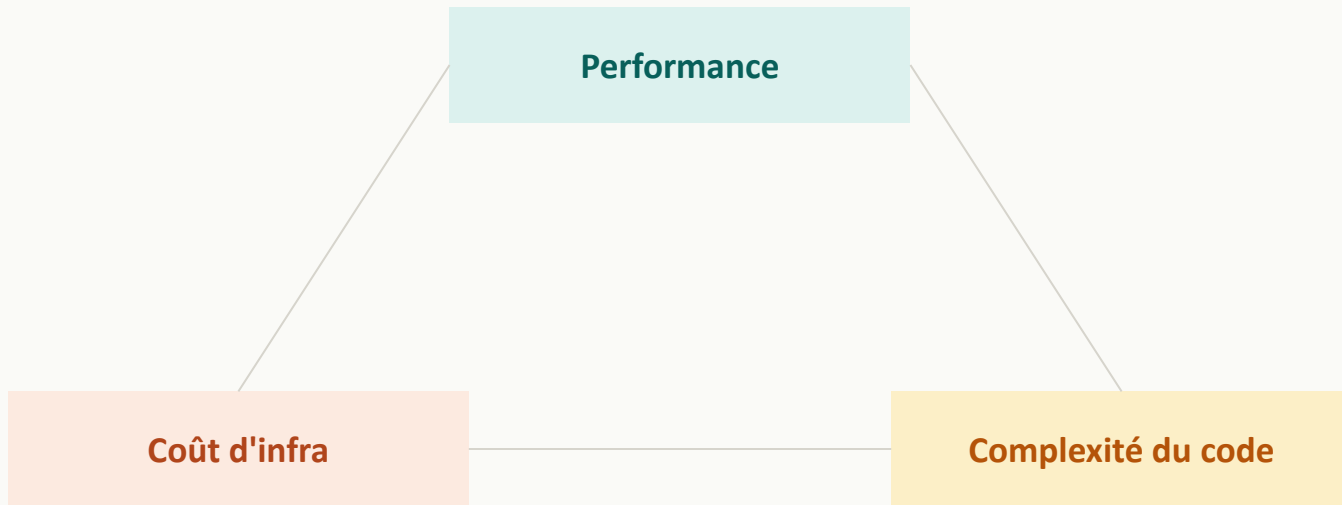
Ce que vos utilisateurs vivent

- Multiples pages, parcours imbriqués
- Mobiles milieu de gamme sur 4G fluctuant
- Zones géographiques distantes
- État applicatif, sessions, A/B tests
- Distribution : pas juste un point

Lighthouse vert + RUM rouge = situation très courante. Le score lighthouse n'est qu'une étape vers l'objectif final.

Une question de compromis

Toute optimisation déplace au moins une des trois variables



Le travail de l'architecte : rendre l'arbitrage explicite. L'objectif n'est jamais le maximum, il n'existe pas

Partie 2

Petite histoire de la performance web

Trente ans de mesures, de patterns et de standards

1995 - 2005 : L'âge de la bande passante

On compresse, on découpe, on minimise les requêtes. Les réflexes posés à cette époque sont toujours valides.

- 1995** Premiers sites publics. Le 56k est la norme. Optimisation = poids des images.
- 1999** GIF / JPEG compression agressive, sprites CSS, tableaux pour la mise en page.
- 2004** Yahoo crée YSlow. Steve Souders quantifie : 80% du temps se passe côté client, pas serveur.
- 2007** Souders publie « High Performance Web Sites ». 14 règles qui structurent le champ pendant une décennie.

Réflexes hérités, toujours valides en 2026

Minimiser le nombre de requêtes - compresser les ressources - mettre en cache agressivement - reporter ce qui n'est pas critique

Les 14 règles de Steve Souders

« High Performance Web Sites » (2007) - quatre familles, encore lisibles aujourd'hui

1	Réduire les requêtes HTTP <i>80 % du temps est client - sprites, Data URIs</i>	8	JS / CSS externalisés <i>Bénéficier du cache sur pages suivantes</i>
2	Utiliser un CDN <i>Distance physique = latence réseau</i>	9	Réduire les recherches DNS <i>Limiter le nombre de domaines hôtes</i>
3	Cache-Control / Expires <i>Forcer le cache local sur les ressources statiques</i>	10	Minifier JS / CSS <i>Supprimer espaces, commentaires, inutiles</i>
4	Compression (gzip) <i>Réduire le poids HTML / CSS / JS</i>	11	Éviter les redirections <i>Chaque 301 / 302 ajoute un aller-retour</i>
5	CSS dans le <head> <i>Évite le FOUC, rendering progressif</i>	12	Pas de scripts dupliqués <i>Bibliothèque incluse deux fois par erreur</i>
6	JS avant le </body> <i>Le téléchargement JS bloque le rendu</i>	13	Configurer les ETags <i>Validation conditionnelle - gare au cluster</i>
7	Pas d'expressions CSS <i>Recalcul à chaque tick - figeait IE</i>	14	Cache des réponses AJAX <i>Cacher aussi les appels asynchrones</i>

 Réseau  Cache  Structure  Code

Quatre familles, posées en 2007 sur la base d'observations du temps réel sur des sites du Top 10 US.

Souders à l'aube de 2026 - ce qui survit, ce qui mute

HTTP/2, bundlers, immutable, AVIF redessinent la prescription, pas le diagnostic

Restées vraies

Les fondamentaux network / cache

- 2 - CDN
- 4 - Compression (brotli > gzip aujourd'hui)
- 5 - CSS critique en haut
- 10 - Minification (souvent automatique)
- 11 - Éviter les redirections
- 14 - Cache des réponses AJAX

À relire en 2026

HTTP/2, bundlers, immutable changent la prescription

- 1 - Multiplexing HTTP/2 rend sprites / concat obsolètes
- 3 + 13 - immutable + fingerprint surclassent Expires + ETags
- 6 - defer / async / type=module plutôt que la position
- 8 - inline du CSS critique revient pour le LCP
- 9 - connection coalescing + preconnect
- 12 - géré par les bundlers modernes

Anti-patterns 2026

Des recettes utiles devenues nuisibles

- 7 - Expressions CSS : éteintes avec IE
- Sprites CSS : HTTP/2 rend l'agrégation contre-productive
- Sharding multi-domaines : casse la connection coalescing
- Cookieless domains pour les statiques : surcouche inutile

Souders avait raison sur le diagnostic, pas toujours sur la prescription. Les principes survivent, les patterns mutent.

2005 - 2015 : Mesure et industrialisation

Naissance du RUM, premiers outils synthetic ouverts, standards modernes du transport

2008	WebPageTest	Patrick Meenan publie l'outil de référence du synthetic ouvert.
2009	Boomerang	Yahoo open-source la première librairie de RUM. Le terrain devient observable.
2010	Mobile-first	L'iPhone et les Android low-end imposent de penser la performance différemment.
2015	HTTP/2	Multiplexage, server push, header compression. Plus besoin de sprites.

Bascule conceptuelle : on cesse de deviner, on commence à mesurer en continu

Le monitoring synthétique

Simuler le parcours utilisateur dans un environnement contrôlé : la mesure de référence du lab

Principe

Scripts + navigateurs automatisés, exécutés régulièrement depuis des environnements contrôlés. Contrairement au RUM, le synthetic est aveugle aux vrais utilisateurs; c'est aussi ce qui en fait un instrument de mesure reproductible.

Baseline

Le point de départ chiffré

Branche main, conditions standard, premier passage. « Ma home charge en 16 s sur mobile 4G ralenti. » La photo avant toute optimisation.

Mesurer le gain

Quantifier l'effet du changement

Mêmes conditions, même protocole, mêmes outils. L'amélioration mesurée est imputable au code, pas à l'environnement, pas au bruit.

Détecter les régressions

La sentinelle CI

Lighthouse CI ou WebPageTest sur PR. Voir tomber un score avant la prod, pas après la plainte du premier utilisateur.

C'est exactement le protocole de chaque module Resonance : starter → modif → re-mesure. Sans baseline figée, pas de progression mesurable.

2015 - 2020 : Codification & outillage Google

Google formalise des modèles mentaux et industrialise l'audit. La perf devient un langage commun.

2015	Modèle RAIL	Response < 100ms, Animation 60fps, Idle, Load. Premier modèle pédagogique unifié.
2017	Lighthouse	Audit synthetic intégré à Chrome. Tout le monde a le même outil, partout.
2018	Pattern PRPL	Push, Render, Pre-cache, Lazy-load. Architecture web orientée perf.
2019	web-vitals.js	Librairie officielle Google pour instrumenter les CWV côté client.
2020	HTTP/3 + QUIC	Transport sur UDP, gestion native de la perte. Mobiles dégradés mieux servis.

Modèle RAIL

Cadre de performance centré utilisateur - Google 2015, ancêtre direct des Core Web Vitals



Response

< 100 ms

Réagir à toute interaction - clic, tap. Au-delà, le décalage devient perceptible.



Animation

60 FPS

Maintenir la fluidité du scroll et des transitions. ~ 16 ms par frame disponibles.



Idle

< 50 ms

Découper les tâches de fond. Rester prêt à répondre à toute interaction utilisateur.



Load

< 5 s

Premier rendu interactif sur mobile 3G. < 2 s sur les chargements suivants.

RAIL a posé le langage perçu de la performance. INP, LCP, CLS en sont les héritiers directs - mêmes phases, métriques précisées.

QUIC : le transport qui propulse HTTP/3

Repenser TCP depuis UDP, conçu Google, standard IETF

QUIC en une phrase

Quick UDP Internet Connections : un transport conçu pour le web mobile moderne, qui se débarrasse du bagage TCP des années 70 sans renoncer à la fiabilité.

Transport UDP

Léger. Pas de handshake bavard. La fiabilité est réimplémentée au niveau protocole, sans l'héritage TCP.

TLS 1.3 natif

Chiffrement intégré. 1-RTT à la première connexion, 0-RTT au retour : les données utiles partent immédiatement.

Streams indépendants

La perte d'un paquet image n'interrompt pas les autres téléchargements. Fin du head-of-line blocking TCP.

Connection ID

Identifiant de connexion stable. Le téléchargement survit au passage WiFi → 4G : gain massif sur mobile.

HTTP/3 est activé chez la plupart des CDN aujourd'hui. Côté code : rien à faire. Côté infra : une case à cocher.

Le coût caché d'une connexion HTTPS

Avant le premier byte utile, plusieurs aller-retours se jouent - TCP, puis TLS, puis la requête

TCP + TLS 1.2

Le standard historique

1 RTT TCP handshake
SYN - SYN-ACK - ACK

2 RTT TLS 1.2 handshake
Hello - Cert - Finished

+ Requête

Total : 3 RTT

À 100 ms de latence
→ 300 ms avant le 1er byte

TCP + TLS 1.3

Réduction notable

1 RTT TCP handshake

1 RTT TLS 1.3 (1-RTT)
ou 0-RTT en reprise

+ Requête

Total : 2 RTT

À 100 ms de latence
→ 200 ms avant le 1er byte

QUIC + TLS 1.3

Le collapse complet

1 RTT QUIC handshake
TLS 1.3 intégré
0-RTT en reprise

+ Requête immédiate

Total : 1 RTT (0 en reprise)

À 100 ms de latence
→ 100 ms, ou 0 ms

0-RTT et QUIC ne sont pas des optimisations marginales - ils suppriment 100 à 300 ms par première connexion.

2020 - 2026 : Core Web Vitals et au-delà

La perf devient critère de classement Google, puis se raffine avec l'arrivée d'INP

2020	Annonce CWV	Google annonce que les Core Web Vitals deviendront le signal de classement.
2021	CWV en ranking	Effective sur mobile en juin, sur desktop en février 2022. Le SEO devient perf-aware.
2024	INP remplace FID	Mars 2024. La nouvelle métrique d'interactivité couvre tout le cycle d'une interaction.
2024-25	Speculation Rules, View Transitions	Prefetch déclaratif, transitions natives. La perf perçue redevient un sujet d'API (Navigateur).
2025-26	Soft Navigations	Métrique expérimentale pour SPA. Le débat « perf SPA = perf MPA » est relancé.

SPA vs MPA - le débat relancé

De nouvelles APIs natives donnent aux MPA les capacités qui définissent les SPA

Quinze ans de SPA

Pendant longtemps, fluidité perçue = SPA. Pas d'écran blanc entre les pages, transitions animées, état préservé. Le navigateur a fini par combler l'écart côté MPA.

View Transitions API

Animations entre pages HTML

Fondu, slide, morph entre deux pages distinctes. Le browser gère et l'avantage esthétique SPA disparaît.

Speculation Rules API

Pré-rendu déclaratif

La page suivante est rendue en arrière-plan avant le clic. Affichage instantané, perception égale ou supérieure à la SPA.

Turbo / HTMX

Fragments HTML, pas de framework lourd

Mise à jour partielle du DOM par le serveur. Réactivité SPA, légèreté MPA.

Le débat n'est plus « quelle techno est la plus rapide », mais « comment rendre n'importe quelle archi instantanée ».

Partie 3

Mesurer la performance

Quoi mesurer, comment, et pourquoi la moyenne ment

Le cycle de vie d'une page

Du clic à l'interactivité : chaque métrique appartient à une phase de ce cycle



CLS

Cumulative Layout Shift : accumulé sur l'ensemble du rendu et des premières interactions. La seule métrique « transverse » du cycle.

Diagnostiquer une mauvaise métrique = remonter à la phase incriminée. Pas l'inverse.

Le critical rendering path

Zoom sur la phase 4 : d'où sortent FCP, LCP, et la moitié des CLS

1

Ressources

HTML, CSS, JS
téléchargés
en parallèle

2

DOM + CSSOM

Modèles d'objet
du document
et du style

3

Render Tree

Fusion DOM + CSSOM
(éléments visibles
uniquement)

4

Layout

Calcul positions
et dimensions
(reflow possible)

5

Paint

Pixels peints
→ FCP, LCP émis
→ CLS si reflow



CSS render-blocking

Tout CSS dans <head> bloque le Render Tree : le browser attend que le CSS soit téléchargé et parsé avant de peindre.



JS parser-blocking

Un <script> sans defer / async interrompt la construction du DOM : le parsing HTML s'arrête jusqu'à exécution complète.



Polices et images sans dimensions

Fonts sans font-display: swap → FOIT. Images sans width/height → reflow au load → CLS qui s'aggrave.

Tout ce qui retarde le Paint retarde le FCP. Tout ce qui décale le Paint retarde le LCP. Tout ce qui repaint déplace le CLS.

Deux mondes : Lab et Field

Complémentaires, pas substituables. Avoir un seul des deux = être aveugle d'un œil.

Lab - Synthetic

Conditions contrôlées, reproductibles

- Détection précoce dans la CI
- Diagnostic, profiling, expérimentation
- Reproductibilité par construction
- Une page, un parcours, une fois
- Ne reflète pas vos utilisateurs

Field - RUM

Mesure sur la vraie distribution d'utilisateurs

- Vérité opérationnelle, suivi SLO
- Détection des cohortes en souffrance
- Validation des optimisations en prod
- Volume nécessaire, donc sampling
- Causalité difficile à isoler

Règle de cohérence : ce qui passe en lab doit être confirmé en field. Et inversement.

Synthetic vs RUM : Comparatif

Mêmes objectifs en surface, propriétés très différentes en profondeur

Axe	Synthetic / Lab	RUM / Field
Reproductibilité	Forte : variables contrôlées	Faible : distribution réelle
Représentativité	Faible : un cas standard	Forte : vos utilisateurs réels
Granularité	Page, étape, frame	Page, session, parcours
Coût opérationnel	Pipeline, runners, durée CI	Volume, sampling, cardinalité
Cas d'usage principal	Diagnostic, CI, expérimentation	Suivi SLO, alerting, cohortes

Sur le terrain : Lighthouse CI sur PR, RUM (Sentry, Datadog RUM) en prod ; alertes sur les deux.

Les trois piliers de l'expérience perçue

Loading, interactivity, visual stability, chacun avec sa CWV et ses métriques de diagnostic

Loading

LCP

Largest Contentful Paint
Diagnostic : TTFB - FCP

Vitesse perçue de l'arrivée du contenu utile

"Est-ce que c'est là ?"

Interactivity

INP

Interaction to Next Paint
Diagnostic : TBT - FID (legacy)

Latence ressentie quand l'utilisateur agit

"Est-ce que ça répond ?"

Visual stability

CLS

Cumulative Layout Shift
Diagnostic : N/A

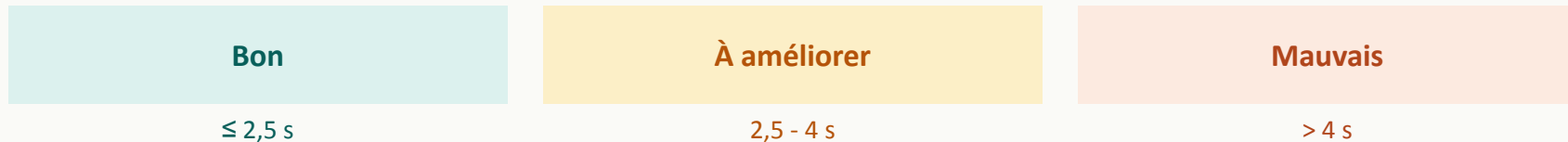
Stabilité visuelle pendant le chargement

"Est-ce que c'est stable ?"

LCP - Largest Contentful Paint

Moment où le plus grand élément visible est rendu - la perception du « c'est arrivé »

Seuils Google



Causes typiques

- TTFB (Time To First Byte) lent : backend ou edge mal cachés
- Ressources bloquantes en <head>
- Images LCP non priorisées, format inadéquat
- Web fonts en FOIT (Flash of Invisible Text), FOUT (Flash of Unstyled Text) mal géré

INP - Interaction to Next Paint

Depuis mars 2024, INP remplace FID : la métrique d'interactivité qui couvre tout le cycle

Définition

Pire latence d'interaction observée sur la durée de vie de la page = input delay + processing + rendering

Seuils Google

Bon

≤ 200 ms

À améliorer

200 - 500 ms

Mauvais

> 500 ms

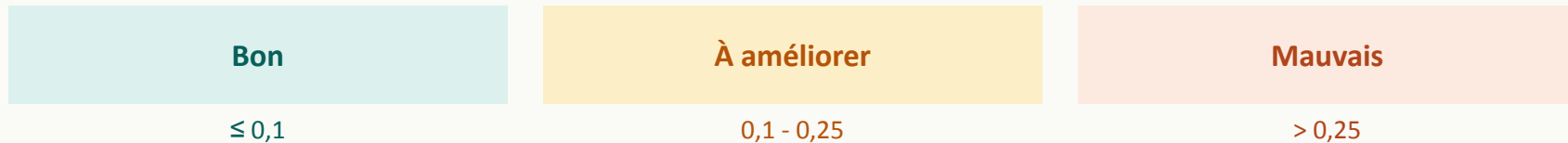
Causes typiques

- Long tasks JS sur le main thread (> 50ms)
- Hydratation coûteuse au mount (SSR/CSR boundary)
- Listeners synchrones lourds, layout thrashing dans les handlers

CLS - Cumulative Layout Shift

Somme des sauts de mise en page non attendus - le grain de sable dans l'œil

Seuils Google



Causes typiques

- Images et iframes sans width/height ou aspect-ratio
- Web fonts qui re-flow (FOUT) sans size-adjust
- Bannières d'ads ou de consentement insérées tardivement
- Contenu injecté au-dessus du viewport actuel

Les métriques de diagnostic

Elles n'apparaissent pas dans Search Console, mais c'est elles qui vous disent pourquoi

TTFB

Time To First Byte. Première mesure du backend + edge. Idéalement < 600ms.

FCP

First Contentful Paint. Premier pixel utile rendu. Diagnostic de blocage critique.

TBT

Total Blocking Time. Somme des dépassements > 50ms sur le main thread.

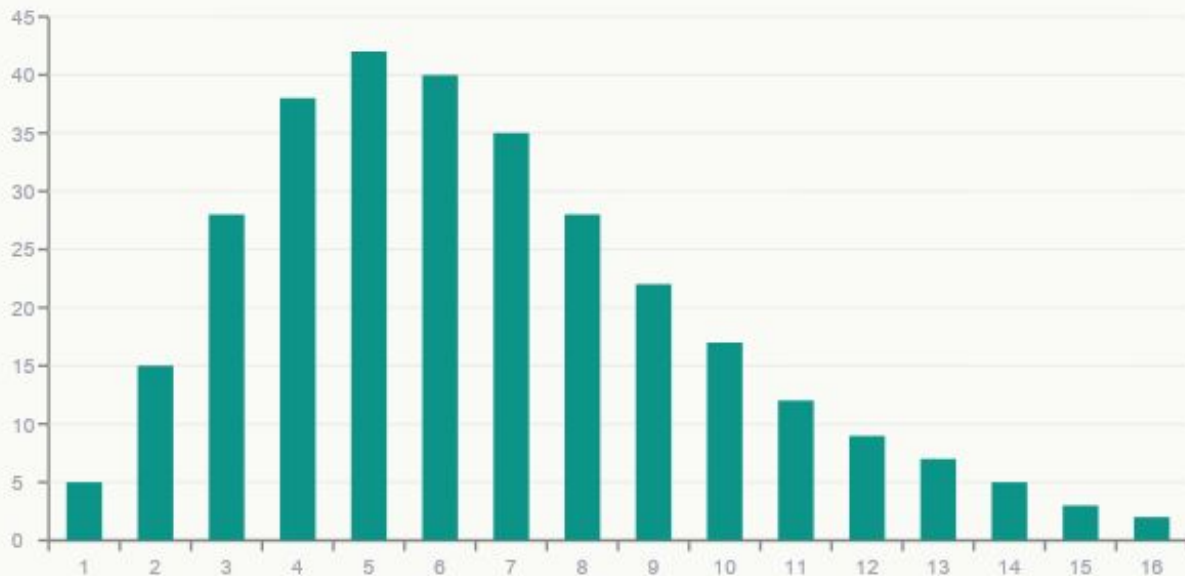
DOMContentLoaded

Fin de parsing HTML + scripts synchrones. Souvent corrélé à l'hydratation.

Diagnostic n'est pas KPI. Ces métriques expliquent les CWV ; Elles ne remplacent jamais leur suivi en p75 sur le RUM.

Pourquoi le percentile et pas la moyenne ?

La distribution de latence est log-normale, pas gaussienne : la moyenne est un faux ami



Lecture

Moyenne ≈ 1100 ms

Masque 30 % des utilisateurs
au-dessus de 1,6s

p75 ≈ 1600 ms

Le seuil Google visibilise la queue

p99 ≈ 3500 ms

Vos utilisateurs en détresse

p50, p75, p99 : ce que chacun raconte

Chaque percentile correspond à une population différente, et à un objectif différent

p50

L'utilisateur typique

Servir de baseline communicable, sans valeur d'alerte

p75

Le seuil Google CWV

Couvrir les trois quarts de la base : l'objectif de pilotage de la perf

p99

Vos utilisateurs en difficulté

Souvent les plus précieux : mobiles bas de gamme, zones rurales, sessions longues

Mesurer sans préciser le percentile, c'est mesurer faux.

Cohortes : segmenter sans perdre le signal

Un score global masque des disparités énormes : le mobile p75 peut être 3× le desktop p75

Device

Mobile / desktop, gammes de CPU, RAM

Réseau

4G, 5G, WiFi, câble, satellite : RTT et bande passante

Géographie

Distance au edge, latence transcontinentale, qualité ISP local

Type de page

Landing, détail produit, checkout, dashboard : patterns très différents

Version applicative

Détecter les régressions, comparer deux releases, valider un canary

Connecté / anonyme

Sessions chaudes vs cold start, données utilisateur, cache hit ratio

Cardinalité : l'ennemi silencieux de l'observabilité

Trop de valeurs uniques sur un label = explosion mémoire, ralentissement, factures qui montent

À éviter

- user_id, session_id, request_id en label
- URL brute avec UUIDs, slugs, ids ressources
- Timestamp précis dans un label
- Combinaisons illimitées (browser × OS × version × langue × ...)

À privilégier

- Routes paramétrées : /users/:id, /posts/:slug
- Buckets de status : 2xx, 3xx, 4xx, 5xx
- Catégorisation : device_type ∈ {mobile, desktop, tablet}
- Top-N + bucket « other » pour les longues queues

Règle empirique : cardinalité d'un label < 1000 ; cardinalité d'une métrique < 100 000.

Les quatre stratégies de sampling

À l'échelle, garder tout n'est pas possible. La stratégie choisie détermine ce qu'on voit, et ce qu'on rate

Head sampling

Décision en tête de trace

1 trace sur N est gardée dès le début, avant de savoir ce qu'elle va contenir.

Quand

Monitoring continu, volumes constants, coûts prévisibles.

Limite

Rate les erreurs rares - si l'erreur a 1 % de chance, et qu'on sample à 10 %, on en garde 0,1 %.

Tail sampling

Décision en queue de trace

La trace est bufferisée intégralement, la décision « garder ou pas » se prend à la fin, en connaissant le résultat.

Quand

Détection d'incidents, analyse post-mortem, garde tout ce qui sort de la norme.

Limite

Mémoire intermédiaire coûteuse - il faut buffer toute la trace en attendant la décision.

Probabilistic

Tirage aléatoire pondéré

Chaque trace a une probabilité fixe p d'être conservée (typique : $p = 0,01$ pour 1 %).

Quand

Baseline statistique, p50/p75/p99, suivi de tendance globale.

Limite

Ne distingue pas routine et incident - l'échantillon ressemble à la moyenne.

Rate-limited

Plafond fixe par seconde

Max N traces/seconde, peu importe le trafic réel. Au-delà, on jette.

Quand

Ingestion à coût fixe (SaaS facturé au volume), budget mensuel à tenir.

Limite

Sur un burst, on perd justement les traces qui auraient été intéressantes.

Choisir une stratégie, c'est choisir ce qu'on accepte de ne pas voir.

Sampling : Décider quoi garder

Quand le volume rend l'observabilité coûteuse, on échantillonne, mais pas n'importe comment

Head-based

Décision à l'entrée, sur l'ID de trace. Simple, déterministe, mais aveugle aux erreurs.

Tail-based

Décision après accumulation : on garde les traces lentes, erreurs, anomalies.

Error-based

Sampling adaptatif augmenté lorsque le taux d'erreurs ou la latence grimpe.

Biais à éviter : un sampling uniforme rate les rares pathologiques. Toujours garder 100 % des erreurs et des slow paths.

Côté backend - les golden signals

Quatre métriques universelles, indépendantes du langage et de la stack

Latency

Temps de réponse, par percentile. Distinguer succès et erreurs : une erreur rapide ment.

Traffic

Volume de requêtes par seconde, par endpoint, par méthode. Indicateur de charge.

Errors

Taux d'erreurs 4xx/5xx, ainsi que les erreurs métier silencieuses (200 OK suspects).

Saturation

Utilisation des ressources sous tension : CPU, RAM, pool de connexions, file d'attente.

SRE Google - 2016. Complété par la méthode USE (Brendan Gregg) côté infra : Utilization, Saturation, Errors.

Relier la perf à la valeur

La perf n'est pas une fin, elle est un driver de métriques métier mesurables

TTV - Time To Value

Premier moment où l'utilisateur obtient ce qu'il est venu chercher.
Souvent < LCP.

Conversion segmentée par perf

Groupez vos utilisateurs par latence vécue, regardez le taux de conversion par bucket.

Coût par requête - FinOps

Compute, bande passante, edge. La perf réduit le coût marginal du trafic.

CO₂ par session - GreenOps

kWh client + kWh serveur. Métrique encore jeune, mais traçable.

Partie 4

L'écosystème outillé

Cartographie d'un champ devenu mature : qui fait quoi, quand

Cartographie de l'écosystème

Quatre quadrants, chacun avec son moment, son budget, ses outils

Côté client

DevTools - Lighthouse - web-vitals - Bundle analyzers

Synthetic

WebPageTest - Lighthouse CI - k6 browser - sitespeed.io

RUM

Datadog RUM - Sentry - SpeedCurve - OTel browser - Vercel SI

Backend & infra

OpenTelemetry - Prometheus / Grafana - Blackfire - pg_stat_statements

Aucun outil ne couvre tout. La maturité = combiner intelligemment, pas accumuler.

Outils navigateur : ce qui tourne déjà sur votre poste

Tout dev senior doit maîtriser ces cinq onglets et leurs spécificités sous Chrome

DevTools - Performance

Profil CPU + main thread + frames. Identifie les long tasks et le coût d'hydratation.

DevTools - Network

Waterfall, priorités, headers, cache. Throttling pour simuler 4G, slow 3G.

DevTools - Coverage

JS et CSS chargés mais inutilisés. Premier signal de bundle gonflé.

Lighthouse

Audit synthetic local. Calibre en mobile + slow 4G. Renvoie LCP, TBT, CLS, et conseils d'amélioration.

web-vitals.js

Instrumentation JavaScript officielle Google. Mesure les CWV à émettre vers votre RUM.

Synthetic : audit en lab, en continu

Pour la CI, le diagnostic, et l'investigation reproductible

WebPageTest

Référence depuis 2008. Multi-locations, multi-devices, scriptable. Filmstrip détaillé.

Lighthouse CI

Audit Lighthouse en pipeline avec budgets et historique. Empêche les régressions.

k6 browser

Charge + perf navigateur dans un seul outil. Réutilise les scripts de tests existants.

Playwright + perf API

Mesure Web Performance native dans des scénarios E2E métier.

sitespeed.io

Auto-hébergeable, dashboard intégré. Idéal en interne pour ne pas dépendre d'un SaaS.

RUM : la mesure de la vraie vie

Vos utilisateurs, vos pages, vos parcours : la vérité de terrain

Datadog RUM

Intégré, complet, cher au volume. Session replay, alerting, parcours.

Sentry Performance

Hérite du contexte d'erreurs. Excellent pour corrélérer crash et perf dégradée.

SpeedCurve

Le SaaS spécialisé perf. Très bon pour les équipes qui pilotent par CWV.

Vercel Speed Insights / Cloudflare Web Analytics

Spécialisés CWV, intégrés à leur stack edge. Suffisent souvent.

OpenTelemetry browser SDK

Standard ouvert. Émet vers n'importe quel backend : Tempo, Honeycomb, Grafana Cloud.

Profilage Node - côté serveur JavaScript

Pour Nuxt en SSR, les middlewares, les jobs et tout ce qui tourne en V8

node --inspect + DevTools

Profiler CPU, heap snapshots, allocations. Le réflexe immédiat.

Clinic.js

Doctor (diagnostic), Flame (flamegraph), Bubbleprof (event loop). Excellent pour les débuts.

0x

Flamegraph interactif. Très pratique pour cibler les fonctions chaudes.

async_hooks / AsyncLocalStorage

Tracer du contexte personnalisé, instrumenter sans overhead démesuré.

Heap profiler V8

Détection de fuites, snapshots comparés. Indispensable sur Node long-running.

Profilage PHP / Laravel

Un écosystème mature, payant pour la prod, gratuit pour le dev

Blackfire

Le standard prod. Profiling en continu, comparaison de profils, intégration CI.

Telescope

Officiel Laravel, dev/staging. Requêtes, jobs, mails, cache, gates.

Clockwork

Dev pur, extension Chrome/Firefox. Léger, complémentaire à Telescope.

Laravel Pulse

Dashboard de prod natif Laravel (2024). Slow queries, slow requests, exceptions.

Xdebug profiler + Cachegrind

Gratuit, profilage local fin. Lent, réservé au diagnostic ciblé.

`opcache_get_status()`

État de l'OPCache. Hit ratio, mémoire, fragmentation, scripts cachés.

Bundle frontend - poids, splitting, dead code

L'enjeu est de livrer le bon code au bon moment

vite-bundle-visualizer / rollup-plugin-visualizer

Treemap interactif. Aller direct aux gros nodes_modules importés.

source-map-explorer

Cartographie issues des sourcemaps. Très précis sur l'origine du poids.

webpack-bundle-analyzer

Référence historique côté Webpack. Toujours utile sur des stacks plus anciennes.

Lighthouse - Unused JavaScript

Premier signal automatique du code mort embarqué. Souvent ~30 % en moyenne.

Import Cost (VS Code)

Visualisation inline du coût de chaque import. Posture préventive.

PostgreSQL - l'outillage essentiel

Postgres expose des observatoires riches, encore faut-il savoir lire

pg_stat_statements

Top requêtes, total time, mean time, calls. Premier réflexe pour identifier les hot paths.

EXPLAIN (ANALYZE, BUFFERS)

Plan réel d'une requête, avec IO en pages. La base du diagnostic ciblé.

auto_explain

Capture automatique des slow queries en prod, avec leur plan. Inestimable.

pg_stat_io - PG 16+

Vue détaillée des IO par backend et type. Permet de localiser le bottleneck disque.

pg_buffercache

Quel index, quelle table occupe le shared_buffers ? Diagnostic de hit ratio.

pgBadger

Analyse de logs PostgreSQL. Rapports HTML détaillés sur les patterns lents.

Observabilité backend - la stack moderne

Standard OpenTelemetry + un backend de votre choix - fini le lock-in

OpenTelemetry

Standard ouvert pour métriques, traces, logs. Instrumentation neutre, un SDK pour tout.

Prometheus + Grafana

Métriques pull. Référence open-source pour le monitoring temps réel.

Tempo / Jaeger

Stockage de traces distribuées. Tempo s'intègre nativement à Grafana.

Loki / ELK

Centralisation des logs. Loki pour l'approche label-based à la Prometheus.

OTel Collector

Pivot local entre vos applis et vos backends. Permet de changer de stack sans toucher au code.

Focus sur k6 : le test de charge orienté développeurs

Grafana Labs, open-source, scripting JavaScript, versionable comme du code applicatif

Le tournant developer-first

Là où JMeter configure des tests via XML et interface graphique, k6 demande du code. Les tests s'écrivent, se versionnent, se composent, comme n'importe quel autre artefact applicatif.

Scripting JS

Le test est du code

Tests en JavaScript, pas XML.
Versionables dans Git. Boucles, conditions, parsing de réponses, toute la logique métier exprimable.

Trois familles

Load - Stress - Soak

Load : tenir N users simultanés.
Stress : trouver le point de rupture.
Soak : fuites mémoire sur longue durée.

Observabilité

Native, temps réel

Métriques streamées vers Prometheus et Grafana. Le test alimente l'instrumentation existante, pas juste un score isolé.

Thresholds CI

La perf comme test

Seuil « p95 < 500 ms » déclaré dans le script. Pipeline CI rouge si dépassé. La performance devient un test au sens strict.

k6 est au test de charge ce que Lighthouse est à l'audit frontal - la vitesse garantie sous charge, pas la vitesse perçue par un seul utilisateur.

k6 en synthèse : prouver les leviers sous charge

Comparer main et final à charge constante : le bilan mathématique du J3

1

Lancer sur main

Branche sans optimisation. Charge réaliste, ~100 users simultanés, scénario métier complet.

2

Voir tomber

TTFB qui explose, taux d'erreur qui grimpe, queues qui débordent. La photographie du point de rupture.

3

Relancer sur final

Branche avec Octane + GIN + cache + PgBouncer. Même protocole, même script, mêmes seuils.

4

Valider × 10

Le serveur encaisse 10× le trafic avec un TTFB stable. Les leviers composés tiennent sous charge réelle.

```
// k6 - la perf comme test CI : le pipeline tombe en cas de régression
export const options = {
  thresholds: {
    http_req_duration: ['p(95)<500'], // p95 < 500 ms
    http_req_failed: ['rate<0.01'], // taux d'échec < 1 %
  },
};
```

La synthèse n'est pas un récit, c'est un benchmark. Les leviers composés tiennent ou ne tiennent pas, mesure à la clé.

La boucle empirique

La méthode qui structurera chaque module de la formation

1. Mesurer

Avant tout : chiffre actuel, distribution, percentile, cohorte. Sans pré-mesure, pas de progression mesurable.

2. Hypothétiser

Une hypothèse falsifiable, une seule variable. « Si on active X, on devrait observer Y sur la métrique Z. »

3. Modifier

Une modification minimale, isolable, reproductible. Pas de big bang qui fait avancer trois choses à la fois.

4. Re-mesurer

Même protocole que l'étape 1. Confirmer (ou invalider). Capitaliser le résultat, incluant les hypothèses fausses.

Pièges classiques de la mesure

Cinq erreurs récurrentes, observées sur la majorité des projets en audit

Mesurer sur sa machine de dev

M2 + fibre + cache chaud. Vos utilisateurs vivent autre chose.

Mesurer une seule fois

La variance d'une métrique de perf est énorme. Toujours répéter, percentiler.

Mesurer après cache chaud sans avoir mesuré à froid

Le cold start est souvent là où la douleur se concentre : vous masquez le problème.

Confondre moyenne et p75

L'erreur la plus banale, la plus coûteuse. Vous racontez une autre histoire que la réalité.

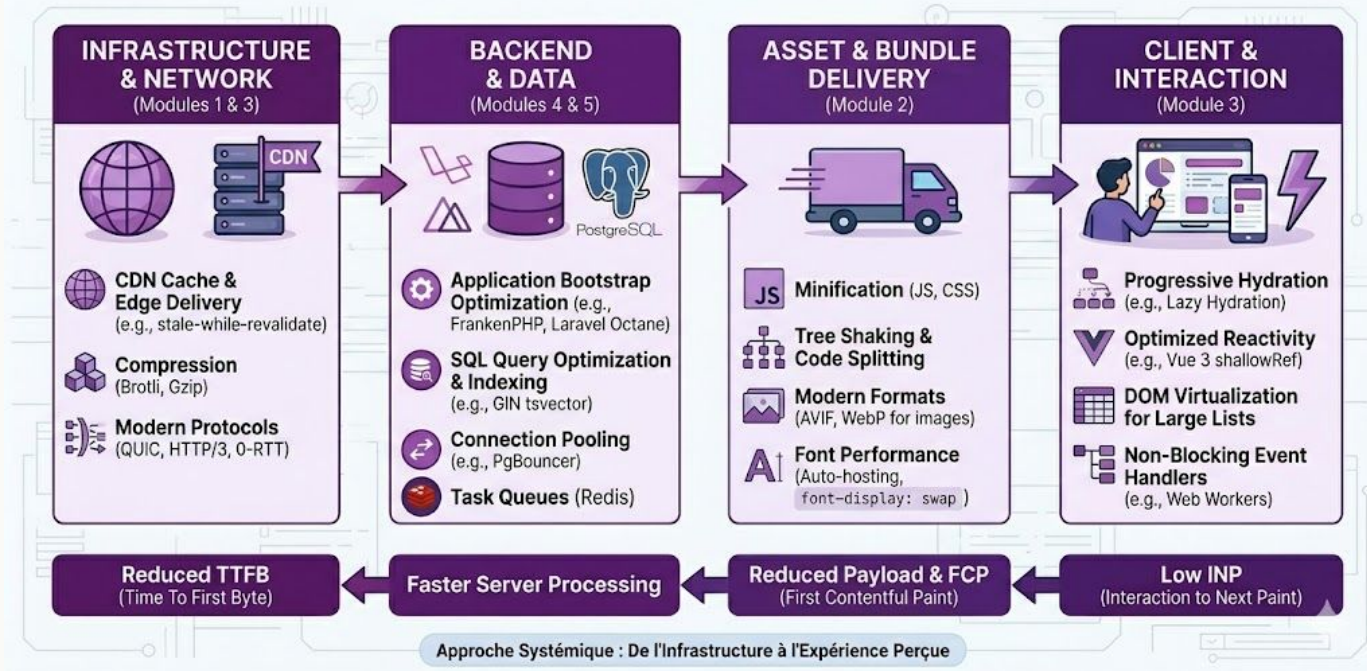
Ne pas re-mesurer après modification

Le « ça a l'air mieux » subjectif ne capitalise rien. La métrique avant/après est non-négociable.

Optimisation de la perf - Vue d'ensemble

Présentation des différents leviers sur lesquels nous allons travailler durant la formation

RÉSONANCE : CARTE COMPLÈTE DES LEVIERS DE PERFORMANCE WEB



Roadmap - Gestion du cycle de vie

Cinq modules, trois jours, où chaque levier opère dans le cycle d'une page

Phases du cycle		1	2	3	4	5	Cible
		Connexion	Serveur	Téléchargement	Rendu	Interactivité	
J1	Module 1 - CDN & cache						TTFB
J2 matin	Module 2 - Bundle frontend						FCP - LCP
J2 aprem	Module 3 - Dashboard & Vue						INP - TBT
J3 matin	Module 4 - Laravel backend						TTFB
J3 aprem	Module 5 - PostgreSQL						TTFB
		↑ convergence					

M1, M4, M5 convergent sur TTFB - réseau, code applicatif, base. Trois angles qui se composent multiplicativement.

Module 1

CDN & cache

Faire chuter TTFB, LCP et payload sans toucher au code applicatif

Branche

`solution/j1-cdn-cache`

Janus Academy - Ali Koudri

Où on en est

Vocabulaire commun, langage de mesure, écosystème cartographié

Vocabulaire

LCP, INP, CLS et leurs métriques de diagnostic : TTFB, FCP, TBT

Méthode

p75 sur RUM, distribution log-normale, cohortes, percentiles

Outils

Synthetic (Lighthouse, k6) + RUM, hiérarchie des caches, observabilité

Boucle

Mesurer → hypothétiser → modifier → re-mesurer. Une variable à la fois.

Maintenant on attaque. Premier morceau : la home de Resonance, qui pêche fort.

La baseline du starter

Mesures Lighthouse + k6 sur la branche main - chiffres bruts, avant toute optimisation

Home - /

Là où ça pêche

Lighthouse Performance	0.55
LCP	16.3 s
FCP	15.9 s
TTFB	2.3 s
k6 home p95	5.3 s
HTML transmis	20 kB

Fiche événement - /events/:slug

Là où ça va déjà

Lighthouse Performance	0.91
LCP	3.0 s
-	
-	
k6 search-load p95	2.9 s
-	

Lecture : la pathologie est localisée - c'est presque toujours le cas. Pas d'optimisation globale, optimisation ciblée.

Décomposer le LCP de 16,3 s

Le temps total se distribue très inégalement entre les phases

Connect

≈ 0,3 s

TTFB

2,3 s

Chargement JS/CSS → FCP

13,6 s

Affichage LCP element

0,4 s

Diagnostic

13,6 s entre TTFB et FCP - soit 83 % du temps total. C'est là que les chunks JS/CSS non compressés se chargent en série, sans HTTP/2.

Différentiel vs absolu

Le garde-fou méthodologique du module, à internaliser avant tout chiffre

Règle

Lire chaque chiffre dans son périmètre. Une branche isolée livre un gain différentiel contre starter, pas une cible absolue.

Cible

LCP home < 2,5 s

Atteinte sur la branche final, après composition de toutes les optimisations.

Comparer cette branche à main = faux négatif.

Gain différentiel J1

LCP home 16,3 s → 5,1 s

Le reste (5,1 → 2,5 s) viendra de solution/j2-bundle - AVIF + polices self-hostées qui débloquent le FCP.

Périmètre cache HTTP + Nginx isolé.

Cf. docs/architecture.md §6 - cibles différentielles vs cibles absolues

Ce que J1 attaque : trois leviers

Trois localisations dans le stack, trois mécanismes complémentaires

1

Compression HTTP

Nginx

Réduit le payload sur le wire : gzip + brotli

2

Cache HTTP browser

Nginx headers + middleware Laravel

Évite la revalidation réseau : immutable + max-age + SWR

3

Cache applicatif

Nuxt Nitro

Évite le recompute SSR - routeRules SWR mémoire

Ce que J1 ne touche pas

Code applicatif Laravel - indexes Postgres - poids des images - taille du bundle Nuxt

Levier 1

Compression HTTP

gzip + brotli côté Nginx - premier coup à zéro effort, gain immédiat sur la connection

Compression HTTP : gzip + brotli

Compression à la transmission, décompression côté client - transparent pour le code

- text/html est compressé implicitement par *gzip on* : pas besoin de l'inclure dans `gzip_types`
- Compression utile sur le texte : HTML, CSS, JS, JSON, SVG, XML, woff2
- `gzip_static` / `brotli_static` : pré-compression au build, zéro CPU à runtime
- Sur Resonance : HTML home 20 kB → 3,4-3,7 kB (÷ 5 à 6)

```
# nginx.conf - bloc http {}
gzip on;
gzip_vary on;
gzip_min_length 1024;
gzip_comp_level 6;
gzip_types text/css
        application/javascript
        application/json image/svg+xml
        font/woff2;
gzip_static on;

brotli on;
brotli_comp_level 6;
brotli_static on;
```

L'image fholzer/nginx-brotli compile déjà le module brotli, il suffit de l'activer.

Brotli - la compression HTTP par défaut en 2026

Négociée par le navigateur, dictionnaire pré-entraîné, meilleur ratio que gzip pour le même CPU

Le mécanisme - négociation HTTP

```
Client → Accept-Encoding: br, gzip, deflate
Serveur → Content-Encoding: br      (si br supporté, sinon gzip)
```

gzip

RFC 1952 - 1996

Compression universelle.
Réduction HTML/JS/CSS typique :
65–75 %.
Supporté par 100 % des clients.
CPU faible.

Brotli (br)

Google - 2015 - RFC 7932

Dictionnaire pré-entraîné sur du web.
Réduction HTML/JS/CSS typique :
75–85 % (≈ 20 % mieux que gzip).
Support 95 %+ aujourd'hui.
CPU comparable à gzip.

Stratégie production - précompresser les assets statiques au build (.br + .gz), compresser à la volée le HTML dynamique. Le coût CPU est largement compensé par les économies de bande passante.

Pièges de la compression

Quatre erreurs récurrentes qui coûtent du CPU sans rendre le service

Compresser le déjà-compressé

PNG, JPEG, WebP, MP4, ZIP : recompression coûte du CPU et peut augmenter la taille.

min_length trop bas

En dessous de 1 kB, l'overhead du header de compression peut dépasser le gain.

comp_level 9

Niveau maximal : gain marginal vs 6, mais coût CPU multiplié par 2 à 3 selon le payload.

Oublier Vary: Accept-Encoding

Si vous compressez selon Accept-Encoding, ce header doit l'annoncer, sinon les caches partagés confondent les versions.

Levier 2

Cache HTTP browser

Cache-Control, immutable, stale-while-revalidate - l'arsenal RFC qu'on sous-utilise tous

Cache-Control : le seul header qui compte vraiment

Tout part de là, les autres mécanismes sont conditionnels

Cache-Control

Prime sur Expires (HTTP/1.0). Définit la politique forte : durée, partage, fraîcheur.

ETag / Last-Modified

Conditional GET (304 Not Modified). Économise le body, pas la requête réseau.

Absence de Cache-Control

Cache implicite selon heuristique browser. Imprévisible, à éviter en production.

Le vrai cache fort = pas de requête réseau du tout. C'est ce qu'on cherche à activer.

Cache-Control : le langage du cache HTTP

Une seule entête, plusieurs directives : qui peut cacher, combien de temps, quand revalider

Directive	Effet
<code>max-age=N</code>	TTL côté client (s)
<code>s-maxage=N</code>	TTL côté shared cache / CDN
<code>public</code>	Cachable par CDN et proxies
<code>private</code>	Seul le navigateur peut cacher
<code>no-cache</code>	Cachable, mais revalider à chaque fois
<code>no-store</code>	Ne jamais stocker
<code>must-revalidate</code>	Si expiré, revalider obligatoirement
<code>immutable</code>	Ne jamais revalider, fingerprint garanti
<code>stale-while-revalidate=N</code>	Servir périmé + revalider en fond
<code>stale-if-error=N</code>	Servir périmé si revalidation échoue

Trois recettes types

Statique fingerprinté

Cache-Control: public,
max-age=31536000, immutable

Assets versionnés (app.abc123.js) - 1 an, jamais revalider.

HTML dynamique avec SWR

Cache-Control: public,
s-maxage=60,
stale-while-revalidate=600

Page liste produits - CDN sert 60 s, rafraîchit en fond jusqu'à 10 min.

Données utilisateur privées

Cache-Control: private,
no-store

Profil, panier, données sensibles - jamais cacher.

Cache-Control n'est pas une option « activer / désactiver », c'est une composition de directives, à régler par type de ressource.

immutable + fingerprintage : cache d'un an sans risque

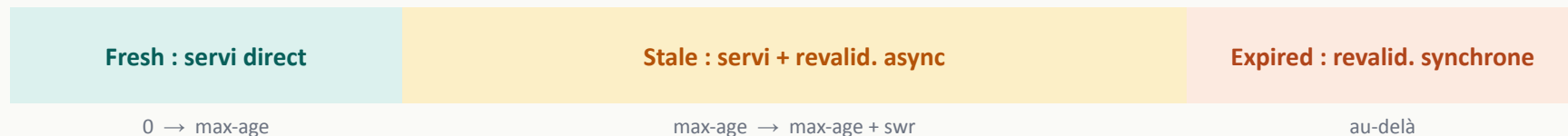
Quand l'URL EST le contenu, on peut cacher éternellement

- Vite / Nuxt hashent les noms de chunks au build - entry.abc123.js
- Le hash dépend du contenu - l'URL change si et seulement si le contenu change
- L'URL devient une identité - cacher 1 an est sans risque de stale
- Si le contenu change, le HTML référence une nouvelle URL - cache miss naturel
- Effet : zéro revalidation sur tous les chunks, tous les utilisateurs

```
# nginx site config
location /_nuxt/ {
    proxy_pass http://nuxt_node;
    add_header Cache-Control "public, max-age=31536000, immutable" always;
}
```

stale-while-revalidate : fraîcheur sans latence

Sémantique RFC 5861 - l'utilisateur n'attend jamais une revalidation



Exemple : $\text{max-age}=300$, $\text{stale-while-revalidate}=900$

- $0 \rightarrow 300$ s : cache hit direct, latence ~ 0
- $300 \rightarrow 1200$ s : cache hit stale, revalidation en arrière-plan ; user voit 0 latence
- Au-delà de 1200 s : cache miss, revalidation synchrone, le user attend

Pièges Cache-Control

Les configurations qui semblent correctes mais désactivent silencieusement le cache

max-age sans public	Browser cache OK, mais caches partagés (CDN, proxies) ne cachent pas. Toujours expliciter public.
must-revalidate	Annule l'effet de stale-while-revalidate quand stale. Force la revalidation synchrone.
Cookies + ressource publique	Vary implicite sur Cookie - les caches partagés ne mutualisent plus entre sessions.
Vary: User-Agent	Cardinalité explose - une entrée de cache par User-Agent unique. Préférer Vary sur Accept-Encoding seul.
Authorization header	Bypass automatique des caches partagés par RFC. Cache local browser uniquement.

Levier 3

Cache applicatif

Nuxt routeRules + Nitro storage : éviter le recompute SSR à chaque requête

SSR - ISR - SWR : trois cycles de vie du cache

Quand sert-on quel rendu ? Trois compromis distincts entre fraîcheur, latence et coût

SSR

Server-Side Rendering

Cycle

Requête



Serveur génère HTML



Réponse

Quand

*Données qui changent
à chaque requête.*

✓ Toujours frais - SEO complet

✗ Coût serveur par requête - Latence = traitement complet

ISR

Incremental Static Regeneration

Cycle

Build initial



Cache statique



Régénération périodique

Quand

*Données semi-statiques
(blog, catalogue, doc).*

✓ Latence minimale (cache) - Fraîcheur contrôlée par TTL

✗ Pic de build à la régénération - Fraîcheur à la granularité du TTL

SWR

Stale-While-Revalidate

Cycle

Requête



Version périmée servie



Revalidation en fond

Quand

*Tolérance à un léger
décalage de fraîcheur.*

✓ Latence quasi nulle - Pas de wait sur revalidation

✗ Premier user reçoit du périmé - Complexité d'invalidation

Le choix n'est pas idéologique : il dépend du couple « volatilité de la donnée × tolérance utilisateur à la fraîcheur ».

Nuxt routeRules - politique de cache par route

Configuration déclarative : pattern matching sur les paths, primitives sémantiques

```
// nuxt.config.ts
export default defineNuxtConfig({
  routeRules: {
    '/': { swr: 60 },
    '/events': { swr: 60 },
    '/events/**': { swr: 300 },
    '/organizer/**': { ssr: false },
  },
})
```

Primitives

swr: N	Cache mémoire SSR
ssr: false	Bascule en CSR
isr: N	ISR plateforme
cache: {}	Bloc cache custom
prerender	Génération statique

Note

swr et isr ne se comportent pas pareil selon le preset de build (Vercel, Cloudflare, node-server). On y revient plus tard, c'est un risque de piège.

Nitro : l'engine serveur de Nuxt

Découple le code applicatif de la plateforme de déploiement - Node, Edge, Cloudflare, Vercel, Deno

Qu'est-ce que Nitro ?

Engine serveur sous-jacent à Nuxt 3+. Il prend le code Nuxt et le compile pour la plateforme cible (Node.js, Cloudflare Workers, Vercel Edge, Deno Deploy, AWS Lambda). Le même code applicatif tourne partout.

Route Rules

Cache & headers par route

Déclarer en config : prerender, swr, cache, isr, headers, redirects, par pattern de route. Sans toucher au code.

```
/api/products/**: { swr: 60 }
}
```

Server Routes

Endpoints type Express

Fichiers dans server/api/ deviennent des routes HTTP. Handlers TypeScript, accès direct au runtime cible.

```
server/api/products.get.ts
```

Storage

KV abstraction

Layer de stockage unifié, adapters Redis, fs, memory, KV. Le code reste portable.

```
useStorage('redis')
```

Hooks

Cycle de vie

Interceptor request/response. Logging, métriques, transformations, rate-limiting. Sans wrapper le serveur.

```
nitroApp.hooks.hook(...)
```

Nitro est la raison pour laquelle « vous écrivez du Nuxt » et « vous déployez sur Cloudflare » sont deux décisions indépendantes.

swr sur preset node-server : comment ça marche

Nitro stocke la page HTML rendue en mémoire du process Node, l'effet est radical

Hit 1

SSR full + put en cache mémoire Nitro

~ 2 s

Hits 2..N

Direct depuis le cache (lecture mémoire)

~ 1-10 ms

Au-delà TTL

Stale servi + SSR async background → cache refresh

~ 1-10 ms
(user)

Limites

- Single-process + multi-instance Node = caches indépendants (cache hit ratio ↓)
- Pas de persistance + reboot Node = cache vide, premier hit cher à nouveau
- Pas d'invalidation déclarative, on attend l'expiration du TTL

ISR vs SWR : pourquoi ça compte sur node-server

Même mot, comportement très différent selon le preset

Sur Vercel / Cloudflare

Plateformes d'hébergement edge

- isr: N délègue au cache plateforme
- Fichiers statiques pré-rendus servis par le CDN edge
- Vraie sémantique « Incremental Static Regeneration »
- swr et isr utilisables, sémantiques distinctes

Sur preset node-server

Notre cas - déploiement Node classique

- isr: N est un no-op runtime - aucun bloc cache généré
- Seul swr: N produit cachedEventHandler
- Storage Nitro en mémoire process, pas d'edge
- Pour vrai ISR sur Node : configurer nitro.storage.cache + cache block

Règle : sur Node, écrire swr. Pas isr. Slide suivante - la preuve par le code généré (slide suivante).

La preuve : .output/server/chunks/nitro/nitro.mjs

Inspectez le build de production : les deux primitives ne génèrent pas le même runtime

`routeRules: { '/': { isr: 60 } }`

```
// Généré :  
"/": {  
  "isr": 60  
}  
  
// → aucun bloc cache  
// → no-op à runtime
```

`routeRules: { '/': { swr: 60 } }`

```
// Généré :  
"/": {  
  "swr": 60,  
  "cache": {  
    "swr": true,  
    "maxAge": 60  
  }  
}
```

À retenir

- Le marketing hébergeur appelle isr ce qui est en réalité un swr avec storage CDN
- Sur Node sans configuration explicite, isr n'a aucun effet runtime, silencieusement
- Lire le build de prod (.output/server/chunks/nitro/nitro.mjs) est un réflexe sain

Middleware Laravel - politique Cache-Control centralisée

Headers de cache appliqués hors des controllers : un seul endroit à maintenir

```
// SetEventsCacheControl.php
public function handle(Request $req, Closure $next): Response {
    $resp = $next($req);
    if (! $req->isMethodSafe() || $resp->getStatusCode() !== 200) {
        return $resp; // GET/HEAD only, 200 only
    }
    $isList = $req->is('api/v1/events');
    [$max, $swr] = $isList ? [60, 300] : [300, 900];
    $resp->headers->set('Cache-Control',
        "public, max-age={$max}, s-maxage={$max}, stale-while-revalidate={$swr}");
    return $resp;
}
```

isMethodSafe() : couvre GET + HEAD (RFC : HEAD renvoie les mêmes headers que GET) - Laravel 11+ : middleware injecté à la définition des routes via Route::middleware()->group()

Patterns

Hiérarchie et transport

Où chaque levier opère dans la chaîne - ce qui se compose, ce qui s'exclut

Six niveaux de cache, six rôles distincts

Du navigateur à la base, chaque niveau a son scope, son temps de vie, son levier de contrôle

Niveau	Où	Scope	TTL typique	Levier	Exemple Resonance
1 Navigateur	Client	Par utilisateur	max-age, expires	Cache-Control	Statiques fingerprint
2 Service Worker	Client	Par utilisateur	Programmable	Workbox, fetch event	Cache offline (PWA)
3 CDN / Edge	Réseau	Partagé géographique	s-maxage	Cache-Control + Vary	Cloudflare devant Nuxt
4 Reverse proxy	Devant l'app	Partagé local	Configurable	Nginx proxy_cache	Cache 200 OK applicatif
5 Cache applicatif	Backend	Process / cluster	Programmable	Redis, Memcached	Cache::remember Laravel
6 Buffer DB	Base	Pages mémoire	Géré par PG	shared_buffers	Postgres hot pages

Chaque levier de cache opère à un niveau précis. Avant « lequel utiliser », poser « à quel niveau le problème survient ».

TTL : quoi cacher et combien de temps ?

Table de référence - point de départ, à ajuster selon le profil de fraîcheur de chaque ressource

Ressource	Directive	Justification
<code>/_nuxt/*.js, *.css</code>	<code>max-age=31536000, immutable</code>	Fingerprinté - URL = identité
<code>/media/* (images)</code>	<code>max-age=86400, swr=604800</code>	Semi-statique, tolère stale
HTML home /	<code>swr: 60 (routeRules)</code>	Liste fréquemment mise à jour
HTML fiche <code>/events/*</code>	<code>swr: 300 (routeRules)</code>	Détail rarement modifié
API liste <code>/api/v1/events</code>	<code>max-age=60, swr=300</code>	Liste publique, fraîcheur OK
API détail	<code>max-age=300, swr=900</code>	Détail public stable
API authentifiée	<code>no-store</code>	Données personnalisées, sensibles

HTTP/2 - multiplexage et keep-alive

Sur la home Nuxt avec 20+ chunks, l'effet est immédiat - configuration triviale

HTTP/1.1

- 6 connexions max par origine
- Requêtes séquentielles dans chaque connexion
- Head-of-line blocking
- Pour 20 chunks - 4 vagues sérielles

HTTP/2

- 1 connexion TCP, 100+ streams parallèles
- Multiplexage natif
- Header compression (HPACK)
- Pour 20 chunks - 1 vague parallèle

```
# nginx site config (Nginx 1.25+)
listen 80 default_server;
http2 on;   # h2c - TLS hors périmètre J1
```

Lab

Mise en œuvre

Cinq étapes, 60-90 min - branche `solution/j1-cdn-cache`, fiche docs/ateliers/j1-cdn-cache.md

Les cinq étapes du lab

Ordre suggéré - chaque étape est mesurable indépendamment

1

Compression Nginx

gzip on, brotli on - bloc http {}

infra/nginx/nginx.conf

2

HTTP/2 + Cache-Control statiques

http2 on, headers immutable sur /_nuxt/*, SWR sur /media/*

infra/nginx/sites/resonance.conf

3

Middleware Laravel

SetEventsCacheControl, appliqué via Route::middleware()

backend/app/Http/Middleware/

4

routeRules Nuxt SWR

swr sur /, /events, /events/** - ssr: false sur /organizer

frontend/nuxt.config.ts

5

Mesurer

make k6, make lighthouse - comparer à docs/benchmarks/k6-starter/

make k6 + make lighthouse

Métriques attendues - cible J1

Gains différentiels mesurés vs starter - cf. docs/benchmarks/j1-cdn-cache-comparison.md

Métrique	Starter	Cible J1	Attendu
Lighthouse home Perf	0.55	> 0.75	0.76 - +38 %
LCP home	16.3 s	< 6 s	5.1 s - -69 %
k6 home p95	5.3 s	< 2 s	2.14 ms - ÷ 2480
TTFB home (cache hit)	2.3 s	-	0 ms
Compression HTML /	20 kB	-	3.4-3.7 kB - ÷ 5-6
Lighthouse fiche Perf	0.91	inchangé	0.91 - hors périmètre J1

À garder en tête : ces chiffres seront mesurés en live. La discussion porte sur les écarts, pas sur l'exactitude.

Smoke tests à dérouler en fin d'atelier

Cinq vérifications curl - fiche §6 pour les commandes complètes

1

Compression active

`curl -H 'Accept-Encoding: gzip' / → Content-Encoding: gzip`

2

Cache-Control immutable

`curl -I /_nuxt/<chunk>.js → max-age=31536000, immutable`

3

Cache-Control applicatif

`curl -I /api/v1/events → s-maxage=60, stale-while-revalidate=300`

4

SWR Nitro fonctionne

5 hits successifs - hit 1 $\approx$ 2 s, hits 2-5 < 10 ms

5

Tunnel d'achat fonctionnel

`POST /auth/login → /me/tickets` - pas de régression

Cap sur le lab

90 min

Branche

solution/j1-cdn-cache

Fiche

`docs/ateliers/j1-cdn-cache.md`

À débriefer ensuite

- Pourquoi $k6\ p95 \div 2480$?
- Pourquoi LCP reste à 5,1 s ?
- Pourquoi la fiche est inchangée ?

Module 2

Bundle, images et polices

Réduire le payload navigateur sans toucher au backend ni au caching

Branche

`solution/j2-bundle`

Janus Academy - Ali Koudri

Transition depuis J1

J1 a effacé le coût de transmission. Reste à attaquer le payload lui-même.

Ce que J1 a livré

- Compression Nginx (gzip + brotli)
- Cache-Control immutable + SWR
- Cache applicatif Nuxt Nitro (mémoire)
- HTTP/2 multiplexage

Ce qui reste à attaquer

- Le poids du JS bundle envoyé au client
- Les images servies en JPEG full-size
- Le round-trip Google Fonts bloquant
- L'hydratation coûteuse du critical path

Rappel décomposition LCP starter : 13,6 s entre TTFB et FCP - c'est ce payload qu'on va attaquer.

Décomposer le payload du starter

Quatre sources de poids, chacune ciblée par un levier J2

JS bundle organizer (non-dashboard)	~ 131 Ko gz	Chart.js importé globalement dans le layout, chargé sur events list, edit, participants alors qu'inutilisé.	→ <i>Code splitting</i>
Image hero home	283 Ko JPEG	Photo full-size servie sans transform : pas de variantes responsive, pas d'AVIF.	→ <i>@nuxt/image</i>
Image card événement	283 Ko JPEG	Même fichier source que le hero : viewer mobile télécharge 1920×1080 pour afficher 320×180.	→ <i>@nuxt/image</i>
Polices Google Fonts	DNS+TLS+CSS bloquant	<link href='fonts.googleapis.com'> dans <head> - round-trip critique bloque le rendering.	→ <i>@nuxt/fonts</i>

Cas d'école : différentiel vs absolu

J2 livre -66 % sur le LCP, mais reste à 5,6 s. Ce n'est pas un échec, c'est une composition à activer.

À garder en tête

5,6 s sur cette branche isolée n'est pas un échec de J2 : c'est exactement la signature attendue d'optimisations qui se composent.

J2 isolé - branche solution/j2-bundle

LCP home 16,3 s → 5,6 s

Reste bloqué par TTFB SSR ~ 1,9 s

Nuxt fetch /api/v1/events à chaque requête sans cache - J2 ne touche pas au backend.

J1 + J2 composés - branche final

LCP home < 2,5 s - cible §9

SWR Nitro de J1 efface le TTFB (~ 0 ms en cache hit)

image légère (J2) + FCP débloqué (J2) + TTFB nul (J1) → cible atteignable.

Cf. docs/architecture.md §6 - toujours lire les chiffres dans leur périmètre

Les 4 leviers de J2

4 angles d'attaque côté frontend, complémentaires et indépendantes

1

@nuxt/image (IPX)

Pipeline server-side - AVIF/WebP négociés, srcset adaptatif, transform à la requête

Cible : Poids des images

2

@nuxt/fonts

Self-hosting Inter - font-display swap, preload poids 400 seul

Cible : Round-trip Google Fonts

3

Code splitting Chart.js

defineAsyncComponent sur SalesChart - Chart.js sort du bundle initial

Cible : First Load JS organizer

4

Lazy hydration

<LazyTicketSelector hydrate-on-idle> - SSR rendu, hydration différée

Cible : Main thread pour le LCP

Ne touche pas : backend Laravel, cache HTTP (J1), indexes Postgres (J3)

Levier 1

Images

Le poste le plus lourd d'une page moderne - 4 leviers techniques à connaître

Formats modernes : AVIF, WebP, et les autres

Le gain compression est massif dès qu'on quitte JPEG/PNG, encore faut-il que les browsers suivent

Format	Ratio vs JPEG	Support (2026)	Usage recommandé
JPEG	Référence	100 %	Fallback universel
PNG	Sans perte, lourd	100 %	Graphics, transparence
WebP	-30 %	~ 97 %	Bonne baseline mobile
AVIF	-50 %	~ 94 %	Format cible 2026, meilleur
JPEG XL	-60 % théorique	Safari seul	Pas encore, Chrome retiré en 2022

Stratégie : négociation via Accept header → chain AVIF → WebP → JPEG (fallback). Le serveur sert le meilleur format que le browser accepte.

Responsive images : srcset + sizes

Un téléphone n'a pas besoin d'une image 1920px, encore faut-il le lui dire correctement

```

```

srcset

Les variantes disponibles, avec leur largeur en pixels (descriptor w).

Le browser choisit la meilleure pour le viewport courant.

sizes

Indique comment l'image occupe l'écran à chaque breakpoint.

Sans sizes correct, le browser télécharge la variante max.

Images : trois attributs qui pilotent le téléchargement

fetchpriority, loading, decoding : hints HTML standards pour piloter ce que le navigateur charge en premier, et comment

fetchpriority

Priorité de téléchargement

Hint au navigateur sur l'urgence relative - utile quand l'heuristique automatique se trompe (ex. LCP non détecté).

auto - défaut, heuristique
high - prioritaire
low - différé

loading

Quand télécharger

Différer le fetch des images hors viewport jusqu'à ce qu'elles approchent - économise bande passante et CPU initial.

eager - immédiat (défaut)
lazy - différé, déclenché à proximité du viewport

decoding

Comment décoder

Le décodage d'image bloque le main thread si synchrone. async permet de décoder en parallèle du rendu.

auto - navigateur choisit
sync - bloque le rendu
async - décodage parallèle

Trois recettes types selon le rôle de l'image

Image hero / LCP

```
fetchpriority="high"  
loading="eager"  
decoding="sync"
```

Au-dessus du fold, candidate au LCP - tout charger maintenant.

Image below the fold

```
loading="lazy"  
decoding="async"
```

Hors viewport initial - attendre que l'utilisateur scroll.

Image décorative / fond

```
loading="lazy"  
fetchpriority="low"  
decoding="async"
```

Optionnelle pour la compréhension - last priority.

Ces trois attributs ne s'opposent pas - ils se composent. Par image, à régler selon le rôle dans la composition de la page.

Priorisation : fetchpriority, lazy ou decoding ?

Le browser doit savoir lesquelles charger en premier, lesquelles différer, lesquelles décoder en async

fetchpriority="high"

Sur le LCP element - pousse la priorité HTTP/2 - premier byte plus tôt

fetchpriority="low"

Sur les below-fold non critiques - libère la bande passante pour le critical path

loading="lazy"

Below-fold uniquement - natif depuis 2019 - intersection observer browser-managed

decoding="async"

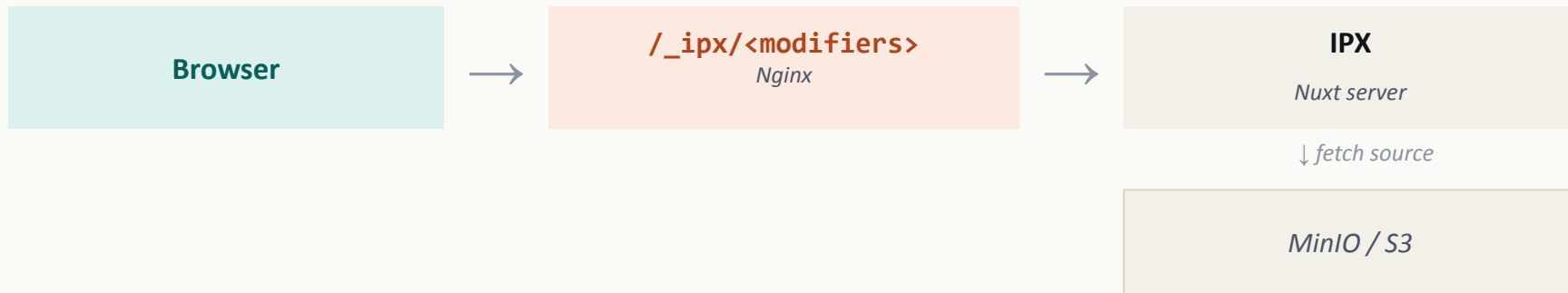
Décodage non-bloquant pour le rendering - défaut depuis Chrome 65

Pattern type - hero LCP

fetchpriority="high" loading="eager" decoding="auto" - pour le contraire (cards lazy) :
loading="lazy" decoding="async"

Pipeline server-side : @nuxt/image + IPX

Transform à la requête : négociation format, resize, quality - pas de pré-build d'images



```
image: {
  provider: 'ipx',
  domains: ['minio:9000', 'localhost:9000'],
  format: ['avif', 'webp'],
  quality: 75,
  presets: { hero: {...}, card: {...} },
}
```

Au final

AVIF négocié, taille adaptative, transform à la requête, mise en cache disque côté IPX.

IPX : qui doit traiter les images ?

Redimensionner, compresser, convertir vers les formats modernes : le bon endroit n'est ni Laravel, ni un SaaS tiers

Le problème : deux mauvaises options

✗ Laravel s'en charge

Traitement d'image = très lourd CPU PHP. Pour 5 tailles responsive × chaque upload → workers saturés. PHP n'est pas le bon outil pour transformer du raster.

✗ Service cloud tiers

Cloudinary, Imgix, Bunny.net. Travail délégué - mais facturation au volume, bande passante doublée (Laravel → cloud → user), dépendance externe.

La solution : IPX, le moteur intégré à @nuxt/image

IPX - Image Proxy

Moteur open-source

Intégré par défaut à @nuxt/image. Pas de service à provisionner.

libvips / sharp

Backend Node.js. Très rapide, asynchrone, charge légère par image.

À la volée + cache

Transformation au premier hit, cache local pour les suivants.

IPX vit côté Nuxt, parle à Laravel, sert au navigateur. Laravel garde son rôle (stocker), Node.js fait ce qu'il fait le mieux (transformer).

Le flux d'une image : trois acteurs en équipe

Laravel stocke brut. Nuxt déclare la transformation. IPX exécute et cache. Le bon outil au bon endroit

Côté composant Nuxt

```
<NuxtImg  
  src="https://api.monsite.com/storage/photo.jpg"  
  width="400"  
  format="webp" />
```

URL générée et interceptée par IPX

```
GET /_ipx/w_400,f_webp/  
  https://api.monsite.com/storage/photo.jpg  
  
// → réponse : photo redimensionnée 400px, WebP
```

1

Upload

User envoie
photo 4 Mo

Laravel

2

Stockage

S3 ou disk
brut, non touché

Laravel

3

URL brute

API renvoie
lien direct

Laravel

4

Transform

IPX fetch +
sharp / libvips

IPX (Nuxt)

5

Cache

Version optimisée
stockée localement

IPX (Nuxt)

Première requête : fetch + transform + cache (≈ 200-500 ms). Requêtes suivantes - cache hit immédiat (≈ 5 ms). Laravel n'est plus touché.

Pourquoi IPX dans la stack : quatre bénéfices structurels

Soulage le backend, sert on-demand, négocie le format moderne, contrôle le périmètre

Soulage Laravel

CPU déporté

PHP n'est pas conçu pour l'image. Le travail va chez Node.js qui gère mieux l'I/O asynchrone et les bindings natifs (sharp / libvips).

On-Demand

Tailles non pré-définies

Pas besoin de générer thumbnail / medium / large au upload. Vous changez width=400 → 420 dans le composant, IPX produit la nouvelle taille au premier hit.

Format auto

f_auto vers AVIF / WebP

IPX négocie le format selon Accept du navigateur. Chrome → AVIF, Firefox → WebP, vieux Safari → JPEG. Sans changer la source.

Contrôle des domaines

Pas de vol de bande passante

Liste blanche dans nuxt.config.ts - IPX refuse de traiter une URL hors de ton API. Évite que ton serveur serve d'optimiseur gratuit pour des tiers.

Configuration nuxt.config.ts pour protéger le périmètre

```
// nuxt.config.ts - extrait pertinent
modules: ['@nuxt/image'],
image: {
  domains: ['api.monsite.com'], // seuls les domaines listés sont traités
  format: ['avif', 'webp'],    // ordre de préférence pour f_auto
}
```

IPX n'est pas un détail de stack, c'est le composant qui décide où va le CPU image, et combien de bande passante on consomme inutilement.

Découverte : domaines avec port (@nuxt/image)

Validation silencieuse qui annule l'optimisation sans erreur explicite

Le piège

Configuration naïve

```
domains: ['minio'],  
// URL: http://minio:9000/...
```

`parseURL().host` → 'minio:9000' ≠ 'minio' - validation échoue silencieusement

Conséquence : srcset retombe sur l'URL brute, pas d'IPX, pas d'AVIF.

Le fix

Inclure le port explicitement

```
domains: ['minio:9000',  
          'localhost:9000'],
```

Test visuel : view-source: + grep srcset

Chaque width descriptor doit pointer vers un `/_ipx/<modifiers>/...` distinct. Si tous identiques - validation a échoué.

Erreur silencieuse - pas de log d'erreur, juste la perte du gain. Réflexe : inspecter le markup généré, pas le code source.

Découverte : preload + srcset = LCP aggravé

Le commit qui annule une optimisation pour en améliorer une autre

L'intuition (mauvaise)

Hero = LCP + preload pour gagner du temps

```
<link rel='preload'  
      as='image'  
      href='hero-1920.avif'>
```

Le preload pointe vers UNE variante précise.

La réalité (mesurée)

Le browser via srcset sélectionne UNE AUTRE variante

- Preload - fetch hero-1920.avif
- Srcset à l'évaluation : fetch hero-640.avif (mobile)
- Deux fetchs au lieu d'un
- Preload gaspillé, bande passante double
- LCP plus lent qu'avec aucun preload

Principe : si vous utilisez srcset/sizes, le browser sait mieux que vous. fetchpriority='high' sur <NuxtImg> suffit.

Levier 2

Polices

Le levier sous-estimé - souvent le gain dominant sur le FCP

Le coût caché des web fonts via CDN tiers

Google Fonts en <link> dans <head> : ce que ça coûte vraiment

1. DNS lookup

fonts.googleapis.com : ~ 30-50 ms sur connexion mobile

2. TLS handshake

Nouvelle origine, nouveau handshake : ~ 100-200 ms

3. Fetch CSS bloquant

Le CSS reçu déclare ensuite N requêtes vers fonts.gstatic.com

4. DNS + TLS gstatic

Deuxième origine, deuxième round trip complet

5. Fetch woff2

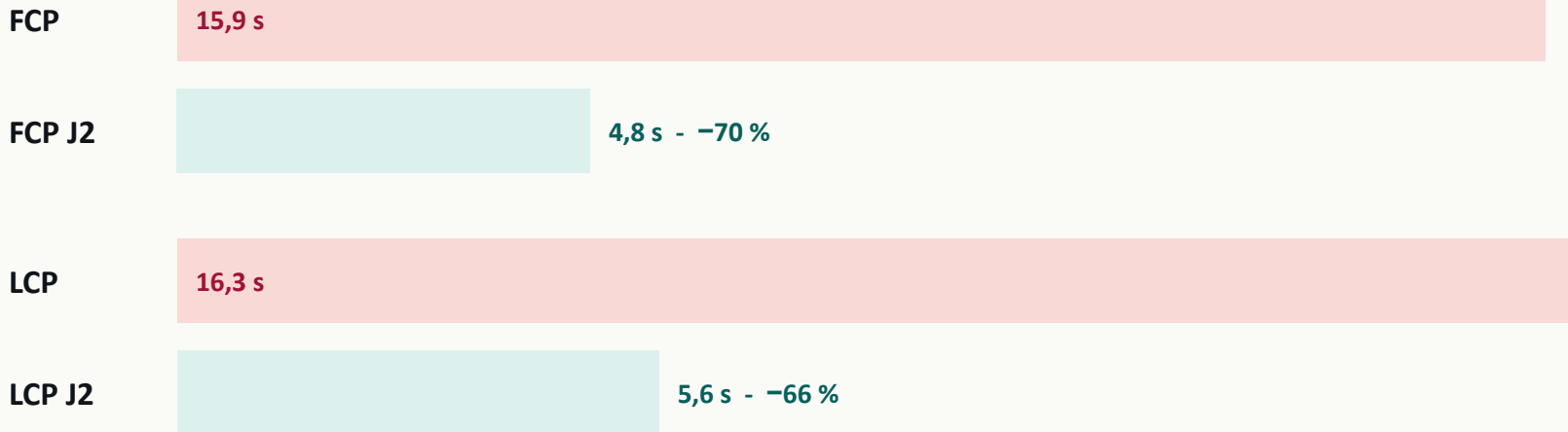
Pour chaque poids/style utilisé : ~ 20-40 Ko chacun

Total sur mobile 4G

1 à 2 secondes ajoutées au FCP : la CSS Google Fonts est dans le critical path render-blocking.

Découverte : FCP -70 % > LCP -66 %

Sur la home Resonance, c'étaient les polices qui dominaient le LCP catastrophique, et non l'image



Leçon

Le LCP catastrophique cachait un problème de polices, pas d'image. Toujours décomposer LCP en TTFB + FCP / TTFB + LCP-FCP avant d'optimiser.

@nuxt/fonts : self-hosting + font-display swap

Une dépendance, une config, et fini le round-trip Google

```
// nuxt.config.ts
fonts: {
  families: [
    // Preload Regular (corps de texte LCP) ; les autres poids
    // chargent sans preload pour ne pas saturer le critical path.
    { name: 'Inter', weights: [400], styles: ['normal'], preload: true },
    { name: 'Inter', weights: [500, 600, 700], styles: ['normal'] },
  ],
  defaults: { fallbacks: { 'sans-serif': ['system-ui', 'sans-serif'] } },
},
```

- Self-hosting automatique : download des polices au build, servies sur `/_fonts/<hash>.woff2`
- font-display: swap par défaut : texte affiché immédiatement avec le fallback, swap quand la police arrive
- Preload sélectif : seul Inter Regular 400 est préchargé (utilisé sur le LCP element)
- Retrait des `<link>` Google Fonts dans `nuxt.config app.head` : `@nuxt/fonts` les remplace

Levier 3

Code splitting

Ne livrer au navigateur que le code qu'il va exécuter dans la prochaine seconde

First Load JS : la métrique frontale

Somme du JS parsé et exécuté avant le premier rendu interactif

< 100 Ko gz

Cible idéale : smartphones milieu de gamme parsent en < 200 ms

< 150 Ko gz

Cible acceptable : recommandation Vercel / Next.js, atteignable sur la plupart des stacks

> 150 Ko gz

Zone rouge : TBT explose, INP dégradé, parsing CPU-bound sur mobiles bas de gamme

Comment savoir

nuxi analyze - treemap interactif des chunks - identifier les libs full-bundle (charting, date, markdown, code-highlight, validation)

Code splitting : découper le monolithe JS

Un seul app.js de 2 Mo bloque tout. La solution : des chunks chargés à la demande, par route ou par composant

Le problème - le monolithe JS

Sans code splitting, le navigateur télécharge un seul énorme fichier app.js, souvent 1-3 Mo en production. Tout doit être parsé, compilé, exécuté avant la première interaction.

Conséquences directes - TBT (Total Blocking Time) qui explose - main thread bloqué pour parser le code du Dashboard alors que l'utilisateur est sur Login - TTI repoussé.

Le concept : deux niveaux de fractionnement

Route-based splitting

Le plus efficace

Chaque page devient son propre fichier JS.
/home → home.js, /dashboard → dashboard.js.

Dans Nuxt 3, c'est automatique : chaque fichier de pages/ génère son chunk dédié.

Component-based splitting

Granularité fine

Isoler les composants lourds non toujours visibles.
Modales, graphes Chart.js, éditeurs riches, annexes Stripe.

Via préfixe Lazy de Nuxt ou defineAsyncComponent de Vue 3.

Le principe est simple : ne télécharger que ce qui est nécessaire au moment où c'est nécessaire. Pas avant.

Implémentation : préfixe Lazy ou defineAsyncComponent

Nuxt offre une convention zéro-config. Vue 3 expose l'API complète quand on a besoin d'options avancées

1. La voie Nuxt : préfixer par Lazy

```
<template>
  <h1>Mon Dashboard</h1>

  <LazyAnalyticsChart v-if="show" />
  <button @click="show = true">
    Afficher les graphiques
  </button>
</template>
```

Ce qui se passe

Pour tout composant du dossier components/, ajouter Lazy en préfixe à l'appel suffit.

Nuxt génère automatiquement un chunk séparé. Le navigateur télécharge AnalyticsChart.js uniquement quand v-if devient true.

2. Avec contrôle fin : defineAsyncComponent

```
<script setup>
const BigChart = defineAsyncComponent({
  loader: () => import('./BigChart.vue'),
  loadingComponent: ChartSkeleton,
  delay: 200,      // ms avant d'afficher le loader
  timeout: 3000,   // ms avant d'abandonner
})
</script>
```

Quand l'utiliser

Quand vous voulez un skeleton de chargement (loadingComponent), un timeout réseau, ou éviter le flash si le chargement est très rapide (delay).

Règle : Lazy pour le 90 % des cas, defineAsyncComponent quand on veut un skeleton ou un timeout précis.

Impact sur les métriques : gains et piège

Trois métriques s'améliorent franchement, une demande de la discipline : anticiper l'arrivée différée d'un composant

Métrique	Impact	Mécanisme
FCP / LCP	Amélioration nette	Moins de bytes à parser avant le premier paint. Le bundle initial passe de 2 Mo à 300-500 Ko.
TBT	Réduction drastique	Le main thread n'analyse plus le code des écrans non visités. Longtasks > 50 ms supprimées.
TTI	Avancé	Page interactive plus rapide : l'utilisateur peut cliquer avant que tout le code soit chargé.
CLS	⚠ À surveiller	Un composant chargé à la demande qui apparaît brutalement décale ce qui est en-dessous.

La parade au piège CLS - réserver la place avec un skeleton

```
<template>
  <!-- Le skeleton occupe la même hauteur que le composant final -->
  <LazyAnalyticsChart v-if="loaded" />
  <ChartSkeleton v-else style="height: 400px" />
</template>
```

Code splitting sans skeleton : on gagne sur la perf brute, on perd sur la stabilité visuelle. Toujours réserver la place avant le chargement différé.

Exemple avancé avec defineAsyncComponent

L'API Vue pour faire d'un import statique un chunk chargé à la demande

```
import { defineAsyncComponent, h } from 'vue'

const SalesChart = defineAsyncComponent({
  loader: () => import('~/components/SalesChart.vue'),
  loadingComponent: () => h('div', {
    class: 'h-64 grid place-items-center text-sm text-slate-400',
  }, 'Chargement de la courbe...'),
  errorComponent: () => h('div', 'Erreur de chargement'),
  delay: 200,           // ms avant d'afficher loadingComponent
  timeout: 10000,       // ms avant d'afficher errorComponent
})
```

- `import('./Foo')` : module ESM dynamique : Vite/Rollup le transforme en chunk séparé
- `defineAsyncComponent` : wrapper Vue 3 qui gère le cycle loading/loaded/error
- Effet bundle : SalesChart absent du bundle initial, fetché au mount du composant parent

Cas Chart.js : -43 % First Load JS organizer

Une lib lourde, importée trop haut dans la hiérarchie - extraction chirurgicale

Avant

Chart.js dans Layouts/organizer.vue

- Chargé sur TOUTES les pages organizer
- Pages affectées sans utiliser : events list, edit, participants
- First Load JS gonflé inutilement
- ~ 162 Ko raw / ~ 75 Ko gz

Après

Composant SalesChart en defineAsyncComponent

- Chart.js dans ~/components/SalesChart.vue uniquement
- Importé seulement dans pages/organizer/dashboard.vue
- Sort du bundle initial, chunk séparé
- First Load JS organizer : ~ 75 Ko gz total (-43 %)

Lecture : isoler les libs lourdes dans le composant qui les utilise, pas dans le layout parent. Réflexe systématique.

Tree shaking : utile, mais pas une excuse

Élimine le code mort au build - ne remplace pas le code splitting

Quand ça marche

- Lib avec exports nommés (ESM)
- Pas de side effects en top-level
- Bundler moderne : Vite, Rollup
- Imports nommés : `import { fn } from 'lib'`

Quand ça échoue silencieusement

- Import full namespace : `import _ from 'lodash'`
- Module CommonJS : pas d'analyse statique
- Side effects en top-level (chart.js, moment)
- Réexports indirects : `re-export * from`

Tree shaking n'élimine pas ce qui est utilisé UNE FOIS. Pour cela : code splitting, nuxi analyse d'abord, tree shaking ensuite.

Tree-shaking : l'effet du style fonctionnel

Le bundler ne peut éliminer que ce qu'il peut prouver inutile : mutation et effets de bord rendent le code inanalysable

Comment marche le tree-shaking

Le bundler (Vite, Rollup, Webpack) analyse statiquement les imports ESM. Il marque comme « mort » tout export qui n'est jamais importé. Mais il ne supprime que ce qu'il est sûr de pouvoir supprimer sans casser un effet de bord.

✗ Impératif avec état partagé

```
// utils.js
let counter = 0      // état partagé

export function bump() {
  counter++          // mutation
  return counter
}

export function unused() {
  document.title = 'x' // side effect
}
```

Le bundler garde TOUT le module - il ne peut prouver que unused n'a pas d'effet utile au chargement.

✓ Fonctionnel, pures, ESM strict

```
// utils.js
export const bump = (n) => n + 1

export const compose = (...fns) =>
  (x) => fns.reduceRight(
    (v, f) => f(v), x)

export const unused = () => 42

// package.json
// { "sideEffects": false }
```

Le bundler élimine unused proprement, garde uniquement ce que vous importez vraiment.

Checklist tree-shake-friendly - pure functions - imports/exports ESM nommés (pas import *) - sideEffects:false dans package.json - éviter les barrels qui ré-exportent tout

Levier 4

Lazy hydration

Différer le travail JS qui n'est pas critique pour le LCP

Hydratation : ce que ça coûte vraiment

Le moment où Vue rattache son réactif au HTML SSR : invisible mais coûteux

1. SSR

Le serveur rend le HTML, le browser l'affiche, l'utilisateur voit le contenu

2. Hydratation

Vue parcourt l'arbre, rattache handlers, watchers, computed : parsing + setup réactif

3. Interactive

L'utilisateur peut cliquer : les handlers répondent - Time To Interactive (TTI)

Coût mesurable

- INP : premier clic peut être bloqué si une long task d'hydratation est en cours
- TBT : une long task > 50 ms pour chaque gros composant hydraté
- Composants below-fold hydratés en premier - même invisibles à l'écran

Lazy Hydration : étaler l'hydratation dans le temps

L'hydratation synchrone produit un pic de TBT au démarrage. Vue 3.5+ permet de déclarer QUAND hydrater chaque composant

Le problème - hydratation synchrone = pic de TBT au démarrage

Le serveur envoie un HTML rendu (SSR). Au chargement, Vue parcourt l'arbre entier et attache la réactivité à chaque composant - d'un coup, sur le main thread.

Profil CPU au boot (synchrone)

[ pic 400 ms - main thread bloqué ] _____ idle _____ → TBT élevé

Le concept : quatre stratégies pour décider du moment d'hydratation

hydrate-on-idle

Au repos CPU

détaillée
slide suivante

hydrate-on-visible

Au scroll viewport

détaillée
slide suivante

hydrate-on-interaction

À l'interaction

détaillée
slide suivante

hydrate-on-media-query

Selon l'appareil

détaillée
slide suivante

Vue 3.5+ / Nuxt 3.12+ : ces directives sont natives. Pas de lib externe, pas de bundle supplémentaire.

Les quatre stratégies de Lazy Hydration

Chacune a son déclencheur natif, son API navigateur sous-jacente, son cas d'usage Resonance

hydrate-on-idle

Au repos du processeur - requestIdleCallback

Hydraté dès que le main thread a fini les tâches critiques. Repoussé si CPU saturé (avec timeout de sécurité).

Resonance - menu de nav secondaire, badges d'info, profil utilisateur en haut à droite.

hydrate-on-visible

Au défilement - IntersectionObserver

Reste un bloc HTML statique tant qu'il n'entre pas dans le viewport. Le JS n'est exécuté qu'au moment du scroll.

Resonance - footer, commentaires clients, mentions légales - tout ce qui vit sous la ligne de flottaison.

hydrate-on-interaction

À l'action utilisateur - click - pointerenter - focusin

Le conteneur écoute les événements natifs. À la première interaction, charge le JS, hydrate, puis rejoue l'action.

Resonance - modale de contact, panneau de filtres latéral, bandeau cookies - masqués ou optionnels.

hydrate-on-media-query

Selon l'environnement - window.matchMedia

Hydratation conditionnelle à une media query CSS - taille d'écran, orientation, préférences système.

Resonance - carrousel tactile sur mobile uniquement, menu de filtres complexe au survol desktop uniquement.

Aucune n'exclut les autres : on les combine. Idle pour ce qui est visible et secondaire, visible pour le below-the-fold, interaction pour ce qui est masqué.

Synthèse : impact mesurable sur le TBT

Au lieu d'un pic de blocage au démarrage, le travail du processeur est lissé dans le temps

Stratégie	Déclencheur	Objectif métrique
hydrate-on-idle	requestIdleCallback	Améliorer le TBT sans retarder l'UX
hydrate-on-visible	IntersectionObserver	Éviter le coût JS des éléments non vus
hydrate-on-interaction	click - pointerenter	Zéro coût JS tant que c'est inutile
hydrate-on-media-query	window.matchMedia	Adapter la charge CPU au format de l'écran

Profil CPU au boot - avant et après

✗ Hydratation synchrone



✓ Lazy Hydration



Lissage - même CPU total consommé, mais étalé dans le temps. Le main thread est rendu disponible aux gestes utilisateur entre chaque hydratation.

Le TBT ne se réduit pas en faisant MOINS de travail - il se réduit en faisant le travail au BON moment.

Cas TicketSelector : hydrate-on-idle

Sur la fiche événement, l'utilisateur lit le hero - la billetterie peut attendre

```
<!-- pages/events/[slug].vue -->
<LazyTicketSelector
  hydrate-on-idle
  :event="event"
  :sessions="sessions"
/>
```

Sans lazy hydration

- TicketSelector hydraté immédiatement
- State local, computed, handlers setup
- Long task ~ 200 ms
- Main thread bloqué pendant le LCP

Avec hydrate-on-idle

- SSR - l'aside billetterie rendue (visible)
- Hydratation différée à requestIdleCallback
- Main thread libre pour le LCP
- Interactivité < 100 ms après LCP

Lab

Mise en œuvre

Sept étapes, 60-90 min - branche `solution/j2-bundle`, fiche docs/ateliers/j2-bundle.md

Les sept étapes du lab

Ordre suggéré - prérequis Node 22 (bump Dockerfile en étape 1)

1	Installer @nuxt/image + helper useImageSrc (réécriture URL server-side)	nuxt.config.ts + composables/
2	Migrer → <NuxtImg> Hero home, hero fiche, galerie, cards - presets adaptés	pages/, components/
3	Self-host Inter via @nuxt/fonts Preload poids 400, retirer <link> Google Fonts	nuxt.config.ts
4	Code splitting Chart.js Extraire SalesChart, defineAsyncComponent	components/SalesChart.vue
5	Lazy hydration TicketSelector Extraire en composant, <Lazy hydrate-on-idle>	components/TicketSelector.vue
6	Fix preload mismatch drop hero preload, fetchpriority="high" suffit	pages/index.vue
7	Mesurer make k6, make lighthouse, nuxi analyze	make + nuxi analyze

Métriques attendues - cible J2

Gains différentiels mesurés vs starter - cf. docs/benchmarks/j2-bundle-comparison.md

Métrique	Starter	Cible J2	Attendu
Lighthouse home Perf	0.55	> 0.65	0.66 - +20 %
LCP home	16.3 s	< 8 s	5.6 s - -66 %
FCP home (effet polices)	15.9 s	-	4.8 s - -70 %
Lighthouse fiche Perf	0.91	≥ 0.90	0.92 - stable
LCP fiche	3.0 s	< 3 s	2.9 s - parité
First Load JS organizer	~ 131 Ko gz	-	~ 75 Ko gz - -43 %
AVIF hero (1600×420)	283 Ko JPEG	-	~ 169 Ko - -40 %
AVIF card (480×270)	283 Ko JPEG	-	~ 26 Ko - -91 %

Smoke tests à dérouler en fin d'atelier

Six vérifications curl - fiche §7 pour les commandes complètes

1

IPX renvoie AVIF

`curl -H 'Accept: image/avif' /_ipx/... → Content-Type: image/avif`

2

Comparaison JPEG vs AVIF (card)

Mesurer taille fichier → attendu ~ -90 %

3

Polices self-hostées

`grep '/_fonts/.+\.woff2' → 1+ URL ; grep 'fonts.gstatic' → 0`

4

Chart.js dans chunk séparé

Inspecter `.output/public/_nuxt/*.js` → un fichier dédié ~ 162 Ko

5

TicketSelector rendu en SSR

view-source événement → `<aside...>` présent au load

6

Parcours visiteur fonctionnel

`POST /auth/login → GET /me/tickets` - pas de régression

Cap sur le lab

90 min

Branche

solution/j2-bundle

Fiche

`docs/ateliers/j2-bundle.md`

À débriefer ensuite

- Pourquoi FCP -70 % > LCP -66 % ?
- Pourquoi LCP reste à 5,6 s ?
- Comment composer j1 + j2 sur final ?

Module 3

Dashboard & refactor Vue

Réduire le coût main thread des interactions et du polling, sans toucher au backend

Branche

`solution/j2-dashboard`

Janus Academy - Ali Koudri

Transition - les deux premières moitiés du voyage

J1 a effacé le coût de transmission. J2 a réduit le payload. Reste l'interactivité côté Vue.

J1 - Cache HTTP

Compression, immutable, SWR Nitro, HTTP/2 → TTFB ~ 0 ms en cache hit

J2 - Payload

@nuxt/image, @nuxt/fonts, code splitting, lazy hydration → LCP -66 %

J2-dashboard - ici

Réactivité Vue 3, virtualisation, lifecycle → INP, TBT, FPS

Le pattern : charger vite (J1), livrer léger (J2), réagir rapidement après le load (J2-dashboard).

Observation starter : la page participants

/organizer/events/600/participants : un event star avec 7 000 tickets - le drame opérationnel

7 001

DOM <tr> total

42 062

DOM nodes total

~ 2 Mo

Table HTML transmis

9 572 ms

TBT pendant scroll 5 s

85

Longtasks pendant scroll

793 ms

Longtask max

25

FPS pendant scroll

104 ms

INP clic searchbox

Conclusion : ce n'est pas tant un problème de LCP qu'un problème d'interactivité.

Cas d'école : refacto comportementale $\neq$ refacto de load

Le point pédagogique central du module : adapter la métrique à la chose qu'on optimise

À garder en tête

Les métriques Lighthouse de load (LCP, FCP, CLS) seront stables avant/après. Ce n'est pas un échec, c'est la signature attendue.

Ce que J2-dashboard ne touche PAS

- Critical path du load
- LCP, FCP, CLS
- Score Performance Lighthouse
- Poids du payload initial

Ce que J2-dashboard améliore

- INP des interactions (clic, scroll, filter)
- TBT pendant le polling et le scroll
- Longtasks et FPS
- DOM count (effet structurel)

Refacto comportementale : state, lifecycle, virtualisation - opère sur les interactions, pas sur le load.

Les quatre leviers de J2-dashboard

Quatre attaques côté Vue 3 : complémentaires, indépendantes, toutes frontend

1

State éclaté + shallowRef

Décomposer le ref monolithique : réactivité de surface sur les collections

2

v-memo sur les rows

Mémorisation par champs lus : zéro travail Vue par tick sur rows inchangées

3

Virtualisation (vue-virtual-scroller)

RecycleScroller : $DOM \div 186$, scroll fluide, INP < 50 ms

4

onScopeDispose pour le lifecycle

Cleanup propre : forward-compatible avec composable usePolling()

Toutes côté Vue 3 frontend : pas de backend, pas d'infra, pas de caching HTTP.

Leviers 1 & 2

Réactivité Vue 3

Deux primitives pour éviter le travail inutile - shallowRef et v-memo

Le piège du state monolithique

Un ref pour tout l'état → invalidation globale à chaque tick de polling

```
// Pattern fréquent (et pas seulement en starter)
const dashboardState = ref<DashboardState>({
  stats: null,
  salesChart: [],
  events: [],
})

// À chaque tick : dashboardState.value = { stats, salesChart, events }
```

Ce qui se passe à chaque tick

- L'objet entier est réassigné → Vue invalides tous les watchers/computed qui lisent l'objet
- Re-rendering global, même pour les fragments inchangés
- Pour les arrays : `ref<T[]>` pose un Proxy par item au mount : coûteux sur grosses listes
- Vous payer la réactivité profonde dont vous n'avez pas besoin

State splitting : ne re-rendre que l'utile

Un seul gros state global = tout se réévalue à chaque changement. Éclater par responsabilité = chaque donnée vit sa vie

✗ Sans éclatement - le state monolithique

Vous mettez user + chartData + searchQuery + notifications dans un seul ref global.

L'utilisateur tape une lettre dans la recherche → Vue considère TOUT l'objet modifié → ré-évaluation du graph, du profil, des notifs alors qu'aucun n'a changé.

Résultat - sur-rendu inutile, micro-saccades.

✓ Avec éclatement - responsabilité unique

Chaque donnée a son propre ref. user, chartData, searchQuery, notifs sont indépendants.

L'utilisateur tape → seul searchQuery se réactive → le graph et le profil ne sont même pas notifiés. Le main thread reste libre.

Résultat - re-render ciblé, fluidité préservée.

L'analogie - l'interrupteur électrique

Sans éclatement - [INTERRUPTEUR GÉANT] → * * * * * (tout s'allume, même pour 1 lampe)

Avec éclatement - [int]→* [int]→* [int]→* [int]→* (un interrupteur par lampe)

Le state global n'est pas mauvais par nature, il est mauvais quand il regroupe des données qui n'ont rien à voir entre elles.

Avant et après : même contenu, deux organisations

Les mêmes données. La différence se voit dans ce qui se re-rend quand l'utilisateur tape une lettre

✗ Avant - state monolithique

```
// Tout centralisé
const dashboardState = ref({
  user: { name: 'Ali', role: 'Admin' },
  chartData: [12, 19, 3, 5],
  searchQuery: ''
})

// L'utilisateur tape 'a'
dashboardState.value.searchQuery = 'a'
// → tout dashboardState marqué dirty
```

Conséquence

Re-render du graph et du profil utilisateur, alors qu'aucun n'a changé.

Impact mesurable

Sur un Dashboard avec 5 composants lourds, passer du monolithique à l'éclaté divise par 4 à 6 le temps CPU de la frappe clavier => TBT immédiatement visible dans le Performance panel.

✓ Après - state éclaté

```
// Chaque donnée vit sa vie
const user = ref({ name: 'Ali', role: 'Admin' })
const chartData = ref([12, 19, 3, 5])
const searchQuery = ref('')

// L'utilisateur tape 'a'
searchQuery.value = 'a'
// → seul searchQuery marqué dirty
```

Conséquence

Le graph et le profil ne sont jamais notifiés. Re-render ciblé sur la barre de recherche uniquement.

Le même contenu de données, et pourtant deux profils CPU radicalement différents.

Trois règles pour éclater : localiser, diviser, isoler

Le state n'a pas vocation à être global par défaut, c'est une exception, pas une règle

1 Localiser

garder le state proche du composant

Une donnée utilisée par un seul petit composant - l'état ouvert/fermé d'un dropdown, le champ focus d'un form - n'a rien à faire dans le store global.

```
// dans Dropdown.vue
const isOpen = ref(false)
// pas useGlobalStore().isOpen
```

Le bon réflexe - commencer local, ne monter que si plusieurs composants en ont besoin.

2 Diviser

un store Pinia par domaine

Au lieu d'un useMainStore qui gère tout, créer un store par domaine métier : user, cart, products, notifications.

```
useUserStore()    // auth, profil
useCartStore()    // panier
useProductStore() // catalogue
```

Chaque store ne notifie QUE les composants qui en dépendent.

3 Isoler

props focalisées sur sous-composants

Un tableau de bord lourd doit déléguer chaque ligne / chaque chart à un sous-composant qui ne reçoit que ses propres data en props.

```
<DashboardRow
  v-for="row in rows"
  :data="row" :key="row.id"
/>
```

Une mise à jour sur une ligne ne force pas le recalcul de tout le tableau.

État global = ce qui est partagé entre plusieurs composants distants. Tout le reste vit localement, ou dans un store de domaine.

shallowRef vs ref : la granularité de la réactivité

Vue 3 trace chaque propriété d'un objet ref : pour 5000 items × 20 props, c'est 100 000 traps actifs en pure perte

Le problème

ref() enveloppe l'objet dans un Proxy profond. Chaque accès à une sous-propriété déclenche un tracking. Sur une liste lue mais jamais mutée en place, c'est 100 % de coût pour 0 % de bénéfice.

✗ ref() - réactivité profonde

```
import { ref } from 'vue'

const events = ref([])

// 5000 events × 20 props
// = 100 000 Proxy traps actifs
events.value.forEach(e => {
  e.timestamp // tracké
  e.user.name // tracké (Proxy imbriqué)
})
```

✓ shallowRef() - réactivité de référence

```
import { shallowRef } from 'vue'

const events = shallowRef([])

// Aucun tracking interne
// Seul .value = ... déclenche update
events.value.forEach(e => {
  e.timestamp // NON tracké
  e.user.name // NON tracké
})
```

Le pattern d'update - compatible style fonctionnel

```
events.value = [...events.value, newEvent] // ✓ déclenche shallowRef (nouvelle référence)
```

Règle : shallowRef quand on remplace, ref quand on mute en place. Le 80/20 des listes en Vue tombe dans la 1ère case.

shallowRef vs ref : quand le Proxy coûte cher

Réactivité profonde par défaut : utile, mais à payer

	ref<T>	shallowRef<T>
Réactivité	Profonde - Proxy récursif	De surface - réagit à .value
Mutation par item	Détectée (events.value[0].x = ...)	Non détectée
Réassignation	Détectée	Détectée
Coût au mount	Proxy posé sur chaque item	Aucun
Cas d'usage	Forms, contrôles partiels	Listes remplacées en bloc

Règle : si vous passez par `events.value = [...]` (remplacement entier) et jamais par `events.value.push()` ni `events.value[i].x = ...`, utilisez `shallowRef`.

Cas dashboard : éclatement du state (state splitting)

Du ref monolithique à 3 refs ciblés : diff minimal par tick

```
// Avant
const dashboardState = ref({
  stats: null,
  salesChart: [],
  events: [],
})
dashboardState.value = { ... }
```

```
// Après
const stats      = ref<Stats | null>(null)
const salesChart = shallowRef<SalesPoint[]>([])
const events     = shallowRef<Event[]>([])

stats.value      = statsResp.data
events.value     = eventsResp.data
```

Effets mesurables

- Chaque tick déclenche un re-render scope-limited : seuls les fragments qui lisent le ref modifié
- shallowRef sur events et salesChart : pas de Proxy par item au mount (économise dizaines de ms)
- Forward-compatible : factorisation future (composable, store) facilitée
- Code plus lisible : trois noms parlants au lieu d'une structure imbriquée

ref vs shallowRef : une question de profondeur

ref surveille tout. shallowRef ne surveille que le premier niveau

ref() - la maison connectée

Capteurs sur la porte, le frigo, les tiroirs - partout.

Vue creuse l'objet et place un Proxy de tracking sur chaque sous-propriété, à chaque niveau. Une mutation profonde déclenche un re-render.

```
[ user ]      • tracké
├── name      • tracké
├── stats     • tracké
│   ├── score • tracké
│   └── level  • tracké
└── profile   • tracké
    └── avatar • tracké
```

shallowRef() - la boîte fermée

Un seul capteur, sur la boîte. L'intérieur est invisible.

Vue trace UNIQUEMENT le changement de référence du .value.
Mutations internes ignorées, pour notifier, il faut remplacer toute la boîte.

```
[ user ]      • tracké
├── name      ○ NON tracké
├── stats     ○ NON tracké
│   ├── score ○ NON tracké
│   └── level  ○ NON tracké
└── profile   ○ NON tracké
    └── avatar ○ NON tracké
```

Le mot « shallow » dit tout : surveillance peu profonde, par opposition au « deep » de ref.

Deux scénarios : modifier en interne ou remplacer tout

La différence ne se voit qu'à l'usage : selon qu'on touche au contenu de la boîte ou qu'on change de boîte

Le setup commun

```
const userRef      = ref({      name: 'Ali', stats: { score: 10 } })
const userShallow = shallowRef({ name: 'Ali', stats: { score: 10 } })
```

Scénario A : modifier en interne

```
userRef.value.stats.score = 20      // ✓
userShallow.value.stats.score = 20  // ✗
```

- ✓ ref : écran mis à jour (deep tracking)
- ✗ shallowRef : changement en mémoire, mais Vue ne le voit pas

Scénario B : remplacer tout l'objet

```
userRef.value      = { name: 'Bob', ... } // ✓
userShallow.value = { name: 'Bob', ... } // ✓
```

- ✓ ref : écran mis à jour
- ✓ shallowRef : écran mis à jour AUSSI, on a changé le 1er niveau, le seul que Vue surveille

À retenir

shallowRef ne casse PAS la réactivité, il la limite au premier niveau. Si on remplace l'objet entier (pattern fonctionnel), shallowRef réagit comme ref, sans le coût.

Le pattern : shallowRef + remplacement intégral. Compatible avec un style fonctionnel (immutabilité, spread, .map).

Quand utiliser shallowRef : deux cas concrets

Données volumineuses lues puis remplacées en bloc, ou instances de bibliothèques tierces lourdes

1. Grosses listes en lecture seule

Dashboard, résultats d'API, exports PostgreSQL

Vous chargez 10 000 lignes depuis ton API. Vous les affichez dans un tableau, un graphique, une carte. Vous ne modifiez jamais une cellule à la main - au prochain refresh, vous remplacez tout.

```
const rows = shallowRef([])

// Au refresh : remplacement complet
async function refresh() {
  rows.value = await fetchRows()
  // 10 000 lignes, 0 Proxy interne
}
```

Gain : 0 trap interne au lieu de 50 000+

2. Instances de bibliothèques tierces

Leaflet, Chart.js, client Axios, instance D3

Vous stockez une carte Leaflet, un Chart.js, un client. Ces objets sont énormes, pleins de méthodes, de circular references et de DOM internes. ref dessus = soit Vue casse l'objet, soit les perfs s'effondrent.

```
import L from 'leaflet'

// L'instance reste intacte, sans Proxy
const map = shallowRef(null)

onMounted(() => {
  map.value = L.map('container').setView(...)
})
```

Gain : l'instance Leaflet reste fonctionnelle

Règle pratique : ref par défaut. shallowRef quand la donnée est volumineuse et remplacée en bloc, OU quand on stocke une instance tierce complexe.

Memoization : échanger du CPU contre de la mémoire

Cacher le résultat d'une fonction pure par ses arguments : si on rappelle avec les mêmes, on retourne le cache

Le principe

Une fonction pure renvoie toujours le même résultat pour les mêmes arguments. Si elle est coûteuse et appelée souvent avec les mêmes entrées, on stocke le résultat dans une Map indexée par les arguments.

computed()

Vue natif

Calcul dérivé de refs réactifs, dans un composant.

```
const total = computed(() =>
  items.value.reduce(
    (s, i) => s + i.price, 0))

// Recalcul SEULEMENT si
// items change
```

useMemoize()

VueUse

Fonction pure appelée plusieurs fois avec mêmes args, hors contexte réactif.

```
import { useMemoize }
  from '@vueuse/core'

const getUser = useMemoize(
  (id) => fetchUser(id))
```

Map manuelle

JS pur

Fonction critique hors Vue. Contrôle fin sur la stratégie d'éviction.

```
const cache = new Map()
function memo(fn) {
  return (k) => {
    if (!cache.has(k))
      cache.set(k, fn(k))
    return cache.get(k)
  }
}
```

Quand NE PAS memoizer : fonctions rapides (< 1 ms), arguments rarement répétés, mémoire contrainte (mobile). La memoization n'est pas gratuite, c'est un budget mémoire.

Mémoïsation : garder le résultat sous le coude

pourquoi recalculer ce qu'on sait déjà ? Garder en mémoire le résultat d'un calcul coûteux

Le principe

Une fonction mémoïsée tient un petit carnet associant ses paramètres à leur résultat. Si on lui repose la même question, elle lit son carnet au lieu de refaire le calcul.

L'analogie - le prof et l'élève

✗ Sans mémoïsation

Prof : « Combien font 347×89 ? »

Élève : (calcule 30 sec) « 30 883 »

Prof : « Et combien font 347×89 ? »

Élève : (recalcule 30 sec) « 30 883 »

→ *Énergie gaspillée à chaque répétition.*

✓ Avec mémoïsation

Prof : « Combien font 347×89 ? »

Élève : (calcule 30 sec, note dans son cahier) « 30 883 »

Prof : « Et combien font 347×89 ? »

Élève : (regarde son cahier) « 30 883 »

→ *Réponse instantanée la deuxième fois.*

La mémoïsation n'accélère pas le premier calcul - elle évite simplement de le refaire.

Le mécanisme : cache hit ou cache miss

Chaque appel suit l'un des deux chemins, selon que la réponse est déjà dans le carnet, ou pas encore

À chaque appel de la fonction

Première étape : regarder dans le carnet si on a déjà la réponse pour ces paramètres. Selon le verdict, deux chemins très différents.

✓ Cache hit : trouvé dans le carnet

carnet[args] existe déjà

Renvoie immédiatement le résultat stocké. Le processeur ne travaille presque pas.

```
if (n in carnet) {  
  return carnet[n]  // instantané  
}
```

→ **Coût : une lecture d'objet (< 1 µs)**

✗ Cache miss : absent du carnet

carnet[args] n'existe pas encore

Fait le vrai calcul, inscrit le résultat dans le carnet, puis renvoie la réponse.

```
const r = calculLourd(n)  // long  
carnet[n] = r             // stocke  
return r
```

→ **Coût : le calcul complet (ms à sec)**

À retenir

La mémorisation amortit son coût : le premier appel est aussi long qu'avant (miss), tous les suivants sont presque gratuits (hit). Plus on appelle avec les mêmes args, plus le ratio profite.

Code progressif : avant et après mémorisation

La même fonction, en deux versions - la classique sans mémoire, et la mémorisée avec son carnet

✗ **Sans mémoire : recalcule à chaque fois**

```
function calculerCarreLourd(n) {  
  // imaginons un calcul long...  
  console.log('Calcul lourd...')  
  return n * n  
}
```

```
calculerCarreLourd(5) // long  
calculerCarreLourd(5) // long  
calculerCarreLourd(5) // long
```

✓ **Avec mémorisation manuelle**

```
const carnet = {}  
  
function calculerCarreMemoise(n) {  
  if (n in carnet) return carnet[n]  
  console.log('Calcul lourd...')  
  carnet[n] = n * n  
  return carnet[n]  
}
```

```
calculerCarreMemoise(5) // long  
calculerCarreMemoise(5) // instantané ⚡
```

Sortie console - observer la différence

```
> Calcul lourd...  
> Calcul lourd...  
> Calcul lourd...
```

```
> Calcul lourd...  
> (rien)  
> (rien)
```

Trois étapes pour mémoriser - un dictionnaire, une vérification de présence, un stockage post-calcule.

v-memo : mémoization des rows

Vue compare la liste passée à v-memo et saute le diff si rien n'a changé

```
<tr
  v-for="ev in events.slice(0, 12)"
  :key="ev.id"
  v-memo="[ev.id, ev.title, ev.city, ev.status]"
>
  ...
</tr>
```

Sémantique

- Vue compare la liste entre les renders successifs
- Tant qu'aucune valeur ne change, Vue saute le diff entier du subtree (et son rendering)
- Sur la table dashboard polled toutes les 10 s - zéro travail par tick pour les rows inchangées
- Marche aussi sur n'importe quel élément, pas seulement les <tr> de v-for

Discipline v-memo : exactement les champs lus

La liste doit contenir toutes les valeurs réactives lues, ni plus, ni moins

Trop peu de champs

Risque

- Données stale affichées
- Le row ne se redessine pas quand un champ lu a changé
- Bug silencieux - pas d'erreur, juste de l'obsolète

Trop de champs

Risque

- Memoization trop restrictive
- Rerenders inutiles dès qu'un champ non-lu change
- Gain v-memo annulé en pratique

Cas Resonance

[ev.id, ev.title, ev.city, ev.status] : exactement les 4 champs lus dans le row. Quand l'API exposera ev.tickets_sold en final, la liste passera à 5.

computed() : la mémoïsation faite par Vue

Au lieu de recalculer dans le template, déclarer une valeur dérivée que Vue cache et invalide pour vous

✗ Sans computed : calcul inline dans le template

```
<template>
  <p>
    {{ user.firstName }}
    {{ user.lastName.toUpperCase() }}
  </p>
</template>
```

Conséquence

Le toUpperCase() est ré-exécuté à chaque re-render, même si user n'a pas changé.

✓ Avec computed : valeur dérivée déclarée

```
<script setup>
const fullName = computed(() =>
  `${firstName.value} ` +
  lastName.value.toUpperCase()
)
</script>
```

Conséquence

fullName n'est recalculé que si firstName ou lastName change. Sinon, cache.

Et dans le template, on l'utilise comme une variable normale

```
<template>
  <p>Utilisateur : {{ fullName }}</p>    <!-- pas d'appel de fonction visible -->
</template>
```

Une valeur dérivée doit toujours être un computed : pas une fonction appelée depuis le template.

Deux super-pouvoirs : dépendances et cache

Vue lit le code à l'intérieur du computed, en déduit les dépendances, et ne recalcule QUE si elles changent

1. Mise à jour automatique

Tracking de dépendances

Vue lit les `.value` accédés dans le getter. Il associe le `computed` à ces dépendances. Quand l'une d'elles change, le `computed` est marqué « périmé ».

```
const fullName = computed(() =>
  firstName.value + lastName.value
  // Vue note : dépend de
  //   firstName ET lastName
)

firstName.value = 'Bob' // fullName périmé
age.value = 40         // fullName intact
```

→ ***Vous n'avez rien à déclarer. Vue infère.***

2. Mise en cache

Mémoïsation jusqu'à invalidation

Tant qu'aucune dépendance ne change, le getter n'est PAS ré-exécuté. Le re-render lit la valeur cachée. Le processeur dort.

```
// Premier rendu
fullName // → 'Ali KOUDRI' (calcul fait)

// 50 re-renders plus tard
fullName // → 'Ali KOUDRI' (depuis cache)

// Aucun appel au getter durant ces 50
```

→ ***Le re-render ne coûte plus rien.***

Les deux mécanismes ensemble : un computed se met à jour quand il le faut, et reste muet le reste du temps.

Quand l'utiliser : trois patterns canoniques

Toute donnée dérivée d'une autre donnée est un computed en attente : filtrer, calculer, valider

1 Filtrer une liste

selon une recherche

L'utilisateur tape dans une barre de recherche. Vous voulez afficher uniquement les items qui matchent - sans muter ta source de données.

```
const filtered = computed(() =>
  products.value.filter(p =>
    p.name.includes(search.value)
  )
)
```

2 Calculer un total

agréger plusieurs champs

Un panier change ligne par ligne. Le total doit toujours refléter l'état courant, sans qu'on ait à le maintenir à la main.

```
const total = computed(() =>
  cart.value.reduce(
    (s, i) => s + i.price * i.qty,
    0
  )
)
```

3 Valider un formulaire

dérivé un booléen

Le bouton « Envoyer » doit être actif si et seulement si tous les champs sont valides. Un booléen dérivé est exactement ça.

```
const isValid = computed(() =>
  email.value.includes('@')
  && password.value.length >= 8
)
```

Le bon réflexe : dès qu'une donnée se calcule à partir d'une autre, ne l'écrivez pas en méthode - écrivez-la en computed.

Levier 3

Virtualisation

Ne rendre que les rows visibles - DOM divisé par 186 sur la page participants

Le coût d'une grosse liste en DOM

7 000 lignes en <tr> : ce que le navigateur doit gérer derrière

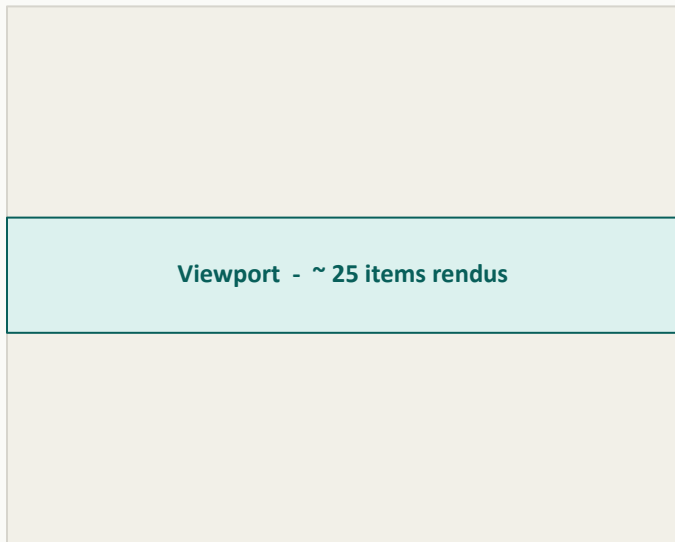
Parsing HTML	~ 2 Mo de markup à parser au chargement initial	<i>Coût initial</i>
DOM tree	42 062 nodes (avec spans, cells, attributs) - arbre énorme en mémoire	<i>Mémoire RAM</i>
Layout / paint	Recalcul de la disposition de toute la table à chaque scroll	<i>CPU sur scroll</i>
Hydration Vue	Chaque row monte un instance composant + handlers	<i>Longues tâches</i>
Interactions	Recherche, filtre, sort : re-évalue v-for de 7 000 items à chaque keystroke	<i>INP dégradé</i>

Le browser ne peut pas reculer - tout est dans le DOM, tout est calculé. Solution : ne rendre que ce qu'on voit.

Principe - windowing / virtual scrolling

Le browser ne rend qu'une fenêtre du dataset - recyclage des DOM nodes au scroll

Dataset - 7 000 items



Mécanisme

- Conteneur hauteur fixe (viewport)
- Conteneur scrollable de hauteur factice : $N \times \text{item_size}$
- Items rendus en position absolue selon scroll
- DOM nodes recyclés au scroll (pool)
- v-show ou re-render selon impl

Effet : ~ 25 rows en DOM au lieu de 7 000, scroll fluide, INP < 50 ms, mémoire divisée.

RecycleScroller : le DOM allégé par recyclage

10 000 lignes affichées, 25 balises HTML : la virtualisation transforme une liste géante en une liste constante

✗ Sans recyclage - v-for naïf

Vue injecte 10 000 balises <tr> ou <div> dans la page. L'écran n'en montre que 20 - les 9 980 autres existent quand même.

Conséquences - RAM saturée, scroll qui saccade (jank), recalculs CSS lents sur la moindre mise à jour.

→ **Le navigateur s'effondre.**

✓ Avec RecycleScroller - virtualisation

Le scroller crée ~25 balises HTML (20 visibles + marge). Quand une ligne sort en haut, elle est déplacée en bas et son contenu est remplacé.

Le navigateur a l'illusion d'une liste infinie. Sous le capot, jamais plus de 25 balises.

→ **DOM constant, scroll fluide.**

L'analogie : le tapis roulant de sushis

Sans recyclage : le chef prépare 10 000 assiettes et les pose sur un tapis qui traverse la ville. Structure s'effondre sous le poids.

Avec recyclage : 8 assiettes physiques. Dès qu'une assiette sort du champ de vision du client, le chef la récupère, change le sushi, et la repose au début du tapis. Illusion de choix infini, 8 assiettes en réalité.

Le recyclage : on ne crée plus, on déplace et on remplit.

Le mécanisme - 25 balises pour 10 000 lignes

Quand l'utilisateur défile, la ligne du haut est déplacée par CSS en bas - son contenu est simplement remplacé par les nouvelles données

Le cycle au défilement - trois étapes

1 Sort de l'écran

La ligne du haut quitte le viewport - elle devient invisible.

2 Déplacée par CSS

Au lieu d'être détruite, sa balise HTML est repositionnée en bas via transform.

3 Remplie

Vue remplace le texte et les données à l'intérieur par celles de la nouvelle ligne.

Le code - composant RecycleScroller + CSS associé

```
<template>
  <RecycleScroller
    class="scroller"
    :items="grossedListeDeLogs"
    :item-size="42"
    key-field="id"
    v-slot="{ item }"
  >
    <div class="user-row">{{ item.message }}</div>
  </RecycleScroller>
</template>
```

```
/* Hauteur du scroller :
   obligatoire */
.scroller {
  height: 500px;
}

/* Chaque ligne :
   hauteur EXACTE du item-size */
.user-row {
  height: 42px;
}
```

La perf vient de la division entre liste de données (10 000 objets en mémoire) et balises rendues (25 dans le DOM).

Les contraintes : hauteur fixe et accessibilité

Le RecyclerView a un prix : on ne peut pas faire ce qu'on faisait avec une <table> native, il faut accepter ses règles

1. Hauteur fixe : item-size

La math de position dépend de la hauteur de chaque ligne

Le scroller doit connaître la hauteur exacte en pixels pour calculer où repositionner chaque balise. Si tes lignes ont des hauteurs variables (texte qui wrap, expand/collapse), RecyclerView ne saura pas faire.

```
:item-size="42"      // ✓ hauteur connue

// Texte variable, expand/collapse
// → utiliser DynamicScroller (plus lourd)
```

Alternative : DynamicScroller mesure chaque ligne au runtime. Plus tolérant, plus coûteux en calcul.

2. Structure CSS et accessibilité

Pas de <table> HTML native possible

Le scroller utilise position: absolute et des transformations CSS pour bouger les lignes, incompatible avec la sémantique <table> / <tbody> / <tr>. La grille se construit en display: grid ou flexbox.

```
<div role="grid">           // pas <table>
  <div role="row"
    :aria-rowindex="i + 1">
  </div>
</div>
```

À retenir : ajouter role="grid", aria-rowindex, aria-rowcount pour que les lecteurs d'écran comprennent la structure.

RecyclerView ne s'utilise pas partout - il s'utilise là où la liste dépasse 100-200 items et où les lignes ont une hauteur prévisible.

Découverte : table → grille CSS

Contrainte technique imposée par RecycleScroller : préservation de l'accessibilité via ARIA

Le piège

- RecycleScroller positionne ses items en absolute
- `<tbody>` ne peut pas contenir d'enfants en position: absolute (browser engine spec)
- Mettre RecycleScroller dans une `<table>` casse le rendu

Le fix

- Passer en grille CSS : `display: grid`
- Conserver la sémantique avec `role="row"`, `role="cell"`, `role="columnheader"`
- Même rendu visuel, mêmes lecteurs d'écran

```
.participants-grid {  
  display: grid;  
  grid-template-columns: 140px minmax(140px, 1fr) minmax(180px, 1.5fr) 130px 90px;  
  align-items: center;  
}
```

Convention data-row : selector robuste au markup

Compter les rows matérialisées sans dépendre du nom de balise utilisé

```
// Dans la console DevTools, sur n'importe quelle liste virtualisée :  
document.querySelectorAll('[data-row]').length
```

Avantages

- Neutre vis-à-vis du markup : table, grid, list
- Survit aux refactor - passage <table> → <div>
- Compte uniquement les rows réellement matérialisées
- Convention documentée - benchmarks/README.md §4

Anti-pattern

- `document.querySelectorAll('tr').length`
- Donne 7 000 avant refactor
- Donne 0 après refactor (faux négatif)
- Conclusion : « la virtualisation ne marche pas »

Discipline : poser [data-row] sur chaque row de liste virtualisée. Devient un selector stable pour la mesure.

Hydration mismatch : le contrat SSR / CSR

Le serveur génère un HTML, le client le reprend et l'« hydrate ». Si les deux divergent = warnings, bugs visibles, ré-rendu

Le contrat SSR + hydration

1. Serveur génère le HTML (sans window, sans localStorage, sans état utilisateur). 2. Client reçoit ce HTML. 3. Vue l'« hydrate » - attache la réactivité, en supposant que SON premier rendu produira exactement le même HTML.

✗ v-if qui dépend du client

```
<template>
  <div v-if="window.innerWidth > 768">
    Desktop nav
  </div>
</template>

// SSR : window undefined → erreur
// ou v-if false côté serveur,
// true côté client → MISMATCH
```

✓ <ClientOnly> + useMounted()

```
<template>
  <ClientOnly>
    <div v-if="isDesktop">
      Desktop nav
    </div>
  </ClientOnly>
</template>

<script setup>
const isDesktop = useMediaQuery(
  '(min-width: 768px)'
)</script>
```

Sources classiques de mismatch - Date.now() ou Math.random() dans le template - window / document / navigator - localStorage / sessionStorage - données utilisateur non hydratées dans le store côté serveur

L'hydration mismatch n'est pas un warning à ignorer - c'est un bug en formation qui devient visible en production sur un autre navigateur.

ClientOnly + v-if : éviter les pièges SSR

Deux gardes nécessaires pour cohabiter avec Nuxt SSR - conditions de mount du scroller

```
<ClientOnly>
  <RecycleScroller
    v-if="participants.length"
    :items="participants"
    ...
  >
  ...
</RecycleScroller>
</ClientOnly>
```

<ClientOnly>

RecycleScroller instancie un ResizeObserver au mount - inexistant en environnement Node SSR.

Skip SSR rendering : la page est de toute façon CSR.

v-if="participants.length"

RecycleScroller émet des warnings sur items=[], garde le mount tant que les données ne sont pas là.

Pattern double : garde côté SSR + garde côté data.

Levier 4

Cycle de vie

Cleanup propre - forward-compatibilité avec les composables

onScopeDispose : nettoyer les effets, partout

onUnmounted ne marche que dans setup(). Les composables ont besoin d'un hook qui marche aussi dans les sous-scopes

Le problème

Un composable crée des effets de bord (setInterval, addEventListener, WebSocket). Si onUnmounted est appelé hors du contexte setup(), par exemple dans un composable imbriqué, ou via un effectScope, il échoue silencieusement. Résultat : memory leaks, écho de listeners, requêtes orphelines.

✗ onUnmounted - contexte setup() obligatoire

```
function useTimer(cb, ms) {
  const id = setInterval(cb, ms)

  // ⚠ Marche dans setup()
  // ⚠ NE MARCHE PAS si le composable
  //   est appelé dans un sub-scope
  //   (effectScope, async setup)
  onUnmounted(() => clearInterval(id))
}
```

✓ onScopeDispose - tout effectScope

```
function useTimer(cb, ms) {
  const id = setInterval(cb, ms)

  // ✓ Marche partout : setup,
  //   composable imbriqué,
  //   effectScope manuel,
  //   branches conditionnelles
  onScopeDispose(() => clearInterval(id))
}
```

À retenir

Dans un composable, utilisez onScopeDispose plutôt que onUnmounted. C'est strictement plus général : pas de cas où onUnmounted marche mais onScopeDispose pas.

onScopeDispose : cleanup rattaché à l'effect scope

L'alternative Vue 3 à onBeforeUnmount pour les ressources type setInterval, observers, timers

```
// Avant
let pollHandle: ReturnType<
  typeof setInterval> | null = null

onMounted(() => {
  loadAll()
  pollHandle = setInterval(loadAll, 10_000)
})
onBeforeUnmount(() => {
  if (pollHandle) clearInterval(pollHandle)
})
```

```
// Après

onMounted(() => {
  loadAll()
  const id = setInterval(loadAll, 10_000)
  onScopeDispose(() => clearInterval(id))
})
```

Avantages

- id capturé par closure : plus de variable mutable hors scope, code plus idiomatique
- Rattaché à l'effect scope courant, pas au lifecycle du composant racine
- Forward-compatible avec une factorisation en composable usePolling()

onUnmounted vs onScopeDispose : deux portées

L'un est prisonnier d'un composant visuel, l'autre suit le scope de réactivité : partout où il vit

✗ onUnmounted : prisonnier du composant

Se déclenche uniquement quand un composant visuel est démonté de l'écran.

Hors composant (store Pinia, plugin, effectScope manuel), il ne s'exécute jamais. L'outil reste actif en mémoire pour toujours.

→ Fuite silencieuse, leak garanti.

✓ onScopeDispose : universel

Se déclenche dès que le scope de réactivité où il a été créé est détruit.

Composant démonté, store Pinia réinitialisé, effectScope arrêté : partout où il y a un scope, il y a un nettoyage.

→ Nettoyage garanti, où qu'on l'appelle.

L'analogie : chambre d'hôtel ou bail de location

onUnmounted : chambre d'hôtel. Le nettoyage se fait quand le client quitte la pièce physique. Si le service est utilisé dans un bureau ouvert (un store global), personne ne vient - il n'y a pas de chambre.

onScopeDispose : contrat de location. Le nettoyage se déclenche dès que le contrat s'arrête, peu importe le lieu - chambre, bureau, espace virtuel. Le contrat précède le lieu.

Le scope n'est pas le composant, c'est l'unité de vie de la réactivité, indépendamment de son support visuel.

Le scope réactif : l'unité de vie d'un effet

Un scope est un contexte créé par Vue pour suivre les effets réactifs et les nettoyer en bloc à sa destruction

Définition

Un scope réactif (effectScope) regroupe tous les watch, computed, listeners et timers créés à l'intérieur. Quand le scope est détruit, Vue exécute tous les onScopeDispose enregistrés.

Trois contextes où onScopeDispose triomphe sur onUnmounted

1 Composant Vue

le scope par défaut

Un composant Vue a son propre scope. onScopeDispose s'y déclenche au démontage, exactement comme onUnmounted.

Comportement identique : les deux marchent.

2 Store Pinia

globalement actif

Un store Pinia n'est pas un composant. Il vit globalement. Si on y appelle un composable avec onUnmounted, il ne se déclenche jamais.

Seul onScopeDispose nettoie au \$dispose du store.

3 effectScope() manuel

découper la réactivité

effectScope() crée un scope autonome qu'on stoppe à la main avec .stop(). Utile pour des modules réactifs sans support visuel.

onScopeDispose se déclenche à .stop().

Tout ce qui marche avec onUnmounted marche avec onScopeDispose. L'inverse n'est pas vrai.

Le composable de polling : avant et après

Un timer qui doit s'arrêter proprement. Selon que `useTimer()` est appelé dans un composant ou dans un store, le hook change tout

✗ Avec `onUnmounted` : marche dans un composant

```
import { onUnmounted } from 'vue'

export function useTimer(cb) {
  const id = setInterval(cb, 2000)

  // ⚠ Ne marche que si useTimer()
  //   est appelé dans setup() d'un
  //   composant visuel.
  onUnmounted(() => clearInterval(id))
}
```

✓ Avec `onScopeDispose` : marche partout

```
import { onScopeDispose } from 'vue'

export function useTimer(cb) {
  const id = setInterval(cb, 2000)

  // ✓ Composant démonté → stop
  // ✓ Store Pinia $dispose → stop
  // ✓ effectScope .stop() → stop
  onScopeDispose(() => clearInterval(id))
}
```

Pourquoi ça compte : le cas Pinia store

```
// stores/notifications.ts
export const useNotifStore = defineStore('notif', () => {
  useTimer(refresh) // ← avec onUnmounted : leak. avec onScopeDispose : nettoyé au store.$dispose()
})
```

Règle absolue dans un composable : toujours `onScopeDispose`. Pas de cas où `onUnmounted` gagne, seulement des cas où `onScopeDispose` vous sauve.

usePolling() : l'objectif final

Pourquoi onScopeDispose plutôt qu'onBeforeUnmount ? Pour pouvoir factoriser

```
// composables/usePolling.ts
export function usePolling(fn: () => void, interval: number) {
  onMounted(() => {
    fn()
    const id = setInterval(fn, interval)
    onScopeDispose(() => clearInterval(id))
  })
}

// Usage dans n'importe quel composant ou composable :
usePolling(loadAll, 10_000)
```

Pourquoi ça marche

onScopeDispose se rebind au scope du callsite, donc le composable propage son cleanup naturellement. Avec onBeforeUnmount, ça ne fonctionnerait que dans le composant racine, pas dans un composable imbriqué.

usePolling : un composable qui se nettoie tout seul

Le polling naïf produit des memory leaks, des race conditions et des fetch orphelins. Le bon pattern fait tout en sécurité

✗ **Polling naïf - setInterval brut**

```
function setupPolling() {
  setInterval(async () => {
    const data = await fetch(
      '/api/status'
    ).then(r => r.json())
    // ...
  }, 5000)
}

// ⚠ jamais clearInterval
// ⚠ requête non annulée au démontage
// ⚠ pas de pause si onglet inactif
// ⚠ race condition si tick > 5s
```

✓ **usePolling - composable safe-cleanup**

```
function usePolling(url, ms) {
  const data = ref(null)
  let controller

  const id = setInterval(async () => {
    controller?.abort()
    controller = new AbortController()
    const r = await fetch(url, {
      signal: controller.signal })
    data.value = await r.json()
  }, ms)

  onScopeDispose(() => {
    clearInterval(id)
    controller?.abort()
  })
  return data
}
```

Cycle de vie - mount → setInterval démarré → onScopeDispose enregistré → unmount → cleanup intégral. Une seule règle, deux garanties (pas de leak, pas de race).

Tout composable qui ouvre une boucle, un listener ou une socket doit fermer dans onScopeDispose. Pas de "je le ferai plus tard".

usePolling - pourquoi en faire un composable

Polling de logs PG, statut de job, notifications - un setInterval brut expose à deux pièges classiques qu'un composable règle une fois pour toutes

Le besoin courant

Resonance polls les logs PostgreSQL toutes les 2-3 secondes pour rafraîchir le dashboard. Ou surveille le statut d'un job Horizon. Ou vérifie les nouvelles notifications. Trois besoins, un seul pattern.

A - Chevauchement de requêtes

L'API est ralentie (4 sec), le polling tourne à 2 sec.

Sans protection - empilement de requêtes parallèles. Le navigateur ouvre des connexions en cascade, le serveur Laravel subit un mini-DDOS de ses propres clients, les réponses arrivent dans le désordre.

```
if (isExecuting) return // sentinelle
// skip si tour précédent pas fini
```

→ *Le tick courant est ignoré poliment.*

B - Memory leak

L'utilisateur navigue vers une autre page Nuxt.

Sans protection - le composant est détruit, mais setInterval reste actif dans le moteur JS. Il continue de harceler l'API Laravel en arrière-plan, pour rien - invisible et persistant.

```
onScopeDispose(stop) // cleanup auto
// scope détruit → stop déclenché
```

→ *Le timer s'éteint dès la sortie du scope.*

Un composable encapsule la sentinelle et le cleanup une fois pour toutes. Vous écrivez la logique métier, pas la mécanique.

Le composable complet - version expert

Interface TypeScript, sentinelle anti-chevauchement, API publique start/stop, cleanup automatique via onScopeDispose

Logique principale - tick + start

```
import { ref, onScopeDispose } from 'vue'

export function usePolling(
  callback,
  { interval = 5000, immediate = true } = {}
) {
  const isActive = ref(false)
  let timerId = null
  let isExecuting = false // ① sentinelle

  const tick = async () => {
    if (isExecuting) return // ② skip
    isExecuting = true
    try { await callback() }
    catch (e) { console.error(e) }
    finally { isExecuting = false }
  }
}
```

Cycle de vie - stop + cleanup auto

```
const start = () => {
  if (isActive.value) return
  isActive.value = true
  if (immediate) tick()
  timerId = setInterval(tick, interval)
}

const stop = () => {
  if (!isActive.value) return
  isActive.value = false
  if (timerId) clearInterval(timerId)
}

onScopeDispose(stop) // ③ cleanup

return { isActive, start, stop }
}
```

① sentinelle isExecuting - bloque le chevauchement | ② tick async sécurisé - try/catch/finally | ③ onScopeDispose - nettoyage universel (composant, store Pinia, effectScope)

Usage dans un composant : l'API publique

Trois lignes pour brancher le polling. Le composable cache toute la mécanique du timer, du cleanup et de la sentinelle

Côté composant : fichier .vue

```
<script setup lang="ts">
import { ref } from 'vue'
import { usePolling } from '@composables/usePolling'

const logs = ref([])
const fetchNewLogs = async () => {
  logs.value = await $fetch('/api/logs/latest')
}

const { isActive, start, stop } = usePolling(fetchNewLogs, { interval: 3000 })
start()
</script>

<template>
  <p>Statut : {{ isActive ? 'Actif' : 'En pause' }}</p>
  <button @click="isActive ? stop() : start()">Toggle</button>
</template>
```

Pour aller plus loin : à grande échelle, sortir du polling

Le short polling reste un sondage côté client. Pour des milliers d'utilisateurs simultanés, l'infra Octane / Laravel sature sous le nombre de connexions HTTP. Passer aux WebSockets ou Server-Sent Events branchés sur un broker (Kafka, Redis Streams) - solution événementielle, ressources serveur divisées par 10 à 100.

Polling = bon pour démarrer. WebSocket = la vraie réponse quand le volume devient sérieux. Le composable usePolling vous permet d'évoluer sans casser l'API consommatrice.

Optimiser pour aujourd'hui ET pour demain

La perf comme propriété forward-compatible : prévenir les régressions futures

shallowRef	Économise dizaines de ms au mount	<i>Évite la régression quand l'API events exposera plus de champs (j3-laravel)</i>
v-memo	Zéro travail Vue par tick sur les rows inchangées	<i>Prêt à intégrer <code>ev.tickets_sold</code> quand exposé sur final</i>
RecycleScroller	DOM ÷ 186, scroll fluide à 7 000 rows	<i>Scale à 50 k / 100 k rows sans dégénérer</i>
onScopeDispose	Code plus idiomatique, capture par closure	<i>Permet la factorisation <code>usePolling()</code> - pattern réutilisable</i>

La perf n'est pas un état, c'est une propriété qui se maintient dans le temps, à mesure que le code évolue.

Mesure

L'interactivité

Adapter la métrique à la chose qu'on optimise - pas LCP/FCP, mais INP/TBT/FPS

Pas LCP/FCP, mais INP/TBT

Les métriques Lighthouse Load mesurent le chargement, pas ce qui se passe après

Lighthouse Load metrics

Mesurent - ce qui se passe AU CHARGEMENT

- LCP - Largest Contentful Paint
- FCP - First Contentful Paint
- CLS - Cumulative Layout Shift
- TTFB - Time To First Byte

Interactivity metrics

Mesurent - ce qui se passe APRÈS le chargement

- INP - Interaction to Next Paint
- TBT - Total Blocking Time
- Longtasks - tâches > 50 ms
- FPS - Frames Per Second pendant interaction

Erreur classique - « on a refait le dashboard, Lighthouse n'a pas bougé, c'est nul » : faux, Lighthouse ne mesure pas tout ce qu'on optimise.

Les bonnes métriques d'un refacto comportemental

Cinq dimensions à instrumenter pendant le scroll, le polling, le filtrage

INP	Interaction to Next Paint - worst-case latency d'une interaction	< 200 ms
TBT	Total Blocking Time - somme des longtasks > 50 ms sur une fenêtre	< 200 ms
Longtasks	Tâches > 50 ms qui bloquent le main thread - count + max durée	0 (idéal)
FPS	Frames Per Second pendant scroll / drag / animation	≥ 60
DOM count	Nombre de nodes matérialisés dans le DOM - effet structurel	Stable

Tous instrumentables dans DevTools : Performance tab - Record + interaction + lecture des longtasks et de la timeline.

Lab

Mise en œuvre

Quatre étapes, 60-90 min : [branche solution/j2-dashboard](#), [fiche docs/ateliers/j2-dashboard.md](#)

Les quatre étapes du lab

Ordre suggéré : shallowRef d'abord (sous-tend les autres leviers)

1

Éclater le state monolithique

ref + shallowRef × 3 - stats / salesChart / events

pages/organizer/dashboard.vue

2

v-memo sur la table d'événements

v-memo="[ev.id, ev.title, ev.city, ev.status]"

pages/organizer/dashboard.vue

3

Virtualiser la table participants

RecycleScroller + grille CSS + ClientOnly + v-if

pages/.../participants.vue

4

Polling lifecycle via onScopeDispose

Remplace let pollHandle - capture par closure

pages/organizer/dashboard.vue

Les marqueurs @perf-debt en starter pointent vers les zones à traiter - à convertir en @perf-fix après l'optimisation.

Vérifications : ce qu'il faut observer

Cinq checkpoints : compile, build, comportement visuel, DOM count, perf

1

Le code compile

`make frontend-typecheck`

2

Le build OK et démarre

`make frontend-rebuild`

3

Polling 10s sans jank visible sur le chart dashboard

Navigateur - `/organizer/dashboard`, observer 30 s

4

Scroll fluide sur 7 000 rows

Navigateur - `/organizer/events/600/participants`

5

DOM count stable au scroll

Console - `document.querySelectorAll('[data-row]').length` → ~25-30

6

Longtasks ~ 0 pendant scroll

DevTools - Performance tab, Record, scroll 5 s, lire les longtasks

Métriques attendues : cible J2-dashboard

Sur /organizer/events/600/participants (7 000 tickets) - cf. docs/benchmarks/j2-dashboard-comparison.md

Métrique	Starter	J2-dashboard	Δ
DOM <tr> total	7 001	0	-7 001
DOM [data-row] viewport	0	~ 23	cible ~30 
DOM nodes total	42 062	226	-99.5 %
Table HTML bytes	~ 2 Mo	17 522	÷ 114
Longtasks pendant scroll 5 s	85	0	-100 %
TBT pendant scroll 5 s	9 572 ms	0 ms	-100 %
Longtask max	793 ms	0 ms	-100 %
FPS pendant scroll	25	85	+240 %
INP clic searchbox	104 ms	24 ms	-77 %

Cap sur le lab

90 min

Branche

solution/j2-dashboard

Fiche

`docs/ateliers/j2-dashboard.md`

À débriefer ensuite

- Pourquoi LCP est stable, et c'est OK ?
- Pourquoi <table> passe en grille CSS ?
- Comment scale ce pattern à 50 k rows ?

Module 4

Laravel & backend

Réduire le TTFB API, le bootstrap PHP et le coût HTTP du tunnel d'achat

Branche

`solution/j3-laravel`

Janus Academy - Ali Koudri

Transition : le voyage côté serveur commence

J1, J2 et J2-dashboard ont saturé l'optim côté navigateur. Le TTFB API n'a pas bougé.

J1 : Cache HTTP

Compression, immutable, SWR Nitro, HTTP/2 → TTFB ~ 0 ms sur cache hit

J2 : Payload

@nuxt/image AVIF, @nuxt/fonts, code splitting, lazy hydration → LCP -66 %

J2-dashboard : Vue 3

shallowRef, v-memo, virtualisation, onScopeDispose → INP < 50 ms, DOM ÷ 186

J3-laravel : ici

Eager, cache Redis, queues, cursor, Octane, OPcache+JIT → TTFB API, throughput

Le pattern : charger vite (J1), livrer léger (J2), réagir rapidement (J2-dashboard), répondre vite (ici).

Observation starter - côté backend

k6 sur substrat prod-like (Nginx + PHP-FPM + Nuxt prod build) - 20-30 VUs, dataset canonique

135 ms

/api/v1/events médian

2.9 s

/api/v1/events p95

5.3 s

Home SSR p95 (20 VUs)

1.39 s

Tunnel achat médian 201

49

Requêtes SQL /events

~ 4 Mo

Payload /events

~ 94/s

Throughput tunnel

FPM neuf

Bootstrap PHP / requête

Conclusion : TTFB API plombé par N+1, payload massif et bootstrap PHP à chaque requête.

Cas d'école : différentiel vs absolu

Le point pédagogique central du module : la signature attendue d'optims qui se composent

À garder en tête

Le TTFB API s'effondre (−99.8 %) MAIS le LCP home reste à 10.4 s. Ce n'est pas un échec de J3, c'est exactement le scope hors-périmètre.

Ce que J3-laravel résout (vs starter)

- /events p95 - 2.9 s → 6.3 ms (−99.8 %)
- Home SSR p95 - 5.3 s → 19.9 ms (−99.6 %)
- TTFB Lighthouse - 2.3 s → 18 ms (−99 %)
- Throughput tunnel - 94/s → 644/s (×6.8)
- SQL /events - 49 → 3 (÷ 16)

Ce que J3-laravel ne touche PAS

- LCP home 10.4 s - image hero JPEG full-size (j2-bundle + j1-cdn-cache)
- Score Perf 0.68 - dépend du LCP, donc même cause
- ILIKE long-tail - seq scan FTS (j3-postgres)
- failed_rate checkout 98.66 % - pas de SELECT FOR UPDATE SKIP LOCKED

Mesurer une branche isolée au regard les cibles absolues donne un faux négatif systématique.

Le coût d'une connexion : HTTP et base de données

Une requête n'est jamais gratuite - au-delà du handshake, il y a deux niveaux de connexion à gérer

Deux niveaux, deux poolings

Front ↔ Backend : keep-alive HTTP, multiplexing HTTP/2. - Backend ↔ Base : connection pooling applicatif. Sans ces deux mécanismes, chaque requête paye un coût d'ouverture complet.

HTTP : réutiliser la connexion TCP / TLS

✗ Connexion neuve à chaque requête

TCP handshake + TLS handshake répétés.

Pour 50 ressources d'une page :

$50 \times (\text{TCP} + \text{TLS}) = \sim 50 \times 200 \text{ ms}$

= 10 secondes en pure ouverture.

✓ Keep-alive + HTTP/2 multiplexing

1 seule connexion TCP / TLS pour toute la session.

HTTP/2 : N requêtes parallélisées sur la même connexion.

Coût d'ouverture amorti sur des dizaines de requêtes.

Base de données : pool de connexions applicatif

✗ Connexion ouverte à chaque query

Fork process, allocation mémoire, auth SCRAM, négociation TLS - 5 à 20 ms par ouverture, × chaque query applicative.

✓ Pool persistant (driver applicatif)

PDO::ATTR_PERSISTENT, Laravel keep-alive - connexion réutilisée entre les queries d'une même requête HTTP.

Ces deux niveaux préparent le terrain pour le 3ème - un pooler externe (PgBouncer) qui mutualise entre tous les workers backend. Vu en Module 5.

Les six leviers de J3-laravel

3 sur la lecture, 1 sur l'asynchrone, 2 sur le runtime PHP - tous backend

1

Eager loading + Resources whenLoaded

Tuer les N+1 sur les chemins lecture - 49 → 3 SQL sur /events

2

Cache Redis applicatif avec tags

Cache::tags(['events'])->remember - TTL 60/300/30 s, tag-flush sur writes

3

Pagination cursor sur /api/v1/events

cursorPaginate(20) - payload ÷ 50 (4 Mo → 80 Ko), cache borné

4

Queues Redis + Horizon

Déporter PDF dompdf + SMTP hors thread HTTP - mock paiement reste sync

5

Octane + FrankenPHP

Workers persistants - fin du bootstrap Laravel à chaque requête

6

OPcache + JIT tracing 64 Mo + optimize

Bytecode compilé une fois, hot paths JIT, config/route/view/event:cache

Aucune modification du frontend, aucune migration DB - périmètre strictement backend.

Middleware Laravel : intercepter la requête, en chaîne

Auth, log, CSRF, throttle, headers : des concerns transverses qui ne doivent pas vivre dans les controllers

Le problème

Sans pattern dédié, les vérifications (auth, validation CSRF, parsing JSON, rate limiting) finissent dans chaque controller. Duplication, oublis, ordre non garanti.

Le pattern : un middleware, une responsabilité

```
class LogRequests
{
    public function handle($request, Closure $next)
    {
        $start = microtime(true);

        $response = $next($request); // → chaîne

        Log::info('req', [
            'route' => $request->path(),
            'duration_ms' => (microtime(true) - $start) * 1000,
        ]);
        return $response;
    }
}
```

Trois niveaux d'application

Global : app/Http/Kernel.php
→ toute requête (CORS, trust proxies)

Groupe : 'web', 'api'
→ par contexte (CSRF, throttle:api)

Route : middleware('auth')
→ par endpoint (auth, can:edit)

handle() vs terminate()

handle() s'exécute AVANT le controller (peut court-circuiter via return). terminate() s'exécute APRÈS l'envoi de la réponse - idéal pour les logs, métriques, jobs asynchrones.

L'ordre des middlewares dans Kernel.php est l'ordre d'exécution. « auth puis throttle » ≠ « throttle puis auth ».

Leviers 1, 2 et 3

Lecture & cache

Trois leviers couplés - eager loading, pagination cursor, cache Redis avec tags

Le problème N+1 : l'ORM qui multiplie les requêtes

Une boucle qui accède à une relation lazy = une requête SQL par item. 100 users = 101 queries

Le scénario qui tue

Vous listez 100 users avec leurs commandes. Eloquent fait 1 requête sur users, puis pour chaque user une requête sur orders. Pour 1000 users → 1001 queries. Le serveur DB s'écroule, le TTFB explose.

✗ Code qui déclenche le N+1

```
$users = User::all();

foreach ($users as $user) {
    echo $user->orders->count();
    //      ^^^^^^^^^
    // chaque accès = 1 query SQL
}

// 100 users
// → 1 + 100 = 101 requêtes
```

Log SQL résultant

```
-- Query 1
SELECT * FROM users;

-- Query 2
SELECT * FROM orders
  WHERE user_id = 1;

-- Query 3
SELECT * FROM orders
  WHERE user_id = 2;

-- ... x100
```

Détection en dev -

`Model::preventLazyLoading()` dans `AppServiceProvider` - toute relation non chargée explicitement déclenche une exception. Le N+1 devient un test, pas une découverte en prod.

Le coût du N+1 : 49 requêtes pour une liste

Sur /api/v1/events index, sans with() : query log Laravel mesure tout

```
// EventController@index - starter
$events = Event::where('status', 'published')->get();

// EventResource : à chaque event, accès $event->organizer + $event->media
// → 1 + N (organizer) + N (media) ≈ 49 requêtes pour 24 events publiés
```

Le mécanisme caché

- Eloquent charge lazy par défaut : l'accès relation déclenche un SELECT
- Multiplié par le nombre d'items et le nombre de relations lues
- Latence cumulée : chaque requête paye le round-trip TCP vers Postgres
- Invisible à l'œil : l'API marche, juste lente. Mesure : `DB::enableQueryLog()` puis `count(DB::getQueryLog())`.

Eager loading + whenLoaded : charger, exposer

Charger les relations en lot avec with(). Les exposer en JSON uniquement quand elles sont vraiment chargées avec whenLoaded()

1. Eager loading : charger en 2 requêtes au lieu de N+1

```
// Au moment du fetch
$users = User::with('orders')->get();

// Après coup, sur une collection existante
$users->load('orders');

// Compter sans charger
$users = User::withCount('orders')->get();
```

```
-- 2 requêtes au total
SELECT * FROM users;

SELECT * FROM orders
  WHERE user_id IN (1, 2, 3, ..., 100);

-- Eloquent assemble côté PHP
```

2. whenLoaded : n'inclure la relation que si elle est chargée

```
// UserResource
public function toArray($request)
{
    return [
        'id' => $this->id,
        'name' => $this->name,
        'orders' => OrderResource::collection(
            $this->whenLoaded('orders')
        )
    ];
}
```

Pourquoi whenLoaded ?

Sans whenLoaded : la sérialisation accède à \$this->orders et redéclenche un N+1 silencieux dans l'API Resource elle-même.

Avec whenLoaded : la clé orders n'apparaît dans le JSON que si la relation a été eager-loaded en amont.

Règle : toujours déclarer ce qu'on charge. Le contrat with() + whenLoaded() rend le N+1 impossible par construction.

Eager loading + Resources whenLoaded()

Deux moitiés du même remède : charger ce qu'on lit, n'exposer que ce qu'on charge

```
// EventController@index - 49 → 3 requêtes
$query = Event::query()
    ->with(['organizer', 'media' => fn ($q) => $q->orderBy('position')])
    ->where('status', Event::STATUS_PUBLISHED);
```

```
// EventResource - n'expose la relation que si chargée
'organizer' => new OrganizerResource($this->whenLoaded('organizer')),
'media'      => MediaResource::collection($this->whenLoaded('media')),
```

whenLoaded() : si la Resource est utilisée ailleurs sans with(), elle ne re-déclenche pas le N+1.

Cache Redis applicatif : le problème avant l'outil

Avant d'introduire Redis, savoir précisément quel problème il résout, et les pièges qu'il introduit

Le scénario

Une query coûteuse (jointures multiples, full-text, agrégation) sert un endpoint très demandé (homepage, dashboard). Pour 10 000 visiteurs/min, c'est 10 000 fois le même travail SQL.

Cache-Aside

Read-through

L'app interroge Redis. Miss → query DB → écrit dans Redis → renvoie. Le plus courant en Laravel.

```
Cache::remember(
    'home',
    300,
    fn () => $this->compute()
);
```

Write-Through

Sync write

Toute écriture en DB met aussi à jour Redis dans la même transaction. Cohérence forte, latence d'écriture élevée.

```
DB::transaction(function () {
    $u->save();
    Cache::put(...);
});
```

Write-Behind

Async write

On écrit dans Redis, un job asynchrone synchronise vers la DB. Lecture / écriture rapides, risque de perte si Redis tombe.

```
Cache::put('k', $v);
SyncToDb::dispatch(
    'k', $v
);
```

Le piège : stampede

À l'expiration d'une hot key, N requêtes simultanées trouvent un miss et lancent N fois la query coûteuse en parallèle. Pic DB violent. Solution - Cache::flexible() (SWR) ou lock autour de la revalidation.

Le cache applicatif protège la DB, mais déplace les problèmes - invalidation, stampede, cohérence. Pas de solution sans contrepartie.

Cache Redis applicatif : Cache::tags + clé par filtres

TTL différenciés - 60 s liste, 300 s fiche, 30 s stats organizer

```
// EventController@index
$payload = Cache::tags(['events'])->remember(
    'events:index:'.md5(serialize($filters).'|cursor='.$cursor),
    60,
    fn () => [
        'data' => EventResource::collection($paginator->items())->resolve(),
        'meta' => [...],
    ]
);
```

La clé encode tout ce qui change la réponse

- Filtres : q, city, category, from, to → hash md5 pour clé courte
- Cursor : paginate-friendly - chaque page est une entrée Redis distincte
- Connexion Redis dédiée : db 1 pour cache, db 0 réservée aux queues
- TTL court : 60 s suffit pour amortir un pic, sans empiler le risque d'obsolescence

Découverte : cacher la réponse JSON, pas l'objet

Laravel 11+ pose `serializable_classes = false` par défaut - prévention du gadget chain

Ce qui casse

- `Cache::remember(..., fn () => $paginator) → __PHP_Incomplete_Class` au reload
- `Eloquent\Collection` cachée → même symptôme
- Cause : `cache.serializable_classes = false` rejette les classes au unserialize

Ce qui marche

- Cacher un array sérialisable
- `EventResource::collection(...)->resolve()` → retourne un array
- `Collection<stdClass> brute` → cast en array (`StatsController@salesChart`)

```
// La forme correcte  
'data' => EventResource::collection($paginator->items())->resolve(),
```

Tag-flush : pourquoi pas l'invalidation par clé

Stratégie d'invalidation côté writes organizer : `Cache::tags(['events']->flush()`

Key-by-key - impossible

- Clé : `events:index:{md5(filtres|cursor)}`
- Espace possible : $q \times \text{city} \times \text{category} \times \text{from} \times \text{cursor}$
- Non borné en pratique : impossible à énumérer côté Laravel

Tag-flush - $O(1)$ côté Redis

- Redis stocke le set de clés taggées
- Flush du tag : invalide tout le set en une opération
- Granularité : `['events']` global, `['organizer:42']` ciblé

```
// Organizer/EventController@store/update/destroy
Cache::tags(['events']->flush();
Cache::tags(["organizer:{$event->organizer_id}"]->flush();
```

OPcache - JIT - artisan optimize : trois caches en amont

PHP recompile, Laravel re-résout : trois caches mémoire qui transforment votre serveur en bête de course

Sans ces caches

À chaque requête, PHP relit et recompile 10 000+ fichiers (vendor + app). Laravel résout les configs, scanne les routes, charge les events. ~100 à 300 ms de pure mécanique avant le moindre code métier.

OPcache

Cache des opcodes PHP

PHP compile chaque fichier en opcodes à la première exécution, puis sert depuis la mémoire partagée.

Gain - 5-10x sur le bootstrap.

JIT tracing

Compilation native, depuis PHP 8

Détecte les opcodes « chauds » et les compile en code machine. Gain principal sur le calcul intensif, marginal sur le workflow web.

Gain - Significatif sur math, image, crypto. Marginal sur CRUD.

artisan optimize

Pré-cache Laravel

Cache la config, les routes, les events à la compilation. Évite la résolution dynamique à chaque requête.

Gain - 10-30 ms par requête. Obligatoire en prod.

Configuration recommandée - php.ini

```
opcache.enable=1           ; activé
opcache.memory_consumption=256 ; MB partagés
opcache.max_accelerated_files=20000 ; max de fichiers cachés
opcache.validate_timestamps=0 ; en prod, pas de re-check mtime
opcache.jit=tracing         ; opcache.jit_buffer_size=128M
```

OPcache + JIT tracing + artisan optimize

Trois couches de compilation : bytecode persistant, JIT à chaud, Laravel pré-compilé

```
; /usr/local/etc/php/conf.d/zz-resonance.ini
opcache.enable=1
opcache.enable_cli=1
opcache.memory_consumption=256
opcache.jit=tracing
opcache.jit_buffer_size=64M
opcache.validate_timestamps=1
opcache.revalidate_freq=2
```

Effet composé

- Bytecode compilé une fois, partagé entre workers
- JIT tracing : hot paths compilés en code natif au fil
- enable_cli=1 : workers Horizon en profitent aussi
- Synergie avec Octane : le bytecode ne se purge plus

php artisan optimize - appelé avant octane:start dans le CMD

- Chaîne config:cache, route:cache, view:cache, event:cache
- Écrit dans bootstrap/cache/ - persiste via le bind-mount
- Libère le worker Octane des coûts de bootstrap au cold start (~ 100-200 ms)

Pagination cursor : pas par hygiène, par douleur

On pagine quand le volume sature le réseau ou bloque le rendu, non comme bonne pratique universelle

On pagine /api/v1/events

- Starter : 1 500 events publiés, payload 4 Mo
- Plombait le LCP home (SSR Nuxt attend la réponse complète)
- cursorPaginate(20) : payload ÷ 50 → 80 Ko
- Bénéfice secondaire : cache Redis viable (clé bornée par cursor)

On NE pagine PAS ici

- /me/tickets : < 30 tickets par user, pas de douleur
- /organizer/events : < 100 events par organizer
- /organizer/.../participants : douleur réelle (5k+) mais résolue par virtualisation J2-dashboard + index J3-postgres

```
$paginator = $query->orderBy('published_at', 'desc')->orderBy('id', 'desc')->cursorPaginate(20);  
return response()->json(['data' => ..., 'meta' => [  
    'next_cursor' => $paginator->nextCursor()->encode(), 'per_page' => $paginator->perPage()]]);
```

La pagination a un coût : front à adapter, méta-données à exposer, cursor à gérer. Introduire sans douleur mesurable = anti-pattern.

Levier 4

Approche Asynchrone

Sortir le PDF dompdf et le SMTP du thread HTTP : queues Redis + Horizon

Le tunnel d'achat synchrone : ce qu'il fait pendant 1.39 s

OrderController@store starter : cinq blocs séquentiels, dont trois bloquants évitables

Validation + transaction	Insert order, tickets, holders : ~ 50 ms	<i>Indispensable</i>
Mock paiement	sleep(rand(800, 1500) * 1000) : ~ 800-1500 ms	<i>Reste sync (métier)</i>
Génération PDF dompdf	200-500 ms × N tickets : bloque le thread HTTP	→ <i>Job ShouldQueue</i>
Envoi SMTP Mailpit	200-400 ms : SMTP synchrone, encore bloquant	→ <i>Job ShouldQueue</i>
Response 201 JSON	Retour client	<i>Inévitable</i>

Objectif - retirer PDF + SMTP du chemin HTTP. Le mock paiement reste sync (bottleneck métier authentique).

Horizon : superviser les queues Laravel + Redis

Le dashboard et la configuration centralisée pour les workers de jobs asynchrones. Métriques, balance, retry

Sans Horizon

php artisan queue:work en boucle dans Supervisor. Pas de dashboard, pas de tag, pas de métrique de throughput, pas de balance entre queues. Combien de jobs en attente ? Lequel a échoué ? Personne ne sait.

Dashboard temps réel

Throughput, wait time, jobs en cours, failed jobs. Une seule URL, accès auth Gate.

Tags & monitoring

`$job->tags = ['user:42', 'invoice']`.
Recherche, filtre, alertes par tag.

Balance strategies

simple, auto, false. Auto redistribue les workers selon la charge par queue en temps réel.

Retry & failed handling

Backoff configurable, retry depuis le dashboard, `$job->failed()` hook pour les notifications.

Configuration type - config/horizon.php

```
'environments' => [
    'production' => ['supervisor-1' => [
        'connection' => 'redis',
        'queue'      => ['critical', 'default', 'low'],
        'balance'    => 'auto',      // workers redistribués selon charge
        'maxProcesses' => 10,
    ]],
],
```

Jobs ShouldQueue + Horizon : deux dispatchs

Code minimal pour basculer : QUEUE_CONNECTION=redis dans .env, supervisor Horizon dans compose

```
// app/Jobs/GenerateTicketPdfJob.php - implements ShouldQueue
public function handle(TicketPdfService $pdf): void { $pdf->generate($this->ticket); }

// app/Jobs/SendOrderConfirmationEmailJob.php - implements ShouldQueue
public function handle(): void { Mail::to($this->email)->send(new OrderConfirmationMail($this->order)); }
```

```
// OrderController@store - dispatch au lieu d'exécution synchrone
foreach ($order->tickets as $ticket) GenerateTicketPdfJob::dispatch($ticket);
SendOrderConfirmationEmailJob::dispatch($order, $user->email);
```

Horizon : composer require + horizon:install + service docker dédié, gate ouvert en local. Assets inline (5.46), pas de location séparée pour /vendor/horizon/.*

Ce qui reste synchrone, et pourquoi

Le mock paiement n'est pas un défaut de conception, c'est un bottleneck métier à respecter

Mock paiement

`sleep(rand(800, 1500) * 1000)` : simule la latence d'un PSP réel sur le path critique. Retirer ce délai serait irréaliste.

Ce que J3-laravel résout sur le tunnel

- Tunnel médian succès : 1.39 s $\rightarrow$ 1.17 s (-16%)
- Throughput global : $\sim 94/s \rightarrow \sim 644/s$ ($\times 6.8$)
- PDF + SMTP en arrière-plan : visibles dans Mailpit et Horizon dashboard

Ce que ça ne résout pas

- `failed_rate` sous charge : toujours plafonné par l'absence de verrou Postgres
- Hors périmètre : `SELECT FOR UPDATE SKIP LOCKED` $\rightarrow$ itération concurrence dédiée
- Repris dans la section 5 : cas d'école métrique pourrie

Leviers 5 et 6

Runtime PHP

Sortir du cycle FPM par requête : Octane + FrankenPHP, OPcache + JIT

Runtime PHP : trois modèles de cycle de vie

PHP-FPM réinitialise tout à chaque requête. Octane et FrankenPHP gardent le framework en mémoire entre requêtes

Le problème - le coût du bootstrap

À chaque requête, FPM charge l'autoload, initialise le container, enregistre les service providers, parse les routes, c'est 50 à 200 ms de bootstrap. Sur un endpoint simple, le bootstrap domine le temps de réponse.

PHP-FPM

Le standard historique

Modèle

1 process = 1 requête.
Fork à la réception, mort à la réponse.

✓ Isolation totale (rien ne fuit)
Debug simple
Mature, partout supporté

✗ Bootstrap × N requêtes
Latence baseline 50-200 ms
TTFB plancher élevé

Octane

Swoole / RoadRunner

Modèle

Workers persistants.
Framework chargé 1 fois par worker, requêtes traitées en boucle.

✓ 5-10× plus rapide que FPM
Mémoire amortie
Support WebSockets (Swoole)

✗ Memory leaks visibles
State global = bug
Providers à recharger manuellement

FrankenPHP

Serveur HTTP Go + PHP

Modèle

Caddy + PHP embarqué.
Mode worker comme Octane. Single binary auto-hébergé.

✓ HTTP/3 + early hints natifs
Déploiement simple (1 binaire)
Mercure / 103 Early Hints

✗ Écosystème plus jeune
Mêmes discipline que Octane
Documentation en cours

Octane et FrankenPHP exigent une discipline - pas de state global muté, pas de cache statique non vidé, providers à reset entre requêtes.

FPM vs Octane : un cycle vs un worker

Le mode FastCGI hérité : rebootstrap Laravel à chaque requête

PHP-FPM (starter)

1. Nginx → FastCGI
2. Worker FPM fork
3. Bootstrap Laravel complet
4. Routes, providers, container
5. Handle request
6. Worker mort, mémoire purgée

Bootstrap répété : ~ 50-150 ms par requête

Octane + FrankenPHP

1. Démarrage : bootstrap une fois
2. N workers persistants en mémoire
3. Routes/providers/container vivants
4. Requête : juste handle
5. Worker réutilisé
6. Recyclage après --max-requests

Bootstrap amorti : ~ 0 ms par requête

C'est ce changement qui fait passer le TTFB API de 135 ms (médian) à 3 ms.

Setup Octane + FrankenPHP - image et CMD

Image dunglas/frankenphp:latest-php8.3 - Debian 12 slim, PHP 8.3, frankenphp embarqué

```
docker compose exec -u app backend composer require laravel/octane
docker compose exec -u app backend php artisan octane:install --server=frankenphp
```

```
# infra/docker/backend.Dockerfile
FROM dunglas/frankenphp:latest-php8.3 AS base
RUN install-php-extensions pdo_pgsql redis intl gd zip pcntl bcmath opcache

CMD ["sh", "-c", "php artisan optimize && exec php artisan octane:start \
  --server=frankenphp --host=0.0.0.0 --port=8000 --workers=auto --max-requests=500"]
```

Nginx - de fastcgi_pass à proxy_pass keepalive

Octane expose HTTP/1.1 sur :8000 - on parle HTTP, plus FastCGI

```
# infra/nginx/sites/resonance.conf
upstream backend_octane {
    server backend:8000;
    keepalive 32;          # économise le 3-way handshake
}

location /api/ {
    proxy_pass http://backend_octane;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
}
```

Piège à éviter

Si vous oubliez de bouger fastcgi_pass et que vous laissez un upstream backend_fpm orphelin, Nginx invoque encore FPM (mort) et vous obtenez un 502 silencieux. À vérifier dans nginx.conf ET sites/resonance.conf.

Piège Octane : l'état partagé entre requêtes

Le worker reste vivant, tout singleton qui mute son état fuit d'un user à l'autre

À chercher dans le code

- Services injectés qui gardent une référence à Request
- Services qui gardent une référence à User ou Auth
- Singletons applicatifs qui accumulent des données
- Properties statiques mutées par requête

Les filets de sécurité

- --max-requests=500 : recycle le worker régulièrement
- Aide à débuser les fuites mémoire sans attendre un crash en prod
- OctaneServiceProvider : hook RequestTerminated pour purger les états
- Tests E2E croisés multi-users sur la même instance

Octane récompense la discipline de programmation : un service stateless reste stateless, juste plus rapide.

Octane quirks ops : premier boot et dev vs prod

Deux détails opérationnels à expliquer en revue avant de pousser en prod

Premier démarrage

- FrankenPHP embarqué dans dunglas/frankenphp 1.1.5 jugé incompatible par Octane
- Octane télécharge une version compatible dans /app/frankenphp
- ~ 50 Mo, ~ 5 s, persistant via bind-mount
- Démarrages suivants instantanés

validate_timestamps - dev vs prod

- Dev : =1 + revalidate_freq=2 → modifs source détectées toutes les 2 s
- Prod : =0 + opcache_reset() au déploiement atomique
- Pourquoi : le check stat() par fichier devient mesurable en charge
- Posez-vous la question au moment où vous écrivez le CI/CD, pas en debug le jour J

Et les seeders ? resonance:seed, dump-database, restore-database fonctionnent via docker exec, ce sont des process séparés du worker Octane.

Mesure

Lire les chiffres

Pas tous les indicateurs sont valides : un % peut mentir, un anti-pattern guette

Cas d'école : la métrique en pourcentage qui ment

k6 checkout failed_rate : 90.8 % → 98.66 % - apparente régression, en réalité un succès

Mesure	Starter	J3-laravel	Lecture
Throughput tunnel	~ 94 req/s	~ 644 req/s	×6.8
Succès absolus (201)	~ 904	~ 904	identique
Quota places event 600	~ 904	~ 904	fixe
Tentatives totales	~ 9 800	~ 67 600	×7
failed_rate (= 422 / total)	90.8 %	98.66 %	trompeur

Le bottleneck n'est pas Laravel : c'est l'absence de SELECT FOR UPDATE SKIP LOCKED sur ticket_categories.sold. Octane absorbe ×7 le RPS, le quota fini se vide proportionnellement plus vite. Le nombre de succès reste identique.

Sessions Octane - pourquoi PAS Redis

Redis est en place pour cache + queues, donc on bascule les sessions aussi ? Non.

Le réflexe à éviter

- « Tant qu'on y est, on passe SESSION_DRIVER=redis »
- Changement de driver : périmètre supplémentaire
- Aucun bénéfice mesurable à exhiber en revue
- Risque : invalidation, sticky-session, debugging

Ce que les chiffres disent

- Sessions DB Postgres avec PK index : < 1 ms par requête
- Octane n'introduit aucune friction sur sessions DB
- Volume Resonance : largement absorbé par la DB
- Le path critique n'attend pas la session

Règle générale

Optimiser ce qui n'est pas mesurable est un anti-pattern de la performance. Le choix Redis vs DB pour les sessions dépend des contraintes prod (volume connexions, durée vie, coût lecture). Pas une optim universelle.

Lab

Mise en œuvre

Six étapes, 90-120 min - branche `solution/j3-laravel`, fiche docs/ateliers/j3-laravel.md

Les six étapes du lab

Ordre suggéré : eager + cache d'abord (gains immédiats), Octane en dernier (refonte image)

- | | | |
|---|--|--------------------------------|
| 1 | Eager loading + Resources whenLoaded()
Controllers \$with([...]), Resources whenLoaded() | EventController, *Resource.php |
| 2 | Cache Redis applicatif avec tags
CACHE_STORE=redis, Cache::tags(['events']->remember | Controllers index/show/stats |
| 3 | Pagination cursor sur /events
cursorPaginate(20) + meta.next_cursor/prev_cursor | EventController + Nuxt front |
| 4 | Queues Redis + Horizon
QUEUE_CONNECTION=redis, 2 Jobs ShouldQueue, service | Jobs/, docker-compose.yml |
| 5 | Octane + FrankenPHP
octane:install + Dockerfile + Nginx proxy keepalive | backend.Dockerfile, nginx/* |
| 6 | OPcache + JIT + artisan optimize
zz-resonance.ini + CMD chained optimize | backend.Dockerfile |

Les marqueurs @perf-debt en starter pointent vers les zones à traiter - à convertir en @perf-fix après l'optim.

Vérifications : smoke tests

Six checkpoints : query count, TTFB cache hit, checkout, Mailpit, Horizon, benchmarks

1

Query count sur /events index

tinker + DB::enableQueryLog() → queries=3

2

TTFB API cache HIT

curl -w %{time_total} × 3 → ~ 4-10 ms par hit

3

Checkout chrono

curl POST /orders → http=201, dur ~ 1.0-1.5 s (mock préservé)

4

Mail asynchrone reçu

GET http://localhost:8025/api/v1/messages → Confirmation de commande #XXXXXX

5

Horizon supervisor up

http://localhost:8081/horizon → jobs/min > 0 après checkout

6

Benchmarks complets

make k6 - make lighthouse → cf. j3-laravel-comparison.md

Cap sur le lab

120 min

Branche

solution/j3-laravel

Fiche

`docs/ateliers/j3-laravel.md`

À débriefer ensuite

- Pourquoi le LCP reste à 10.4 s, et c'est OK ?
- Pourquoi failed_rate « augmente » sans régression ?
- Pourquoi on cache JSON et pas le Paginator ?
- Pourquoi on ne pagine PAS /me/tickets ?

Module 5

PostgreSQL : index, tuning, PgBouncer

Diminuer le coût SQL des hot paths et amortir le bootstrap connexion sous charge

Branche

`solution/j3-postgres`

Janus Academy - Ali Koudri

Transition : reste la requête SQL

Les couches HTTP, payload, Vue et runtime PHP ont été optimisées. Le SQL n'a pas bougé.

J1 - Cache HTTP

Compression, immutable, SWR Nitro, HTTP/2 → TTFB ~ 0 ms cache hit

J2 - Payload

AVIF, fonts self-host, code splitting, lazy hydration → LCP -66 %

J2-dashboard - Vue 3

shallowRef, v-memo, virtualisation, lifecycle → INP < 50 ms

J3-laravel - Backend PHP

Eager, cache Redis, queues, cursor, Octane, OPcache → TTFB API -99 %

J3-postgres - ici

Index GIN tsvector, postgresql.conf tuné, PgBouncer → Seq Scan → Index

Tout ce qui pouvait être effacé côté navigateur et runtime l'a été, reste à attaquer le plan d'exécution SQL.

Le coût d'une connexion PostgreSQL

PG est process-based : une connexion = un process fork. Le démarrage coûte cher en CPU, en mémoire, et en RTT

Anatomie d'une connexion

PG n'a pas de threads - il fork un process par connexion (postmaster → backend worker). Chaque ouverture déclenche une séquence d'étapes mesurables.



Total à 100 ms de latence

≈ 400-500 ms par ouverture, + 10 MB de RAM allouée. Pour 100 workers backend qui se reconnectent à chaque requête → la DB s'écroule avant la moindre query métier.

C'est ce coût qui rend le pooling indispensable. Sans pooler, le plafond max_connections est atteint avant que la perf métier compte.

Observation starter : EXPLAIN ANALYZE

Les 5 requêtes représentatives - dataset canonique (1 500 events, 2 500 sessions, 200 k tickets)

Requête	Plan starter	Durée
Full-text events - 'concert paris'	Seq Scan + ts_match filter	78.3 ms
Listing participants event star top 100	Seq Scan + Hash Join	13.6 ms
Stats organizer revenus 30j	Seq Scan + GroupAggregate	21.7 ms
Events ville + catégorie - 'Paris'	Seq Scan + filter	0.37 ms
Analytics tickets vendus par event	Seq Scan + Sort	26.0 ms

TTFB API médian 135 ms, p95 2.9 s - Home SSR médian 4.4 s, p95 5.3 s - Tunnel throughput ~ 94/s.

Cas d'école : le LCP qui régresse, sans régression

Le point pédagogique central du module : faux négatif maximum en mesure isolée

À garder en tête

Le TTFB API s'effondre (−68 % médian, −69 % p95) MAIS le LCP régresse apparemment de +41 % (home) et +60 % (fiche). Ce n'est pas une régression - c'est le faux négatif systématique.

Ce que J3-postgres résout (vs starter)

- EXPLAIN full-text : 78.3 ms → 0.06 ms (×1262)
- /events p95 : 2.9 s → 890 ms (−69 %)
- Home SSR médian : 4.4 s → 1.78 s (−60 %)
- TTFB Lighthouse : 2.3 s → 1.0 s (−56 %)
- Throughput tunnel : 94/s → 200/s (×2.1)

Ce que la mesure isolée fait remonter

- LCP home : 16.3 s → 23.0 s (+41 %)
- LCP fiche : 3.0 s → 4.8 s (+60 %)
- Cause : image hero JPEG full-size domine plus
- Plus le backend est rapide, plus le bottleneck visible se déplace vers l'image non optimisée
- Résolu par composition j1-cdn-cache + j2-bundle

C'est précisément ce que la note méthodologique veut éviter de prendre pour un échec.

Les trois leviers de J3-postgres

Un sur le plan d'exécution, un sur la configuration moteur, un sur la concurrence

1

Index secondaires + GIN tsvector

9 index ciblés sur les hot paths : Seq Scan → Bitmap Index Scan / Index Scan

2

postgresql.conf tuné pour le dataset

shared_buffers 1 GB, work_mem 16 MB, effective_cache_size 3 GB, random_page_cost 1.1 (SSD)

3

PgBouncer transaction pooling

25 backends partagés entre 4-20 workers FPM : amortit le bootstrap connexion

Aucune modification frontend, runtime ou applicative : périmètre strictement base de données + pooler.

Levier 1

Index

Donner au planner les bons outils : B-tree, partial, GIN tsvector

Seq Scan vs Index Scan : lire un EXPLAIN

Trois plans à reconnaître au premier coup d'œil : rows + cost + buffers comptent

Seq Scan

Postgres parcourt toute la table.

- Coût : $O(N)$ en rows lues
- Acceptable sur petites tables (< quelques milliers de lignes)
- Catastrophique sur 200k tickets sans WHERE indexable
- Signe : Filter: (...) sans Index Cond

Index Scan / Bitmap Index Scan

Postgres utilise un index pour limiter les rows à lire.

- Index Scan : une cond précise, peu de rows attendues
- Bitmap Index Scan : plusieurs cond ou beaucoup de rows
- Coût : $O(\log N)$ en rows lues
- Signe : Index Cond ou Recheck Cond

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) SELECT * FROM events WHERE status='published' ORDER BY published_at DESC LIMIT 20;
```

Cinq types d'index : cinq cas d'usage

« Mettre un index » ne suffit pas, encore faut-il le bon type. Mettre un B-tree sur un tsvector ne sert à rien

Choisir son index

PG propose 5 types d'index courants. Chacun a une géométrie de recherche pour laquelle il excelle, et des opérations qu'il ne peut pas accélérer.

Type	Cas d'usage	Exemple SQL	À éviter
B-tree	=, <, >, BETWEEN, IN, IS NULL, ORDER BY	CREATE INDEX ON users(email)	JSONB, arrays, full-text
Hash	Égalité stricte uniquement	CREATE INDEX ON users USING HASH(id)	Tout sauf =. Peu d'intérêt vs B-tree depuis PG 10.
GIN	Recherche dans collections : tsvector, JSONB, arrays	CREATE INDEX ON docs USING GIN(content_tsv)	Égalité simple (B-tree meilleur)
GiST	Range types, géo (PostGIS), trigrams	CREATE INDEX ON rooms USING GIST(period)	Plus lent que GIN sur full-text classique
BRIN	Très grandes tables ordonnées physiquement (logs, timeseries)	CREATE INDEX ON logs USING BRIN(created_at)	Tables non triées, petites tables

Règle d'or - B-tree par défaut, GIN dès que la colonne contient une « collection » (JSONB, array, tsvector). Le reste est rare.

Les 9 index choisis : un index par hot path

Migration 2026_05_07_140100_add_performance_indexes.php - un index = une intention

Table	Index	Type / intention
events	idx_events_status_published	B-tree partiel WHERE status='published'
events	idx_events_city	B-tree - filtre ville
events	idx_events_category_published	B-tree composite (category, published_at DESC)
events	idx_events_search	GIN tsvector - full-text français
event_sessions	idx_sessions_event_starts_at	B-tree composite - ordre temporel par event
tickets	idx_tickets_session_created	B-tree composite - listing participants
tickets	idx_tickets_order	B-tree - jointure orders → tickets
orders	idx_orders_user_paid (partial)	B-tree partiel WHERE status='paid'
media	idx_media_mediabile	B-tree composite polymorphique

Chaque index est tracé à une requête mesurée du suite EXPLAIN - pas d'index spéculatifs.

Index partiel : WHERE status = 'published'

Un index qui n'indexe qu'une fraction des rows : plus petit, plus efficace, planner-friendly

```
-- migration : un index, deux propriétés  
CREATE INDEX idx_events_status_published  
ON events (status, published_at DESC)  
WHERE status = 'published';
```

Avantages

- Taille : ~ 80 % des rows ont status='published', l'index couvre uniquement celles-là
- Sélectivité : le planner sait que c'est très ciblé
- Cohérent avec les requêtes : Resonance ne liste que published

Quand l'utiliser

- Filtre stable et répété dans les requêtes
- Distribution déséquilibrée, une catégorie domine
- Économie de stockage, pas d'entrée pour les rows non utilisées

GIN tsvector : recherche full-text français

L'index spécialisé pour les opérateurs @@ et to_tsquery : 78.3 ms → 0.06 ms (×1262)

```
-- index sur expression tsvector française
CREATE INDEX idx_events_search
  ON events USING gin (to_tsvector('french', title || ' ' || description));
```

```
// EventController@index - requête côté Laravel
$query->whereRaw(
  "to_tsvector('french', title || ' ' || description) @@ plainto_tsquery('french', ?)",
  [$q],
);
```

Le planner cherche la même expression dans l'index - trouvée → Bitmap Index Scan - vue dans EXPLAIN.

Découverte : l'expression doit être strictement identique

Le planner matche l'expression caractère par caractère : une virgule de plus → Seq Scan

Trois différences invisibles qui désactivent l'index

<code>title description</code>	<code>≠ title ' ' description</code>	<i>espace de séparation</i>
-----------------------------------	--	-----------------------------

<code>to_tsvector('simple', ...)</code>	<code>≠ to_tsvector('french', ...)</code>	<i>config text-search</i>
---	---	---------------------------

<code>to_tsvector(...)</code> (sans config)	<code>≠ to_tsvector('french', ...)</code>	<i>config implicite</i>
---	---	-------------------------

Mitigation : `default_text_search_config = 'pg_catalog.french'` dans `postgresql.conf` ET 'french' explicite dans l'index + la requête.

Quand NE PAS poser d'index

Petites tables : Seq Scan reste compétitif, et le planner a raison de l'ignorer

Mesure

- events : 1 500 rows
- event_sessions : 2 500 rows
- Stats organizer revenus 30j : 21.7 ms starter → 19.9 ms (-8 % seulement)
- Le planner ne change pas de stratégie après création de l'index

Pourquoi le planner a raison

- Seq Scan séquentiel sur 1 500 rows : une page RAM
- Index Scan ajoute du jump de cache + dérèf btree
- Coût overhead > coût lecture séquentielle complète
- Les stats Postgres connaissent la cardinalité - pg_class.reltuples

Leçon générale

Un index n'est pas universellement bon. Le planner Postgres décide. Si EXPLAIN ne change pas après création, c'est que l'index est inutile à cette échelle - envisager de le retirer.

Levier 2

Configuration moteur

Sortir des defaults conservateurs : [postgresql.conf](#) adapté au dataset

postgresql.conf : les paramètres qui comptent

Les defaults sont conservateurs : calibrés pour un Raspberry Pi. Cinq paramètres transforment radicalement les performances

Pourquoi ça compte

postgresql.conf a ~300 paramètres. Les defaults sont volontairement bas (PG tourne sur n'importe quoi). Sur un serveur sérieux, ils sous-utilisent massivement la RAM disponible, le planner choisit des plans dégradés.

Paramètre	Défaut	Recommandé	Ce que ça change
shared_buffers	128 MB	25 % RAM	Cache pages PG en mémoire. Sans, chaque query relit le disque.
effective_cache_size	4 GB	50-75 % RAM	Hint au planner sur la RAM disponible. Influence le choix index vs seq scan.
work_mem	4 MB	10-50 MB	Mémoire par sort / hash. ⚠ multiplié par connexion × opération.
maintenance_work_mem	64 MB	256 MB – 1 GB	VACUUM, CREATE INDEX. Trop bas = maintenance lente.
random_page_cost	4.0	1.1 (SSD)	Coût relatif d'un seek aléatoire vs séquentiel. SSD ≠ HDD.
max_connections	100	≤ 100 + pooler	Plafond brut. Au-delà, pas de scaling - il faut un pooler.

Règle - *postgresql.conf est un fichier qu'on ajuste, pas un fichier qu'on laisse tel quel. Sans calibrage, ton serveur tourne à 30 % de sa capacité.*

Pourquoi `effective_cache_size` décide Index vs Seq

Un paramètre qui n'alloue rien - il influence uniquement le calcul de coût du planner

Le paradoxe

`effective_cache_size` n'alloue PAS de mémoire. C'est un hint : « voici combien de RAM le système (OS + Postgres) a probablement pour cacher des pages ».

Valeur trop basse

- Planner pense que les random I/O coûtent cher
- Préfère Seq Scan pour limiter les random reads
- Index ignorés silencieusement
- EXPLAIN montre Seq Scan + filter

Valeur adaptée (3 GB)

- Planner pense que les pages sont en cache
- Random I/O cheap : Index Scan préféré
- Combiné à `random_page_cost=1.1` (SSD) - cohérent
- EXPLAIN bascule sur Bitmap Index Scan / Index Scan

default_text_search_config : cohérence index ↔ requête

Le réglage qui ferme la porte aux index GIN qui ne s'utilisent jamais

```
# infra/postgres/postgresql.conf
default_text_search_config = 'pg_catalog.french'
```

Pourquoi le poser explicitement

- Sans : `to_tsvector(text)` utilise la config courante de la session, qui dépend du locale
- L'expression de l'index doit matcher caractère par caractère celle de la requête
- En fixant 'french' dans `postgresql.conf`, on ferme une porte d'inconsistance silencieuse

Discipline complémentaire

Même quand `default_text_search_config` est posé, on garde 'french' explicite dans l'index ET dans la requête - ceinture + bretelles. Une migration future qui change la config ne casse pas l'index.

Levier 3

PgBouncer

Transaction pooling : partager 25 backends entre N workers FPM

Pooler de connexions : mutualiser ce qui coûte cher

Au lieu que chaque worker backend ouvre sa propre connexion PG, ils empruntent à un pool partagé géré par un middleware

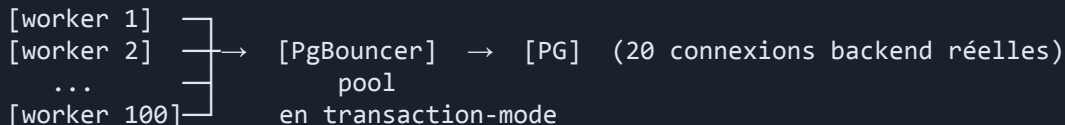
✗ Sans pooler - 1 worker = 1 connexion PG

100 workers Octane → 100 connexions PG ouvertes en permanence.
100 × 10 MB = 1 GB RAM côté PG, juste pour les connexions.
Au pic, max_connections = 100 atteint, refus de connexion.
Nouveau worker → ouvrir/fermer en boucle = coût × N requêtes.

✓ Avec pooler - N workers, M < N connexions PG

100 workers → pooler → 20 connexions PG mutualisées.
Les workers se connectent au pooler (gratuit, local).
Le pooler emprunte une connexion PG le temps d'une transaction.
Mutualisation 5:1, 10:1, parfois 50:1 selon la charge.

Architecture résultante



Le pooler n'est pas un cache, c'est un multiplexeur. Il découple le nombre de clients et le nombre de connexions DB physiques.

Outils - PgBouncer (le standard, C, léger, single-threaded) - PgCat (Rust, multi-threaded, replicas) - Odyssey (Yandex)

Pourquoi un pooler : l'arithmétique des connexions

Sans pooler : $N \text{ workers FPM} \times M \text{ requêtes} = \text{explosion de bootstraps connexion}$

Sans PgBouncer

- Chaque worker FPM ouvre sa propre connexion à Postgres
- Pool dynamic 4-20 workers, jusqu'à 20 connexions actives
- Bootstrap par connexion : handshake SCRAM, charge config
- Sous charge, Postgres passe du temps en handshake plutôt qu'en query
- Connexions persistantes côté FPM possibles, mais limitées

Avec PgBouncer transaction pooling

- `default_pool_size = 25` : 25 backends Postgres réutilisés
- `max_client_conn = 100` : 100 clients FPM acceptés en parallèle
- Backend libéré au COMMIT/ROLLBACK : rendu au pool
- Bootstrap connexion amorti : une fois par backend, pas par requête
- Mesure `pg_stat_activity` : ~ 7 connexions actives sous charge k6

Différentiel mesuré : k6 home SSR médian 4.4 s → 1.78 s, tunnel throughput 94/s → 200/s.

PgBouncer : trois modes, trois compromis

Quand le pooler libère-t-il une connexion PG ? Le mode détermine le ratio de mutualisation et les patterns SQL compatibles

Session

Pool par session client

Connexion PG attribuée pour toute la durée du client.

✓ Tout marche - prepared statements, SET LOCAL, advisory locks, LISTEN/NOTIFY

✗ Ratio worst case - 1:1 si tous les clients tiennent la connexion ouverte

*Sûr mais peu de gains.
À choisir si on a un doute.*

Transaction

Pool par transaction

Connexion libérée après chaque COMMIT / ROLLBACK.

✓ Ratio élevé - 5:1, 10:1 typique - Le mode standard production

✗ Casse - SET LOCAL hors transaction, prepared statements avant PG14 / PgBouncer 1.21

*Le bon défaut en prod.
90 % des cas.*

Statement

Pool par statement

Connexion libérée après chaque requête.

✓ Ratio maximal - 20:1, 50:1 - pas de transaction tenable

✗ Casse - toute transaction multi-statement, cursors, savepoints, prepared

*Très rare.
Reporting analytique pur.*

Le mode transaction couvre 90 % des charges Laravel. Vérifier deux points avant de l'activer : SET LOCAL et prepared statements (cf. slides suivantes).

Transaction pooling : ce qu'il interdit

Le backend change entre transactions : rien ne survit au COMMIT/ROLLBACK côté pool

```
# infra/pgbouncer/pgbouncer.ini
pool_mode = transaction
default_pool_size = 25
```

Ce qui casse en transaction pooling

- Prepared statements server-side qui survivent au COMMIT : la prochaine requête tombe sur un backend qui n'a jamais vu cette PREPARE
- Sessions persistantes : SET search_path, SET timezone, advisory locks - perdus au retour au pool
- LISTEN / NOTIFY : besoin d'une connexion stable, incompatible avec rotation
- Cursors WITH HOLD : curseurs qui survivent à une transaction - perdus
- Le code applicatif doit traiter chaque transaction comme isolée

Prepared statements + PDO : parser une fois, exécuter N

Chaque query SQL passe par *parse* → *plan* → *execute*. Pour les queries répétitives, on cache le *parse/plan* côté PG

Le problème

PG *parse*, *analyse*, *planifie* chaque requête. Pour la même query exécutée 1000 fois avec des paramètres différents, c'est 1000 fois le même travail.

✗ Query inline - *parse* + *plan* à chaque appel

```
foreach ($ids as $id) {  
    $row = DB::select(  
        "SELECT * FROM users " .  
        "WHERE id = $id"  
    );  
    // ⚠ injection SQL  
    // ⚠ parse + plan × N  
}
```

✓ Prepared + bind - *parse* + *plan* une fois

```
$stmt = $pdo->prepare(  
    'SELECT * FROM users WHERE id = ?'  
);  
foreach ($ids as $id) {  
    $stmt->execute([$id]);  
    $row = $stmt->fetch();  
    // ✓ paramètres bindés  
    // ✓ plan caché côté PG  
}
```

Le piège - prepared statements × PgBouncer transaction mode

En transaction mode, chaque transaction peut tomber sur une connexion PG différente. Les prepared statements (caches de plans) ne sont pas portables entre connexions. Avant PG14 + PgBouncer 1.21 → erreur. Depuis : protocol-level prepared, transparent.

```
// config/database.php - désactiver l'émulation PDO pour de vrais prepared côté serveur  
'options' => [PDO::ATTR_EMULATE_PREPARES => false],
```

Piège : prepared statements Laravel + PDO

PDO Postgres utilise des prepared server-side par défaut, cassé en transaction pooling

Symptôme

Erreur erratique sous charge : prepared statement "pdo_stmt_XYZ" does not exist. Le PREPARE a été fait sur un backend, le EXECUTE arrive sur un autre.

```
// backend/config/database.php
'pgsql' => [
    // ... DB_HOST=pgbouncer, DB_PORT=6432
    'options' => extension_loaded('pdo_pgsql') ? [
        PDO::ATTR_EMULATE_PREPARES => true,
    ] : [],
],
```

Effet : PHP substitue les placeholders côté driver avant l'envoi. Aucune trace de PREPARE côté backend Postgres. Sécurité préservée par PDO (échappement strict).

Dualité : pgsql vs pgsql_direct

Deux connexions Laravel : une pour le HTTP via pooler, une pour les DDL via Postgres direct

Connexion	Cible	Usage
pgsql (défaut)	pgbouncer:6432	FPM workers, queue jobs, runtime HTTP
pgsql_direct	postgres:5432	migrations, seeders, dump, restore

Pourquoi cette dualité

- Migrations : DDL longues qui ouvrent une transaction stable, incompatibles avec le transaction pooling
- Seeders : RealisticDatasetSeeder fait des inserts batch 200 k, sessions multi-tx
- Dump / restore : pg_dump et psql utilisent des prepared statements server-side

```
# Makefile
migrate: ; $(COMPOSE) exec -u app backend php artisan migrate --database=pgsql_direct
seed:    ; $(COMPOSE) exec -u app backend php artisan resonance:seed # bascule la default en pgsql_direct
dump:    ; ... --database=pgsql_direct ... --schema=public
```

SCRAM passthrough : l'auth qui traverse le pooler

Sans passthrough, le pooler authentifie tous les clients sous un seul user. Avec, chaque user reste identifié côté PG

Le problème : l'auth perdue à travers le pooler

Configuration classique : les clients s'authentifient sur PgBouncer, qui ouvre les connexions PG sous un user technique. Tous les workers vus comme un seul user côté PG - RLS / audit / GRANT par user perdus.

✗ Sans passthrough

1. Clients s'auth sur PgBouncer (alice, bob, carol).
2. PgBouncer ouvre une connexion PG sous un user technique.
3. Côté PG : tous les workers apparaissent sous le même user.
→ RLS, audit, GRANT par user perdus.

✓ Avec SCRAM passthrough

1. Clients s'auth sur PgBouncer (alice, bob, carol).
2. PgBouncer forward le challenge SCRAM à PG.
3. Côté PG : chaque user reste authentifié individuellement.
→ RLS, audit, GRANT par user fonctionnels.

Configuration - pgbouncer.ini

```
[pgbouncer]
auth_type = scram-sha-256          ; passthrough natif depuis 1.21
auth_user = pgbouncer_introspect    ; user technique pour la lookup
auth_query = SELECT username, passwd FROM pg_shadow WHERE username = $1
```

SCRAM passthrough rend le pooler transparent côté sécurité. Pas un détail d'infra - un prérequis pour toute architecture multi-tenant ou auditée.

SCRAM passthrough : pas de hash committé

PgBouncer doit lire pg_shadow, via une fonction SECURITY DEFINER scoppée au rôle pgbouncer

```
-- infra/postgres/init/01-pgbouncer-auth.sh (exécuté UNE fois au premier initdb)
CREATE ROLE pgbouncer LOGIN PASSWORD :'pgbouncer_password';
CREATE SCHEMA pgbouncer;
CREATE FUNCTION pgbouncer.user_lookup(IN i_username TEXT,
    OUT uname TEXT, OUT phash TEXT) RETURNS RECORD AS $$
BEGIN SELECT username, passwd FROM pg_shadow WHERE username = i_username INTO uname, phash;
    RETURN;
END; $$ LANGUAGE plpgsql SECURITY DEFINER;
```

Trois propriétés

- SECURITY DEFINER : la fonction s'exécute en tant que propriétaire (superuser au bootstrap), pas en tant qu'appelant
- Le rôle pgbouncer n'a pas accès direct à pg_shadow, seulement à la fonction
- Aucun hash committé dans le repo, seul PGBOUNCER_PASSWORD (cleartext) transite via .env, userlist.txt généré par edoburu

Init script Docker, exécuté UNE fois

/docker-entrypoint-initdb.d/ : ne ré-exécute pas après le premier initdb du volume

Le piège

Vous modifiez infra/postgres/init/01-pgbouncer-auth.sh, vous exécutez docker compose restart postgres, et rien ne change. Normal : les scripts ne s'exécutent qu'au premier initdb du volume.

Recréer le volume

- docker compose down -v
- docker compose up -d
- Le script s'exécute sur le nouveau volume
- Perd les données : faire make restore après

Exécuter le SQL à la main

```
docker compose exec postgres \  
psql -U postgres -d resonance \  
-v pgbouncer_password=\   
$PGBOUNCER_PASSWORD \  
-f /docker-entrypoint-initdb.d/...
```

Préserve le volume et les données.

Mesure

Lire les chiffres

Le faux négatif systématique : préparer la prod aussi

Cas d'école : le LCP qui régresse, sans régression

L'exemple le plus pur de faux négatif : une optim qui dévoile le bottleneck suivant

Mesure	Starter	J3-postgres	Lecture
TTFB API médian	135 ms	43 ms	-68 % 
TTFB Lighthouse home	2.3 s	1.0 s	-56 % 
Home SSR médian k6	4.4 s	1.78 s	-60 % 
LCP home Lighthouse	16.3 s	23.0 s	+41 %  apparent
LCP fiche Lighthouse	3.0 s	4.8 s	+60 %  apparent

Diagnostic

Le LCP est dominé par l'image hero JPEG full-size (1920×1080, ~ 290 Ko). Quand le backend était lent, le hero arrivait avec retard derrière un long délai serveur. Maintenant que le backend répond vite, l'image domine le timing. Résolu en composition - j1-cdn-cache (Cache-Control immutable) + j2-bundle (AVIF + IPX).

CREATE INDEX CONCURRENTLY : atelier vs prod

La décision atelier est pédagogique : la décision prod dépend de votre contexte. Trois options.

Le conflit

CREATE INDEX bloque les SELECT/INSERT/UPDATE (ACCESS EXCLUSIVE lock). CONCURRENTLY l'évite, mais exige d'être hors transaction.
Migrations Laravel : par défaut wrap les statements dans une transaction.

- (a) **withinTransaction = false**
public bool \$withinTransaction = false; dans la migration + CREATE INDEX CONCURRENTLY. Une migration Laravel native, mais hors transaction.
- (b) **Migration + script SQL**
Migration Laravel pour le DDL « rapide ». Script SQL séparé pour les index lourds, exécuté manuellement par DBA en fenêtre de maintenance.
- (c) **Migration no-op + DDL manuel**
Migration vide enregistrée dans la table migrations. DDL réel exécuté à la main. php artisan migrate:status vérifie l'état.

Lab

Mise en œuvre

Six étapes, 90-120 min : [branche solution/j3-postgres](#), [fiche docs/ateliers/j3-postgres.md](#)

Les six étapes du lab

Ordre suggéré - index + recherche full-text d'abord (gain visible), PgBouncer en dernier (refonte image)

- | | | |
|---|--|---------------------------|
| 1 | Migration des 9 index secondaires
DB::statement(...) - partial, composite, GIN tsvector | migrations/2026_05_07_... |
| 2 | postgresql.conf tuné + montage volume
1 GB / 3 GB / 16 MB / 1.1 - default_text_search_config | infra/postgres/ |
| 3 | Recherche full-text Laravel
whereRaw to_tsvector @@ plainto_tsquery + 'french' explicite | EventController@index |
| 4 | PgBouncer + init script SCRAM passthrough
pgbouncer.ini transaction + auth_query + edoburu | infra/pgbouncer/ |
| 5 | Dualité pgsqsl / pgsqsl_direct
config/database.php + Makefile + commandes Artisan | config/database.php |
| 6 | Régénération dump SQL - make dump
--database=pgsqsl_direct --schema=public : ~ 11 MiB | infra/seeds-dump/ |

Les marqueurs @perf-debt DB en starter pointent vers les zones à traiter - à convertir en @perf-fix après l'optim.

Vérifications : smoke tests

Six checkpoints : count indexes, pg_settings, EXPLAIN Bitmap, full-text curl, pg_stat_activity, benchmarks

1

Count des index applicatifs

`SELECT count(*) FROM pg_indexes WHERE indexname LIKE 'idx_%' → 10`

2

Paramètres postgresql.conf appliqués

`SELECT name, setting FROM pg_settings WHERE name IN (...) → 131072 / 16384 / 393216 / 1.1`

3

EXPLAIN Bitmap Index Scan

`make explain-no-restore → Bitmap Index Scan on idx_events_search pour (a)`

4

Full-text curl en < 50 ms

`curl /api/v1/events?q=sapiente | jq .data | head`

5

PgBouncer pooling sous charge

`SELECT count(*) FROM pg_stat_activity WHERE datname='resonance' → ~ 7 connexions`

6

Benchmarks complets

`make k6 - make lighthouse - make explain → cf. j3-postgres-comparison.md`

Cap sur le lab

120 min

Branche

solution/j3-postgres

Fiche

docs/ateliers/j3-postgres.md

À débriefer ensuite

- Pourquoi le LCP régresse, et c'est OK ?
- Pourquoi events n'a pas besoin d'index ?
- Pourquoi EMULATE_PREPARES = true ?
- Pourquoi pgsql_direct pour les migrations ?

Synthèse

Le voyage : starter → final

Récap, composition multiplicative, leçons transférables, pour aller plus loin

Branche

`solution/final`

D'où on part : Resonance starter Phase 4-bis

Mode prod-like volontairement non optimisé : Nuxt 4 build prod + PHP-FPM + Nginx sans tuning

0.55

Lighthouse home perf

16.3 s

LCP home

15.9 s

FCP home

9 572 ms

TBT scroll 5 s

5.3 s

k6 home SSR p95

2.9 s

k6 search p95

135 ms

TTFB API médian

90.8 %

k6 checkout failed_rate

Dataset réaliste 200k tickets - N+1 partout, Seq Scans, bootstrap PHP par requête, payload 4 Mo.

Où on arrive : branche final composée

Cinq branches solution mergées - cf. docs/benchmarks/final-comparison.md

Mesure	Starter	Final	Δ	Cible §9
TTFB API médian	135 ms	3 ms	×45	✓
k6 search p95	2.9 s	4.8 ms	×604	✓
k6 home SSR médian	4.4 s	1.3 ms	×3 384	✓
DOM participants	42 062	315	÷133	✓
Lighthouse home perf	0.55	0.89	+0.34	⚠ -1 pp
LCP home	16.3 s	3.29 s	-80 %	⚠ +32 % cible
Tunnel achat médian	3-5 s	1.37 s	-66 %	✗ mock PSP @design

Quatre cibles atteintes, deux quasi, deux hors scope par design (mock PSP, SKIP LOCKED).

La stack finale : vue d'ensemble

Cinq branches mergées sur solution/final - périmètres complémentaires

Browser	Lighthouse, k6, utilisateurs réels
Nginx	J1 - HTTP/2, brotli, Cache-Control immutable, proxy_pass keepalive 32
Nuxt 4 SSR	Octane + FrankenPHP
J1 SWR Nitro 60-300 s - J2-bundle AVIF/WebP + fonts self-host J2-bundle code-split + lazy hydration J2-dashboard shallowRef + v-memo + virtualisation	J3-laravel eager + Resources whenLoaded() Cache Redis tags + Queues Horizon + cursor pagination OPcache + JIT tracing 64 Mo
PgBouncer	J3-postgres - transaction-mode, SCRAM passthrough, 25 backends, dualité postgres/postgresql_direct
Postgres 16	J3-postgres - postgresql.conf tuné + 9 index secondaires + GIN tsvector français

Récap branche par branche

Les 5 branches

Levier - gain isolé - leçon transférable

J1 - cdn-cache : couches HTTP et cache serveur

Cache de réponse côté Node (SWR Nitro) - bénéficie aussi aux clients sans cache (k6, crawler)

Levier

Nginx HTTP/2 + brotli + Cache-Control immutable sur `_nuxt/*` et SWR sur `/media/*`. Nitro routeRules SWR (home 60 s, `/events` 60 s, `/events/**` 300 s, `/organizer/**` ssr off). Middleware Laravel SetEventsCacheControl pour cache HTTP applicatif.

Gain isolé mesuré

- Lighthouse home perf : 0.55 → 0.76
- LCP home (cache HIT) : 16.3 → 5.1 s
- k6 home p95 (cache HIT) : 5.3 s → 2.14 ms

Leçon transférable

Le cache de réponse HTML rendu côté Node (SWR Nitro) bénéficie même à un client sans cache (k6, crawler). C'est différent d'un Cache-Control browser; c'est un cache applicatif serveur que Nitro gère.

J2 - bundle : poids transféré et critical path

Images responsives, fonts self-host, code splitting - gotcha NuxtImg preload piégeux

Levier

@nuxt/image (IPX, AVIF/WebP, srcset width-based), @nuxt/fonts (Inter self-host, font-display swap, preload poids 400), defineAsyncComponent sur Chart.js (-43 % First Load JS organizer), lazy hydration TicketSelector sur fiche événement.

Gain isolé mesuré

- Lighthouse home perf : 0.55 → 0.66
- FCP home : 15.9 → 4.8 s (fin du roundtrip Google Fonts)
- Hero AVIF: -40 % poids, card AVIF : -91 %

Leçon transférable

<NuxtImg preload> est piégeux sur srcset width-based : il preload la plus grande variante (2048w) au lieu de celle choisie par le browser sur mobile (640w), doublant le poids. Préférer fetchpriority="high".

J2 - dashboard : réactivité Vue et virtualisation

shallowRef ciblés, v-memo, RecycleScroller - DOM 42 062 → 315 nodes (-99.5 %)

Levier

Éclatement du state dashboard en 3 shallowRef ciblés (stats, salesChart, events). v-memo="[ev.id, ev.title, ev.city, ev.status]" sur les rows de la table events. RecycleScroller sur participants. onScopeDispose pour le cleanup du polling 10 s.

Gain isolé mesuré

- INP click searchbox : 104 ms → 24 ms (-77 %)
- TBT scroll 5 s : 9 572 ms → 0 ms
- FPS scroll : 25 → 85 fps

Leçon transférable

La virtualisation devient critique à partir de quelques milliers de lignes, pas à cause du paint (le browser optimise) mais des handlers DOM (click, hover) qui multiplient les listeners. shallowRef suffit pour les arrays remplacés en bloc.

J3 - laravel : backend applicatif

Eager + Resources whenLoaded, cache Redis tags, queues, cursor, Octane, OPcache+JIT

Levier

with([...]) sur tous les controllers + Resources whenLoaded() (49 → 3 SELECTs). Cache::tags(['events'])->remember TTL 60 s liste / 300 s fiche. Queues Redis + Horizon (PDF dompdf + SMTP). cursorPaginate(20) sur /events. Octane + FrankenPHP. OPcache + JIT tracing 64 Mo.

Gain isolé mesuré

- k6 search p95 : 2.9 s → 6.3 ms (-99.8 %)
- TTFB Lighthouse home : 2.3 s → 18 ms
- Tunnel throughput : 94 → 644 req/s (×6.8)

Leçon transférable

Laravel 11+ pose `cache.serializable_classes = false` (gadget chain). Impossible de cacher des objets Eloquent / Paginator / Resource. Cacher la réponse JSON sérialisée (->resolve() ou ->toJson()) puis array natif, pas l'objet brut.

J3 - postgres : base de données et frontal pool

9 index + GIN tsvector + postgresql.conf tuné + PgBouncer transaction pooling

Levier

9 index secondaires (8 B-tree partiels + 1 GIN tsvector français). postgresql.conf tuné (shared_buffers 1 GB, work_mem 16 MB, random_page_cost 1.1). PgBouncer transaction-mode (SCRAM passthrough auth_query, pas de hash committé). Dualité Laravel pgsql / pgsql_direct.

Gain isolé mesuré

- EXPLAIN full-text : 78 ms → 0.06 ms (×1262)
- k6 search p95 (sans Redis) : 2.9 s → 890 ms
- TTFB Lighthouse home : 2.3 s → 1.0 s

Leçon transférable

Index GIN sur expression : l'expression dans la requête doit être strictement identique à celle indexée (même espace !). PgBouncer transaction pooling interdit les prepared cross-transaction - EMULATE_PREPARES + pgsql_direct pour les sessions longues.

Effets multiplicatifs

Composition

Pourquoi le merge final apporte plus que la somme des gains isolés

Composition #1 : Cache Redis × SWR Nitro

Deux caches en cascade : même le 1^{er} MISS dans la fenêtre TTL retombe sur HIT

Sans cache	Home /events fetch /api à chaque visite, 3 SELECTs avec joins, TTFB ~ 50-150 ms	150 ms
j1 seul (SWR Nitro)	HTML rendu en cache, mais le 1 ^{er} MISS paye le TTFB API complet (3 SELECTs)	150 ms
j3-laravel seul (Redis)	Cache Redis HIT à 1-3 ms côté API, mais SSR Nuxt re-render à chaque requête	20 ms
Composé (final)	SWR Nitro cache le HTML, le 1 ^{er} MISS retombe sur Redis HIT bout-en-bout 1-3 ms	2.8 ms

Mesure k6 home p95 : starter 5.3 s → j1 seul 2.14 ms → j3-laravel seul 19.9 ms → final 2.77 ms.

Composition #2 : GIN tsvector × Cache Redis

Le cache absorbe 95 % du trafic, les 5 % qui passent ne pèsent plus rien

Sans composition

- j3-postgres seul : Bitmap Index Scan 0.06 ms - payé sur chaque requête
- j3-laravel seul : Cache Redis 60 s, mais le MISS retombe sur ILIKE Seq Scan 78 ms
- Chaque optimisation paye sur son périmètre, mais n'absorbe pas l'autre

Composé sur final

- Cache Redis (j3-laravel) absorbe les HITs à 1-3 ms (~ 95 % du trafic k6 search)
- Sur les MISS (~ 5 %), la requête tombe sur GIN Index Scan 0.06 ms (j3-postgres) au lieu de Seq Scan 78 ms
- Cache + index travaillent en relais, jamais Seq Scan

Sans GIN, le MISS Redis serait une catastrophe rare. Sans Redis, le GIN serait sollicité 20× plus. La composition rend les deux résilients.

Composition #3 : Octane × PgBouncer

Sans PgBouncer, Octane saturerait max_connections sous burst

L'arithmétique

Octane à 8 workers × 1 connection chacun → 8 connexions actives en idle, plafond max_connections=100 atteint en burst (4 instances × 8 workers = 32 connexions + bootstrap).

Octane apporte

- App en mémoire, workers persistants
- Pas de bootstrap Laravel par requête
- Throughput tunnel ×6.8 (94 → 644 req/s)
- TTFB API médian 135 ms → 3 ms

PgBouncer apporte

- 25 backends Postgres partagés
- pg_stat_activity : ~ 7 connexions actives sous charge
- max_connections n'est plus le bottleneck
- Bootstrap connexion amorti

Cas d'école : composition $\neq$ sommation

Le gain composé est le maximum ou un produit, jamais la somme des gains isolés

Branche	TTFB API médian	Gain	Lecture
Starter	135 ms	-	baseline
j3-postgres seul	43 ms	-92 ms	Seq Scan → Index Scan
j3-laravel seul	3.1 ms	-132 ms	cache Redis HIT
Final composé	3.05 ms	-132 ms	dominé par j3-laravel

Lecture

j3-postgres ne rajoute pas de gain sur le HIT cache j3-laravel - le HIT vaut 1-3 ms quel que soit le coût du MISS sous-jacent. Mais sur les MISS (~ 5 %), j3-postgres divise le MISS par 2-1000× selon la requête. La composition rend l'optimisation résiliente, pas plus rapide en moyenne.

Méthodologie

Leçons transférables

Ce qui reste vrai au-delà de Resonance : cinq disciplines à emporter

Leçon 1 : mesurer avant d'optimiser

Une baseline figée - un protocole reproductible - exclure les optims qui ne mesurent pas

Discipline appliquée sur Resonance

- Lighthouse + k6 + EXPLAIN starter committés dans docs/benchmarks/{lhci,k6,sql}-starter/
- make benchmark : restore + lighthouse + k6 chaînés
- Pre-hook make restore : garantit le même état de données entre runs
- Sans pre-hook : fail_rate 91 % → 97 % entre deux runs successifs

À reprendre en projet réel

- Une baseline figée avant tout chantier perf
- Le protocole de mesure dans le repo (Makefile, scripts)
- Pas de "propreté de code" sans bench, ce n'est pas de la perf
- Chaque PR perf montre son delta

Leçon 2 : différentiel $\neq$ absolu

Une branche solution isolée ne peut pas atteindre les cibles : briefer en différentiel

Le piège systématique

Juger une branche solution isolée sur les cibles produit un faux négatif systématique. Une cible absolue suppose la composition.

Discipline appliquée : chaque fiche d'atelier a 2 encarts

- « Ce que cet atelier améliore (gains différentiels mesurés) » - tableau delta vs starter
- « Ce qui n'est PAS dans le périmètre » - cibles hors d'atteinte, attribuées à la branche qui les apporte
- Cf. docs/architecture.md - Cibles différentielles vs cibles absolues

Leçon 3 : régression apparente $\neq$ régression

Lighthouse $\times$ throttling $\times$ composition : un faux négatif courant qui disparaît au merge

Ce qui s'est passé sur Resonance

- j3-postgres seul : LCP home +41 %, LCP fiche +60 % vs starter
- j3-laravel seul : même schéma sur certaines métriques visuelles
- Cause : composition Web Vitals $\times$ throttling Slow 4G / 4 $\times$ CPU. Plus le backend est rapide, plus l'image hero non optimisée domine le timing
- Sur final : LCP home 16.3 $\rightarrow$ 3.29 s (−80 %), score 0.55 $\rightarrow$ 0.89 - les régressions s'inversent

Comment réagir

- Vérifier que le gain différentiel attendu (TTFB API, etc.) est bien là
- Documenter la régression apparente avec sa cause attendue
- Ne pas chercher à "corriger" la régression apparente sur la branche isolée
- Vérifier la levée sur final

Leçon 4 : latence métier vs dette technique

Le mock PSP préservé en @design : pas une dette, un bottleneck incompressible

PaymentMockService::process

`usleep(rand(800, 1500) * 1000)` : simule un acquittement PSP synchrone. Préservé @design jusqu'en final - un PSP réel a la même latence incompressible.

Ce qui justifie cette préservation

- Pas une dette technique mais un comportement métier réaliste
- Le tunnel ne peut pas répondre 201 avant l'acquittement PSP
- Tordre le mock pour cocher la cible perd la valeur pédagogique

Ce que ça justifie : l'usage des queues

- Tout ce qui n'a pas besoin d'être bloquant passe en queue Redis (j3-laravel)
- PDF dompdf + SMTP confirmation déportés du thread HTTP
- Le client reçoit le 201 quand le PSP acquitte, pas après le PDF
- Tunnel : 3-5 s starter → 1.37 s final

Leçon 5 : marquer la dette dans le code

@perf-debt - @perf-fix - @design - trois marqueurs traçables au grep

@perf-debt

Optimisation volontairement omise dans le starter. Le code marche, juste lent. Indique la zone à traiter dans le lab.

@perf-fix

Optimisation appliquée dans une branche solution. @perf-debt converti en @perf-fix après l'optim, avec référence à la mesure.

@design

Comportement préservé volontairement, même si « lent ». Mock PSP, latence métier authentique. Ne PAS toucher.

grep -rn "@perf-debt" backend/ frontend/ : reste la source de vérité de l'état du voyage.

Vécu

Pièges au merge final

Trois fix(merge) qu'on n'aurait pas anticipés en lisant les fiches

Fix #1 : postgresql-client v15 vs v16

Changer de base image système peut casser silencieusement des dépendances natives

Symptôme

Sur final, make dump abort. pg_dump côté backend = v15, Postgres serveur = v16 - version mismatch.

Cause

j3-laravel switch PHP-FPM-alpine (qui shippe postgresql-client v16 via apk) vers FrankenPHP Debian 12 (qui ne shippe que v15). j3-postgres exige v16. Le merge change la base image sans que personne ne le remarque.

Fix + leçon

Ajout dépôt apt PostgreSQL officiel + postgresql-client-16 dans backend.Dockerfile. Leçon : tester make dump + make restore après tout changement de base image système - un changement perf peut casser un workflow ops invisible.

Fix #2 : EventResource nested non-résolues

->resolve() ne descend qu'au premier niveau - toJson() résout récursivement

Symptôme

SSR Nuxt crashe avec `s.ticket_categories.reduce` is not a function après le 1^{er} MISS Redis. En dev tout marche, en cache rechargé tout casse.

Cause

Cache stocke `(new EventResource($event))->resolve()` qui retourne un array contenant des `EventSessionResource::collection(...)` NON résolues. Au reload, Laravel 11+ unserialize avec `allowed_classes=false` - les Resource nested deviennent `__PHP_Incomplete_Class`.

Fix + leçon

`json_decode((new EventResource($event))->toJson(), true)` pour matérialiser la sérialisation récursive avant cache. Leçon : `cache.serializable_classes=false` est un défaut sécurité qu'on ne désactive pas, toute valeur cachée doit être un arbre de scalaires + arrays natifs, jamais d'objet PHP même de classe connue.

Fix #3 : template vs script après merge auto

Le merge git valide la syntaxe, pas la sémantique : refactor d'identifiant + template modifié

Symptôme

Build et type-check passent. SSR organizer/dashboard crashe au render - dashboardState is undefined. Aucune erreur côté build.

Cause

j2-bundle remplace `<canvas>` inline par `<SalesChart :points="dashboardState.salesChart" />`. j2-dashboard refactor dashboardState en 3 refs ciblés (stats, salesChart, events). Merge auto = template j2-bundle (vieux nom) + script j2-dashboard (plus de dashboardState).

Fix + leçon

`:points="dashboardState.salesChart"` → `:points="salesChart"`. Leçon : un refactor qui renomme un identifiant côté script ne se propage pas automatiquement côté template si la branche cible avait modifié le template avant le renommage. Tester le rendu réel après merge, pas seulement la compilation.

Ouverture

Pour aller plus loin

Cibles partielles : atelier concurrence - Phase 6 documentation

Cibles : état honnête

Quatre atteintes, deux quasi, deux hors scope par design - pas de mesure tordue

✓ Atteintes

- TTFB API médian < 50 ms → 3 ms
- k6 search p95 < 300 ms → 4.8 ms
- k6 home SSR médian < 100 ms → 1.3 ms
- DOM participants < 1000 nodes → 315

⚠ Quasi-atteintes

- Lighthouse home perf $\geq 0.90 \rightarrow 0.89$ (~1 pp)
- LCP home < 2.5 s → 3.29 s (+32 % cible)
- Levier : redimensionner source hero (1920 → 1280) au build
- Ou edge CDN : Cloudflare Images, ImageKit

✗ Hors scope par design

- Tunnel checkout médian < 500 ms : impossible sans changer le mock PSP, contraire au @design
- Tunnel checkout 422 failed_rate < 5 % : nécessite SELECT FOR UPDATE SKIP LOCKED sur ticket_categories.sold, hors scope Phase 5 (perf, pas correctness sous concurrence)
- Honnêteté pédagogique : le repo n'a PAS essayé d'atteindre ces cibles par contournement, faute de temps et parce que les objectifs pédagogiques sont atteints.

Atelier futur : j3-concurrence

Le levier manquant : sérialiser les acquittements de quota sans perdre le throughput

Le problème à résoudre

failed_rate k6 checkout 90.8 % → 98.75 % en final. Le nombre absolu de succès (~ 904) reste identique : le quota fixe est saturé en quelques secondes, final l'absorbe juste plus vite (×7.3 throughput). Sans verrou, on ne peut pas augmenter le succès.

1.

Verrou ligne pessimiste

SELECT ... FOR UPDATE SKIP LOCKED sur ticket_categories.sold pour sérialiser les acquittements

2.

File d'attente "réservation"

Redis sorted set TTL 5 min - le client réserve un slot avant d'engager le PSP

3.

Pattern Stripe-like

PaymentIntent réserve le slot, confirmation du paiement = consommation du slot

Phase 6 : documentation et annexe PSP

Finalisation du repo de formation : README stagiaire + remplacement du mock paiement

README stagiaire

- Parcours guidé sur 3 jours
- Pré-requis machine et environnement
- Ordre suggéré des branches solution
- Liens vers les fiches d'ateliers
- FAQ et troubleshooting

Annexe Stripe

- Remplacement du PaymentMockService par un vrai PSP
- Stripe Checkout ou PaymentIntents
- Webhooks pour la confirmation asynchrone
- Idempotency keys
- Pose une vraie contrainte de latence (300-800 ms)

Ce que ça donne pédagogiquement

Une formation autonome : le stagiaire récupère le repo, suit le README, joue les 5 ateliers dans l'ordre, vérifie ses mesures contre les baselines committées. L'annexe Stripe transforme le mock @design en bottleneck réel mesurable, passe la main aux patterns webhook + idempotency : utile si la formation est rejouée sur un environnement avec PSP réel.

Merci

Trois jours, six modules, cinq branches solution et une synthèse

À débriefer ensemble

- Quelle leçon vous emportez en projet réel ?
- Sur quelle branche revenir en autonomie ?
- Quel pattern vous a le plus surpris ?
- Et le LCP qui régresse sur les branches isolées ?

Repo de référence

Branche - **solution/final**

`docs/architecture.md`

`docs/benchmarks/`
`final-comparison.md`

`docs/ateliers/j{1,2,3}-*.md`

Janus Academy - Ali Koudri